Solana JIT

Lessons learned from fuzzing a smart-contract compiler

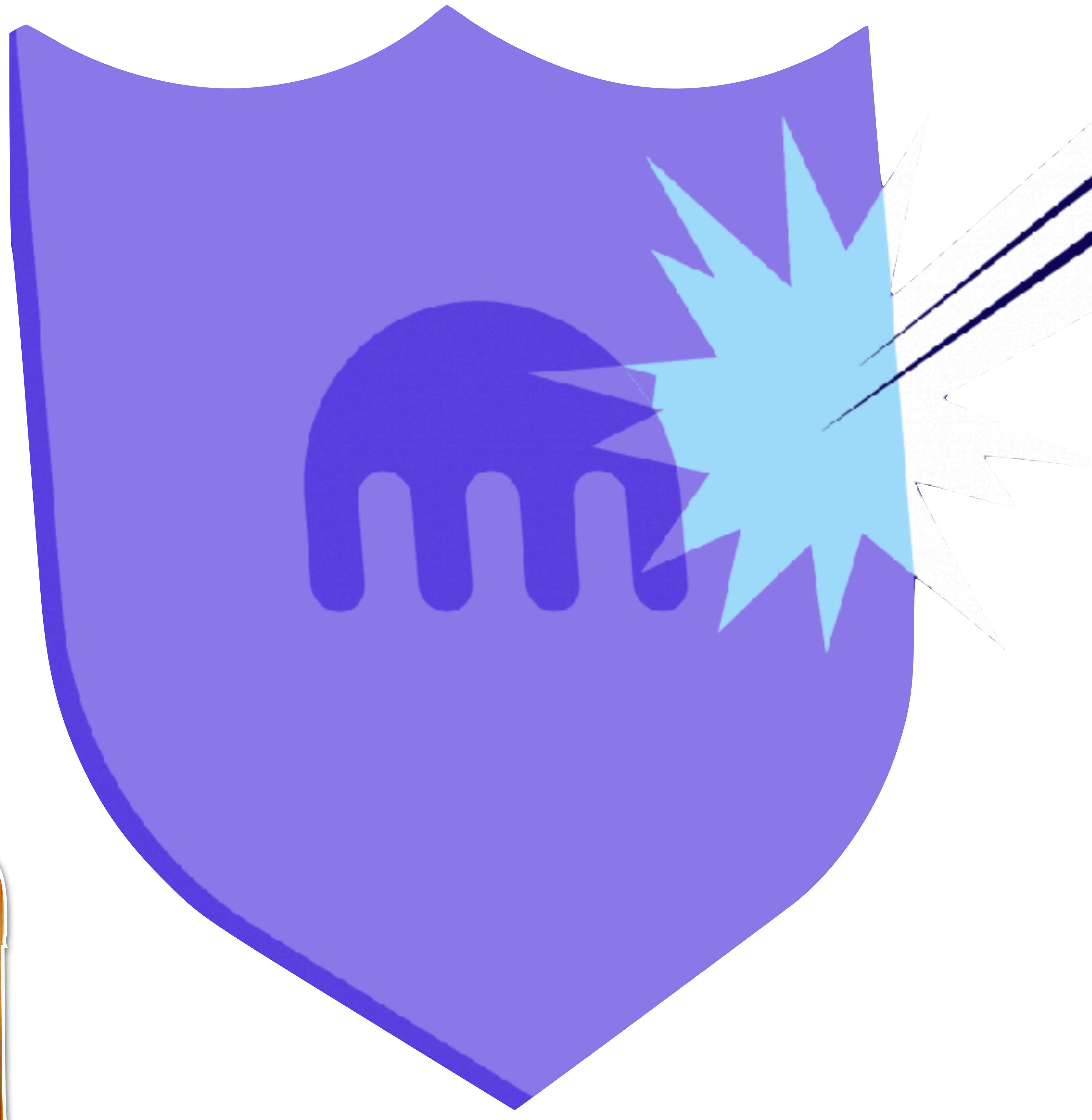
DEF CON 30

kraken

Kraken Security Labs

Thomas Roth

- Security research division of Kraken



kraken

Thanks!

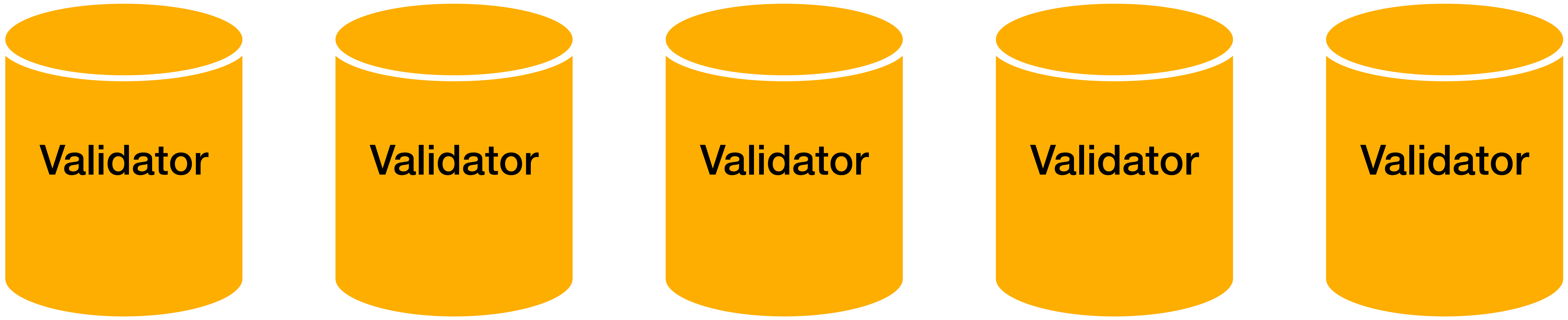
- The Solana team
- secret.club's
- The AFL++ team

Solana

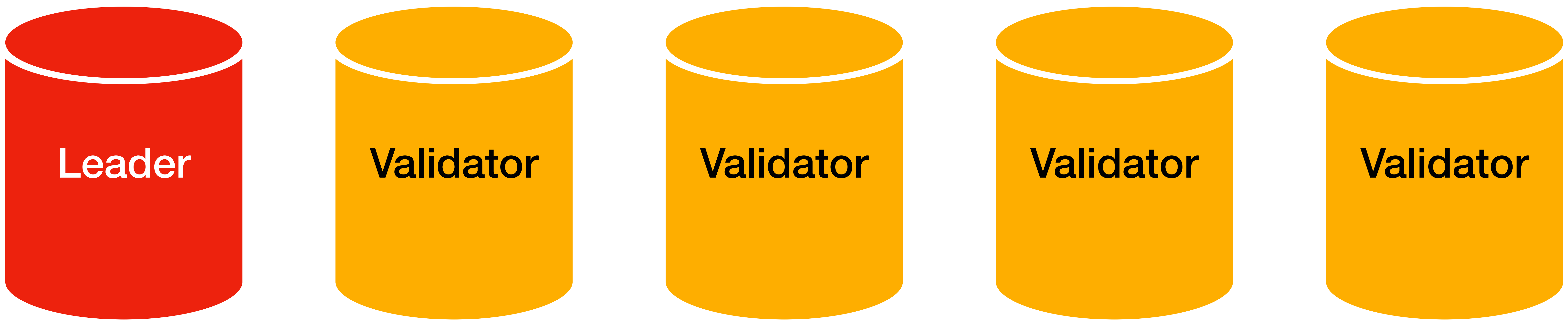
- "High performance" blockchain
- Runs smart-contracts
  - That are written in C/C++/Rust
- Proof-of-stake / proof-of-history

kraken

Solana

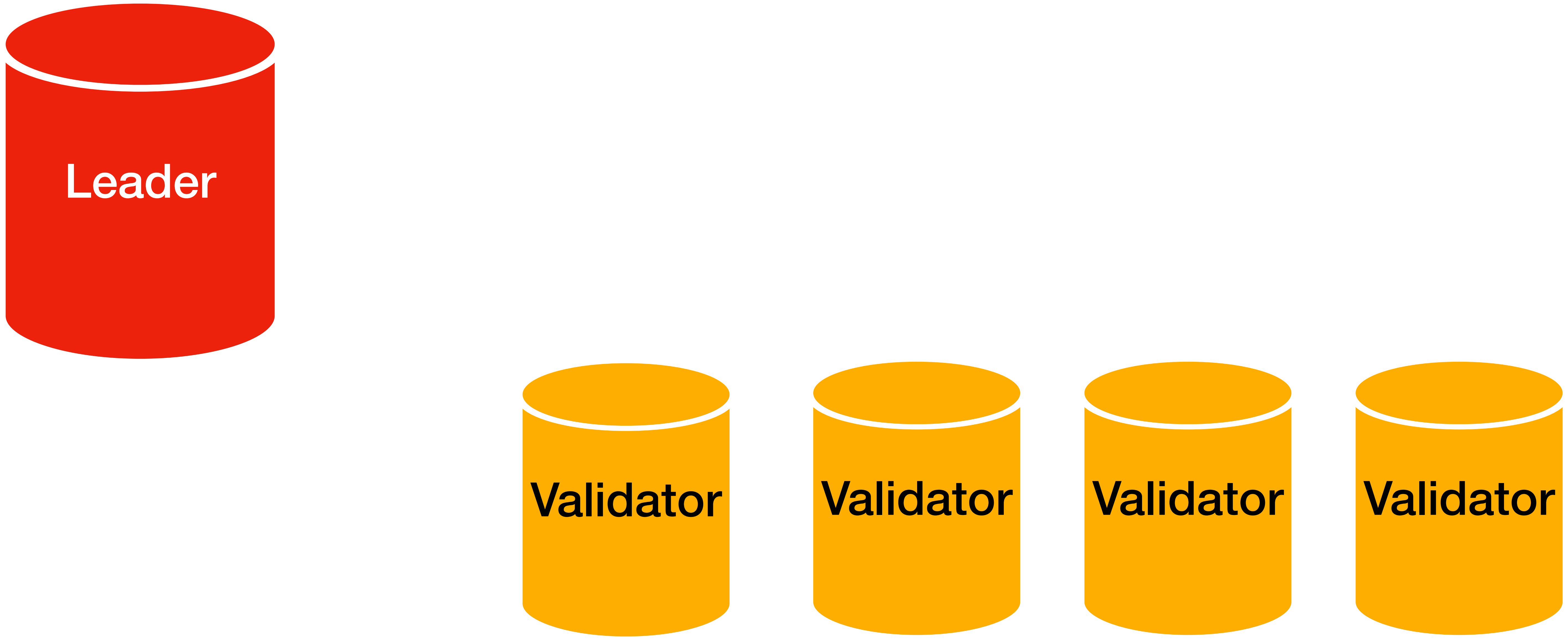


Solana



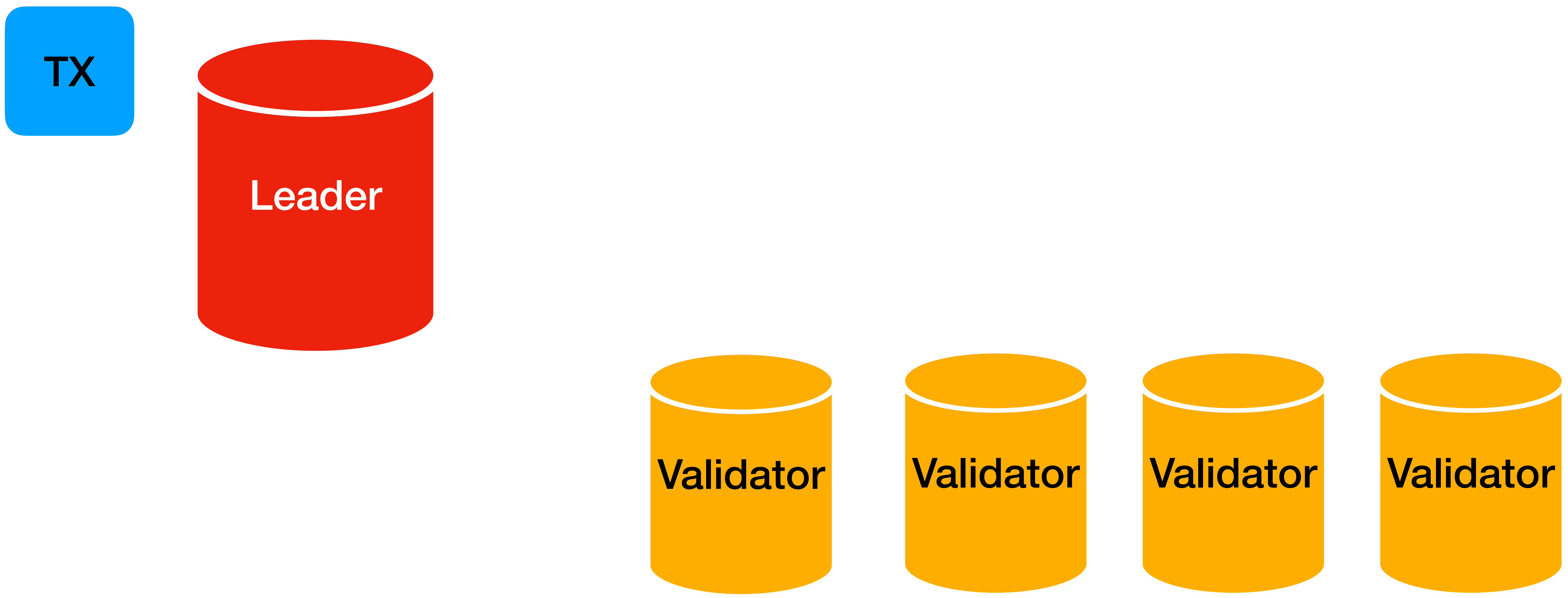
kraken

Solana

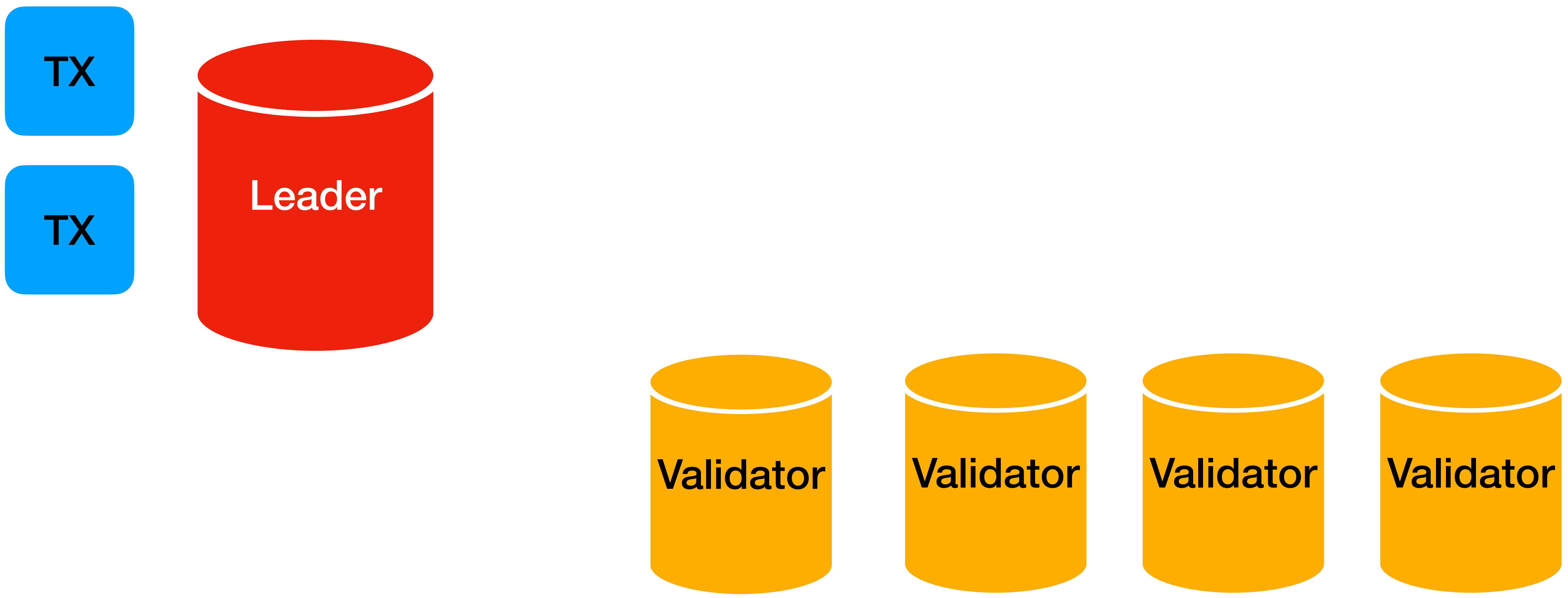


kraken

Solana

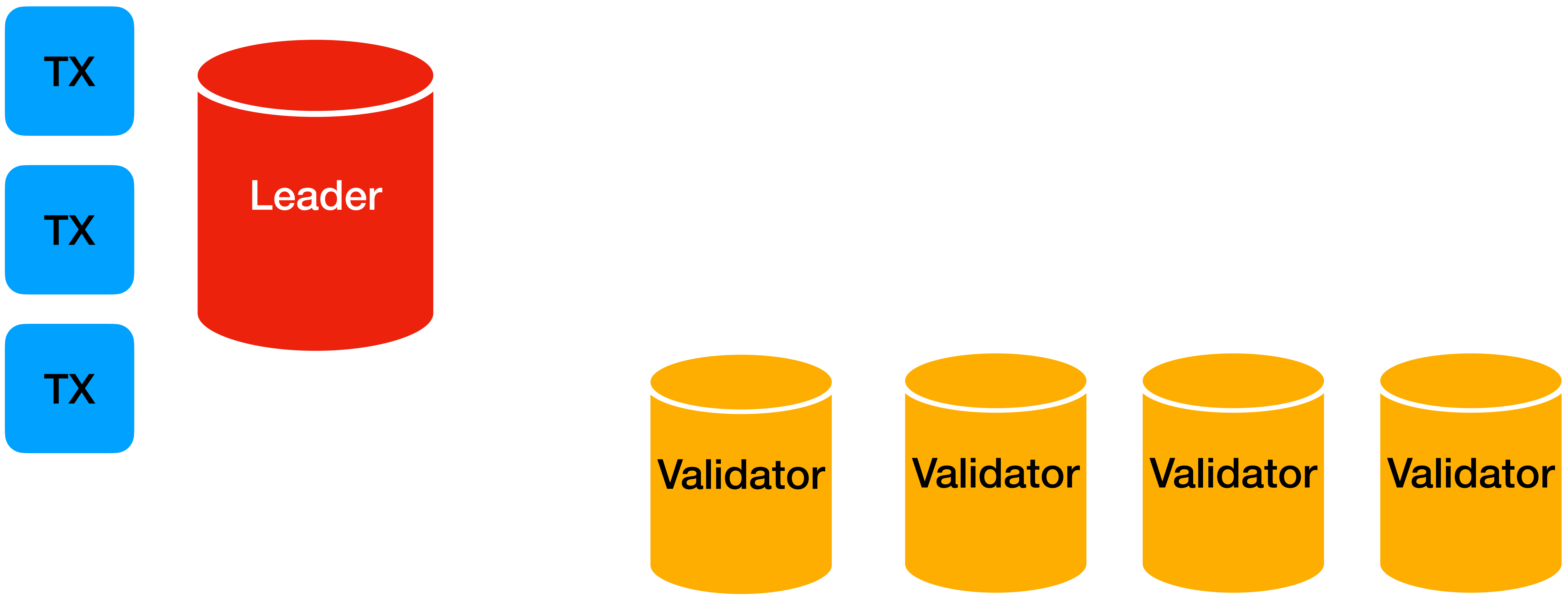


Solana

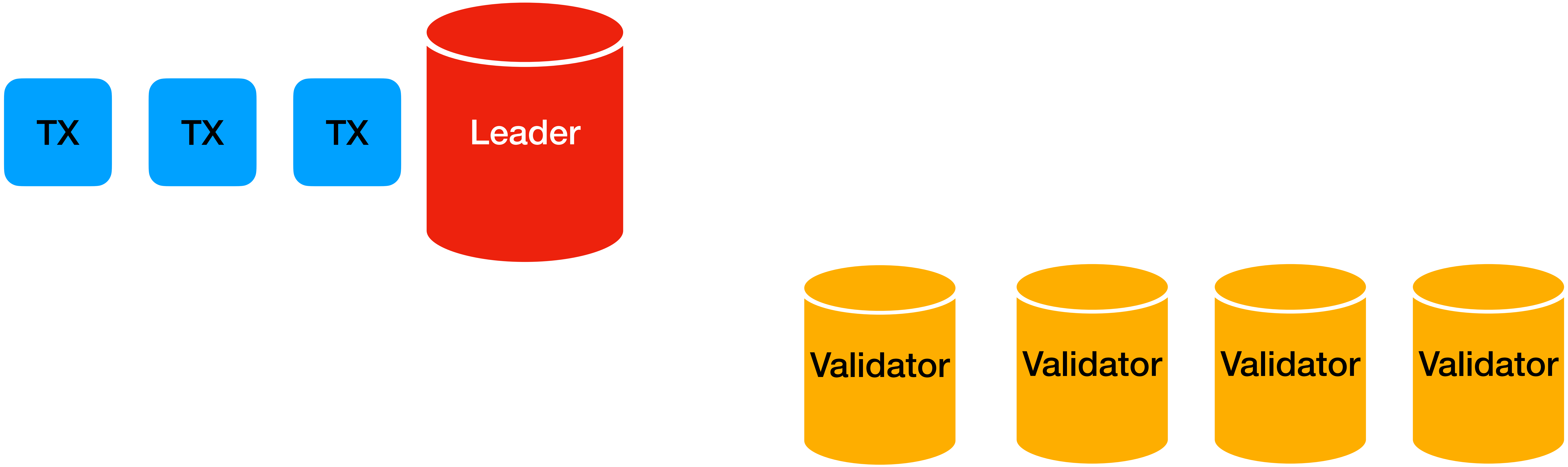


kraken

Solana

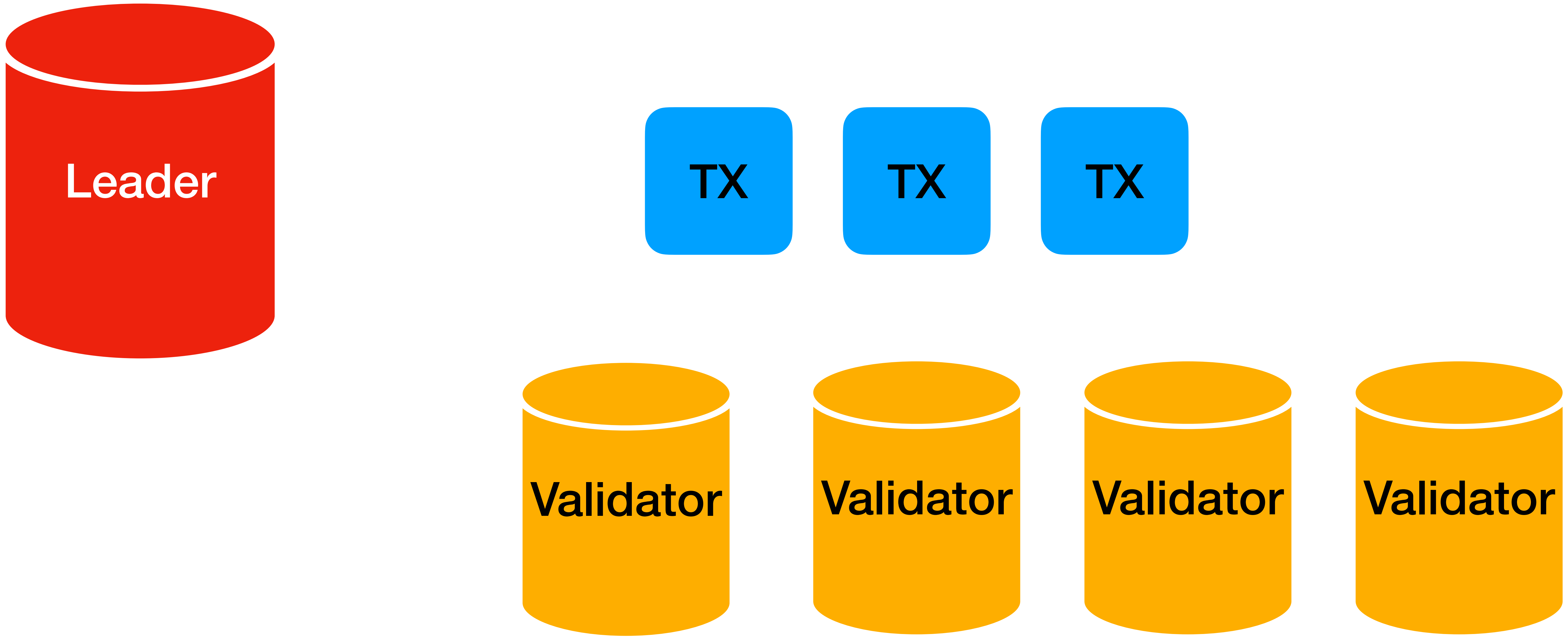


Solana

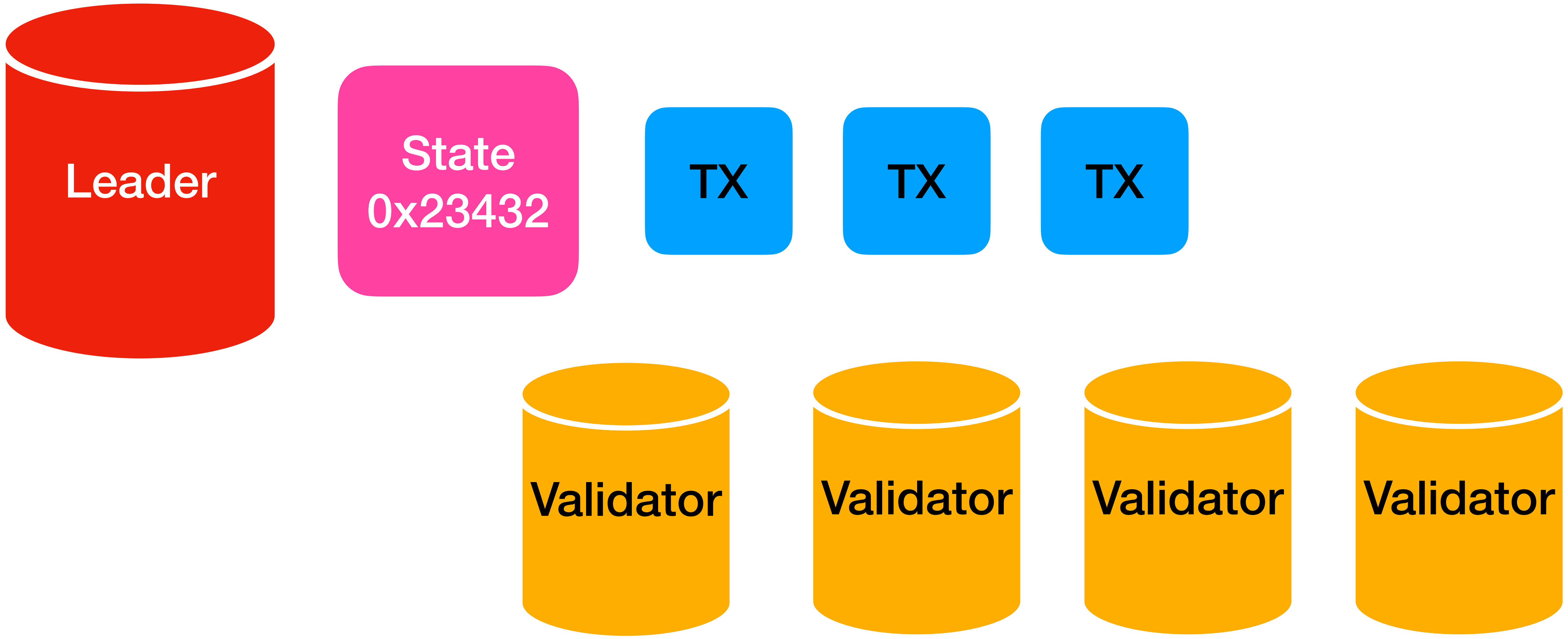


kraken

Solana

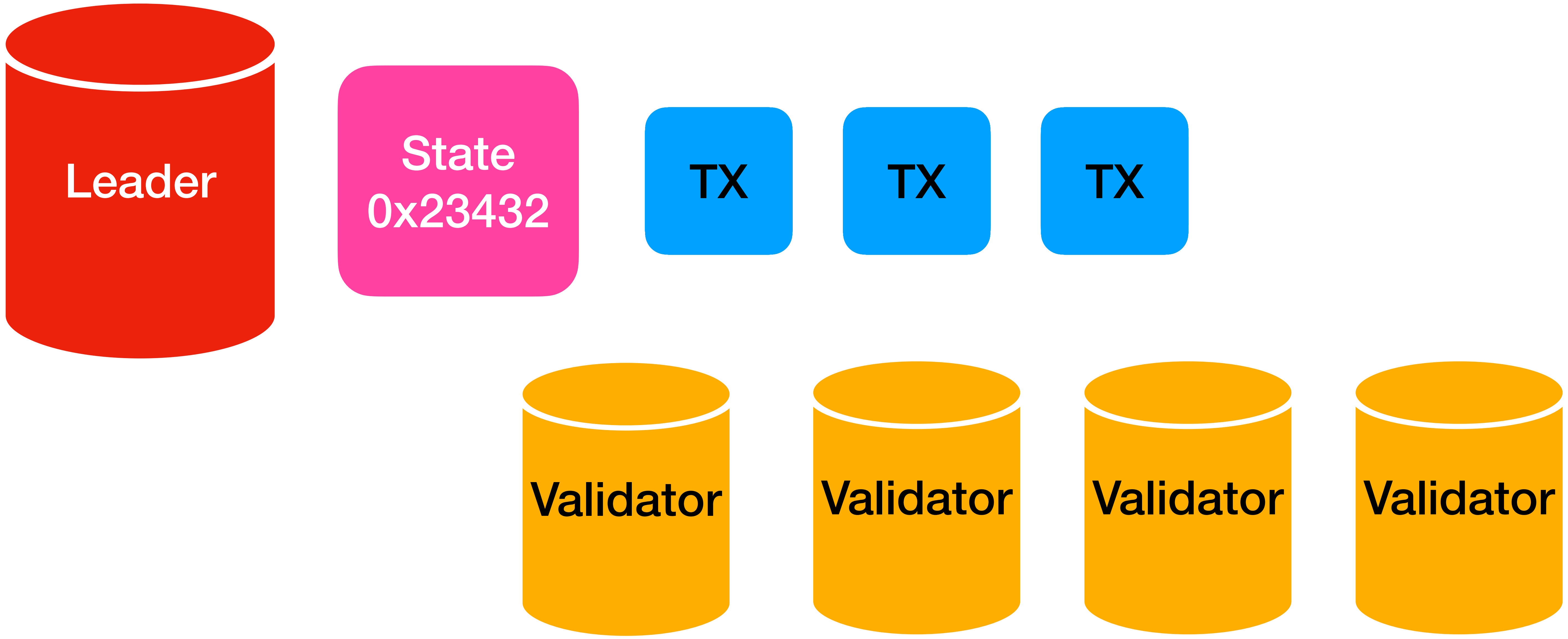


Solana

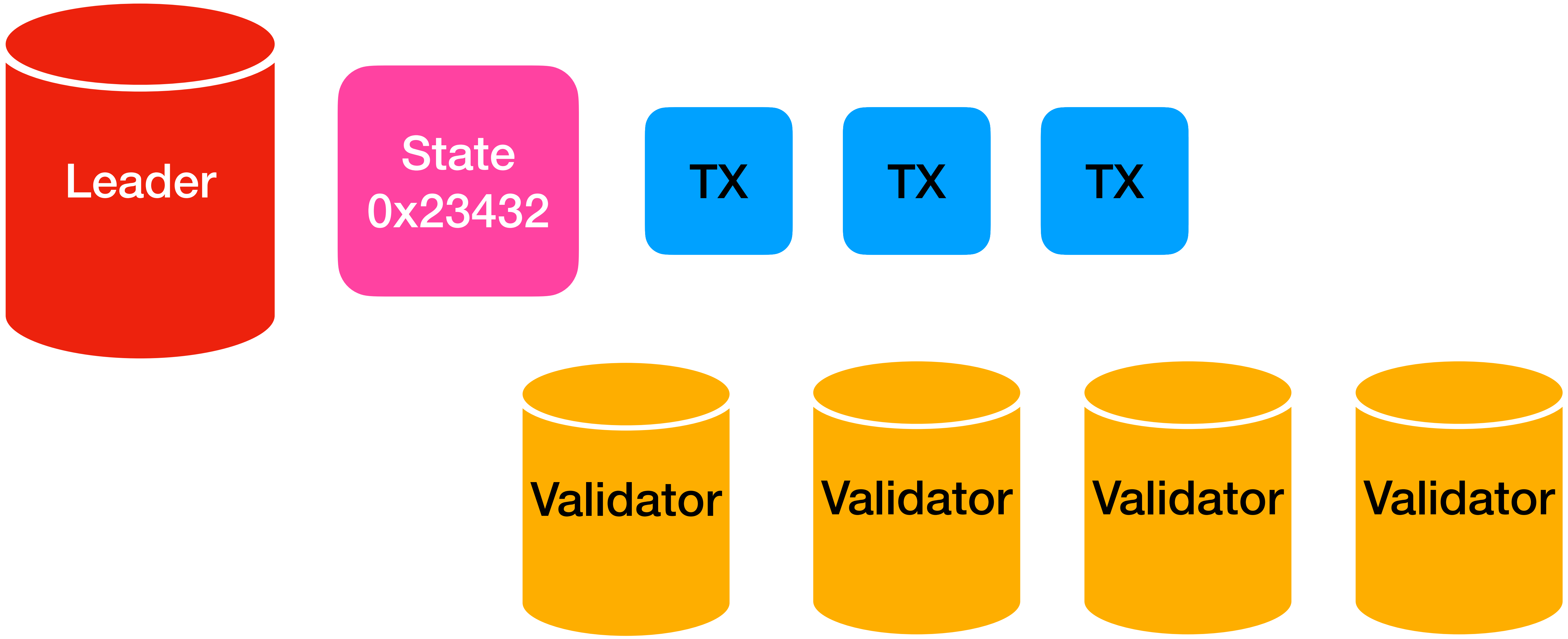


kraken

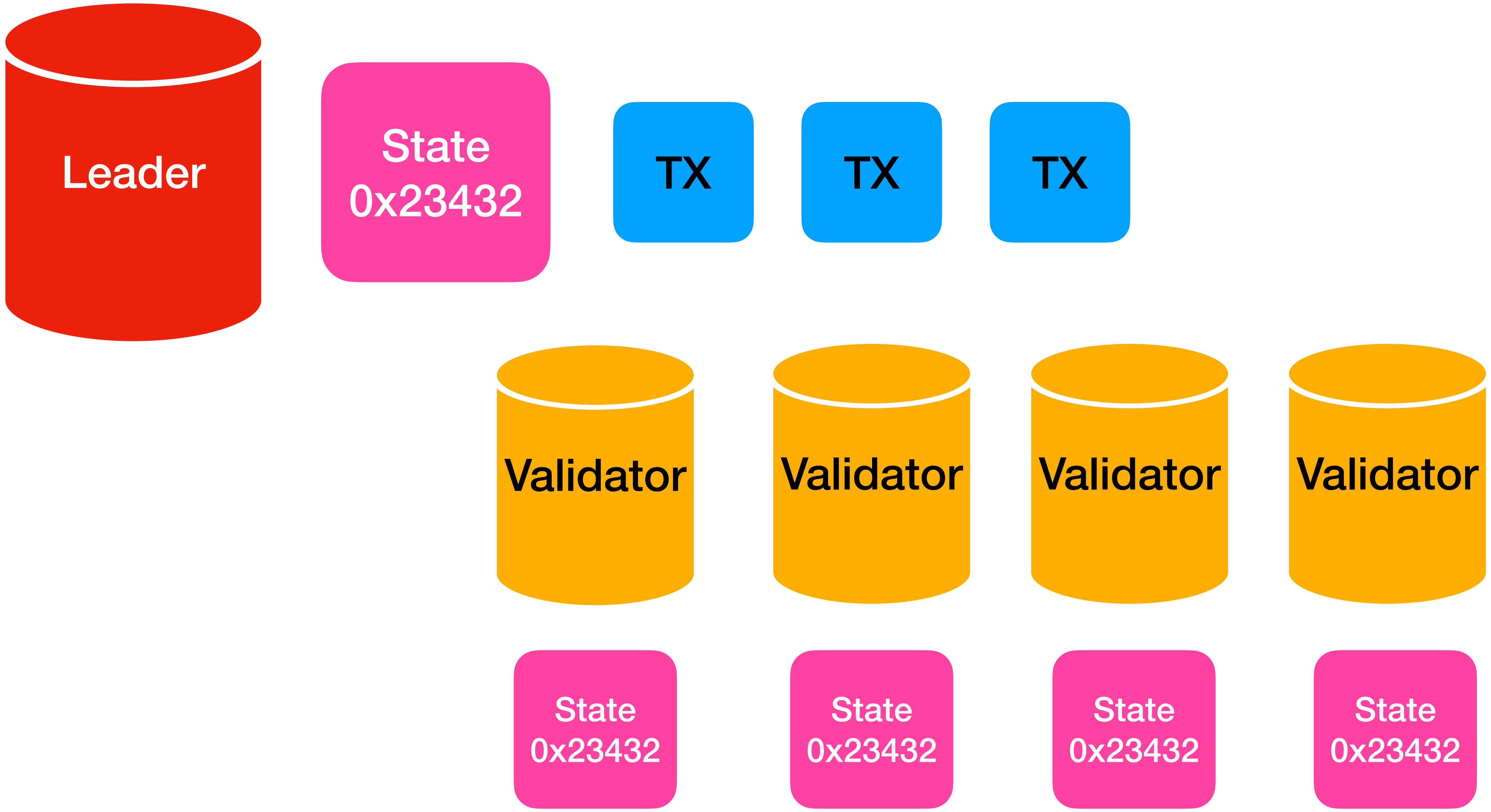
Solana


Leader
State
0x23432
TX
TX
TX
Validator
Validator
Validator
Validator

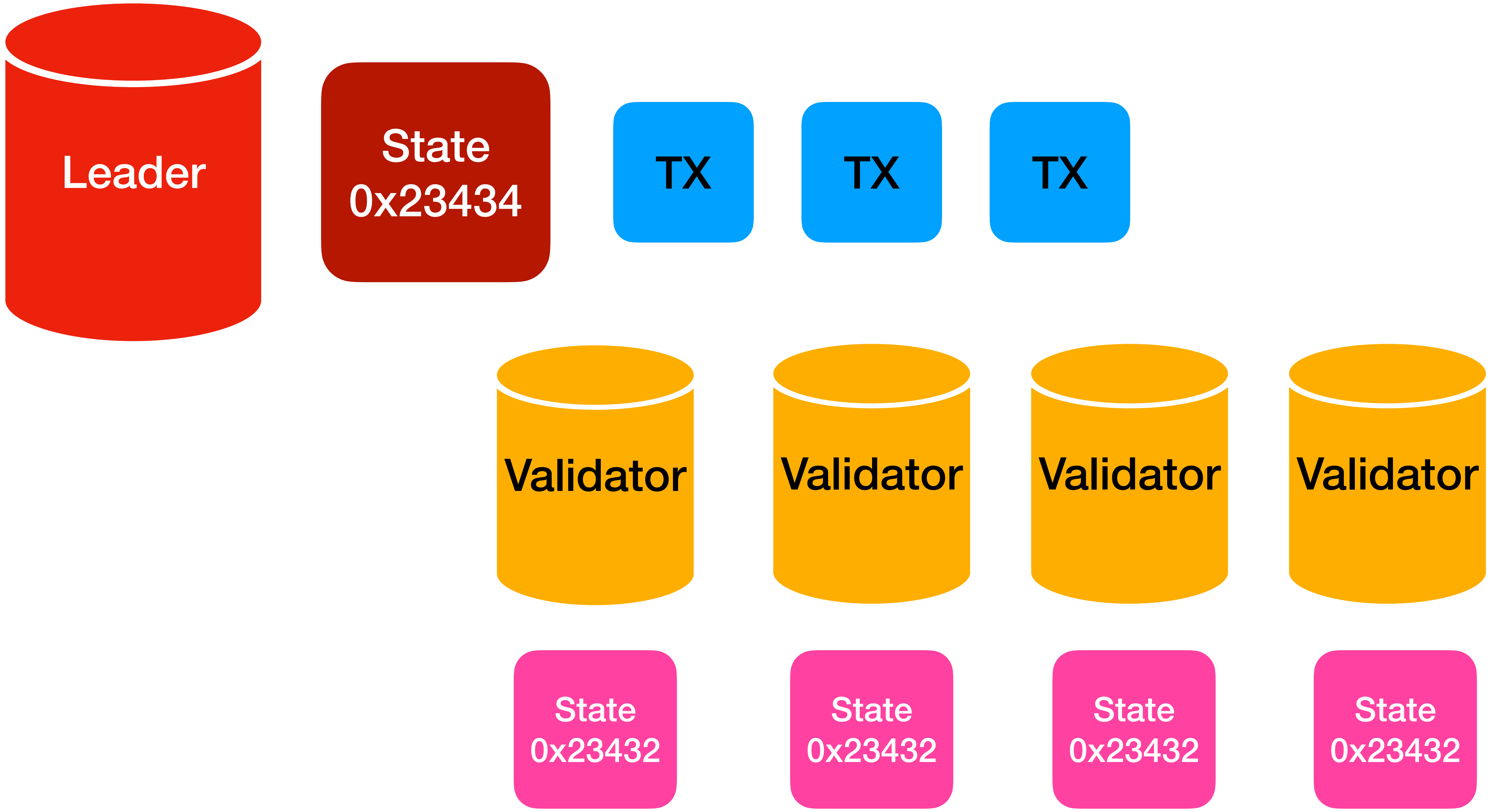
Solana


Leader
State
0x23432
TX
TX
TX
Validator
Validator
Validator
Validator

Solana


Leader
State
0x23432
TX
TX
TX
Validator
Validator
Validator
Validator
State
0x23432
State
0x23432
State
0x23432
State
0x23432

Solana



Leader
State
0x23434
TX
TX
TX
Validator
Validator
Validator
Validator
State
0x23432
State
0x23432
State
0x23432
State
0x23432

Solana

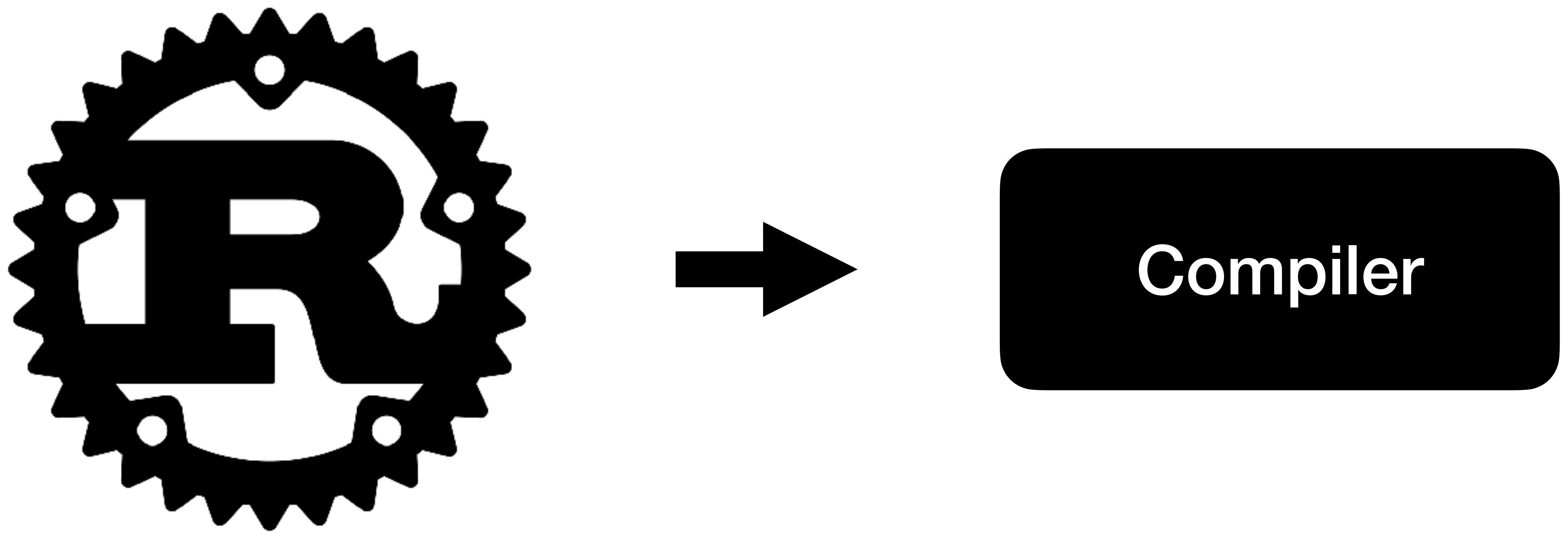
Smart-contracts

Solana

Smart-contracts

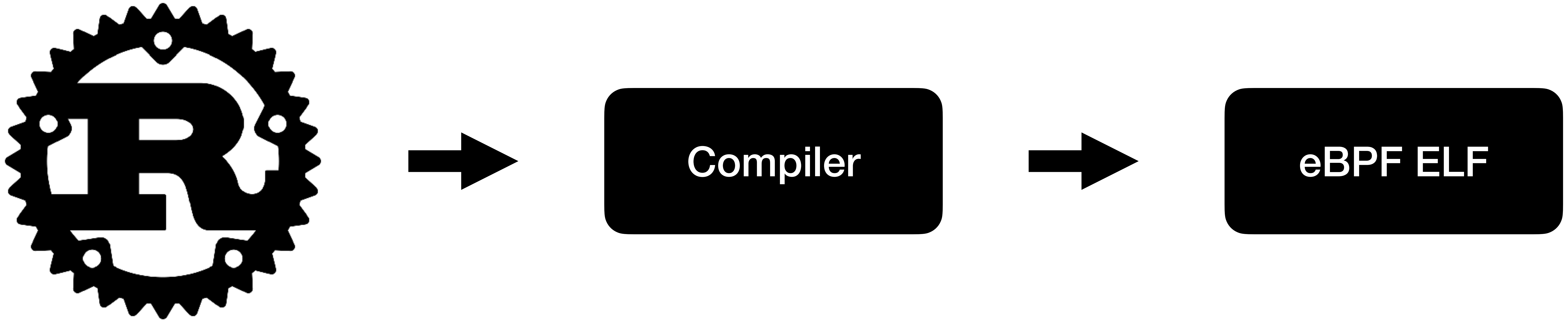
Solana

Smart-contracts



Solana

Smart-contracts



eBPF

extended Berkely Packet Filter

- eBPF
  - 64-bit RISC machine
  - Fixed-length instructions
  - Designed for packet-filtering
  - Virtual machine (with JIT) in the Linux kernel

eBPF

extended Berkely Packet Filter

- 10 registers + frame-pointer
- 64-bit wide instructions
    - Two different encodings (one with appended 64-bit immediate)
- 8 different instruction classes

class	value	description
BPF_LD	0x00	non-standard load operations
BPF_LDX	0x01	load into register operations
BPF_ST	0x02	store from immediate operations
BPF_STX	0x03	store from register operations
BPF_ALU	0x04	32-bit arithmetic operations
BPF_JMP	0x05	64-bit jump operations
BPF_JMP32	0x06	32-bit jump operations
BPF_ALU64	0x07	64-bit arithmetic operations

kraken

eBPF

extended

- 10 registe
- 64-bit wic
  - Two dif)
- 8 different

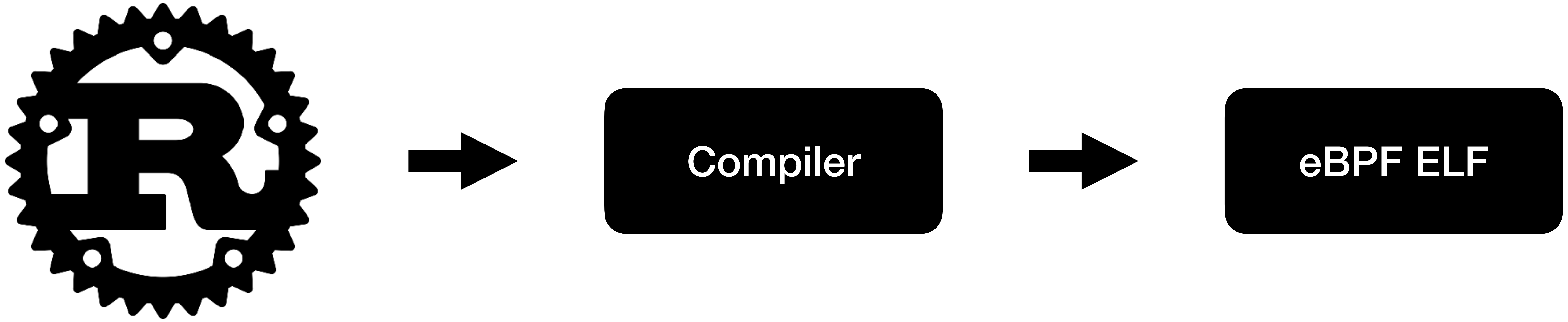
CVE-2022-23222	kernel/bpf/verifier.c in the Linux kernel through 5.15.14 allows local users to gain privileges beca
CVE-2022-0500	A flaw was found in unrestricted eBPF usage by the BPF_BTF_LOAD, leading to a possible out-of- allows a local user to crash or escalate their privileges on the system.
CVE-2022-0264	A vulnerability was found in the Linux kernel's eBPF verifier when handling internal data structure eBPF code to the kernel can use this to leak internal kernel memory details defeating some of the
CVE-2021-4202	A use-after-free flaw was found in nci_request in net/nfc/nci/core.c in NFC Controller Interface (N while the device is getting removed, leading to a privilege escalation problem.
CVE-2021-41864	prealloc_elems_and_freelist in kernel/bpf/stackmap.c in the Linux kernel before 5.14.12 allows u
CVE-2021-4135	A memory leak vulnerability was found in the Linux kernel's eBPF for the Simulated networking d local user could use this flaw to get unauthorized access to some data.
CVE-2021-4001	A race condition was found in the Linux kernel's ebpf verifier between bpf_map_update_elem and (cap_sys_admin or cap_bpf) can modify the frozen mapped address space. This flaw affects kern
CVE-2021-3490	The eBPF ALU32 bounds tracking for bitwise ops (AND, OR and XOR) in the Linux kernel did not p and therefore, arbitrary code execution. This issue was fixed via commit 049c4e13714e ("bpf: Fi v5.12.4, v5.11.21, and v5.10.37. The AND/OR issues were introduced by commit 3f50f132d840 2921c90d4718 ("bpf:Fix a verifier failure with xor") (5.10-rc1).
CVE-2021-3489	The eBPF RINGBUF bpf_ringbuf_reserve() function in the Linux kernel did not check that the allo kernel and therefore, arbitrary code execution. This issue was fixed via commit 4b81ccebaeee ("b v5.12.4, v5.11.21, and v5.10.37. It was introduced via 457f44363a88 ("bpf: Implement BPF ring
CVE-2021-34866	This vulnerability allows local attackers to escalate privileges on affected installations of Linux Ke order to exploit this vulnerability. The specific flaw exists within the handling of eBPF programs. T confusion condition. An attacker can leverage this vulnerability to escalate privileges and execute
CVE-2021-31440	This vulnerability allows local attackers to escalate privileges on affected installations of Linux Ke order to exploit this vulnerability. The specific flaw exists within the handling of eBPF programs. T attacker can leverage this vulnerability to escalate privileges and execute arbitrary code in the co
CVE-2021-20320	A flaw was found in s390 eBPF JIT in bpf_jit_insn in arch/s390/net/bpf_jit_comp.c in the Linux ke confidentiality problem.
CVE-2021-20268	An out-of-bounds access flaw was found in the Linux kernel's implementation of the eBPF code ve local user to crash the system or possibly escalate their privileges. The highest threat from this v
CVE-2016-4557	The replace_map_fd_with_map_ptr function in kernel/bpf/verifier.c in the Linux kernel before 4.5 of service (use-after-free) via crafted BPF instructions that reference an incorrect file descriptor.
CVE-2016-2383	The adjust_branches function in kernel/bpf/verifier.c in the Linux kernel before 4.5 does not cons memory by creating a packet filter and then loading crafted BPF instructions.

class	value	description
		non-standard load operations
		load into register operations
		store from immediate operations
		store from register operations
		32-bit arithmetic operations
		64-bit jump operations
		32-bit jump operations
		64-bit arithmetic operations

kraken

Solana

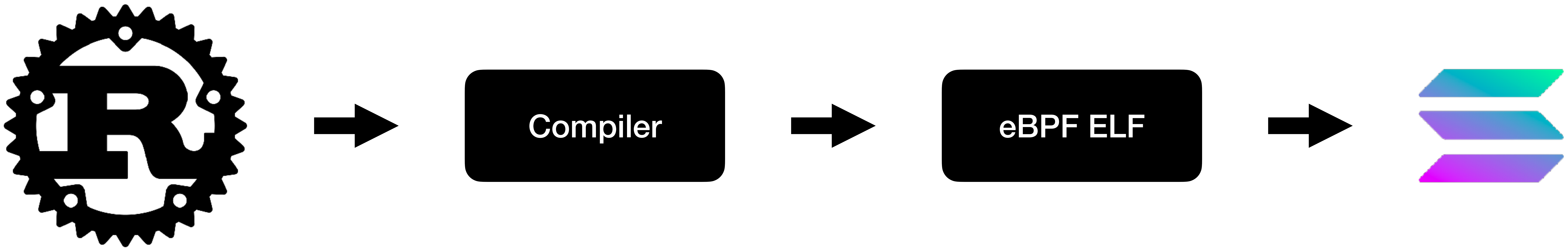
Smart-contracts



kraken

Solana

Smart-contracts



Solana

Smart-contracts

Smart
Contract

Solana

Smart-contracts

Smart
Contract

```
contract Foo {
    string public text = "Hello";

    function bla() {
        text = "test";
    }
}
```

Solana

Smart-contracts

Smart
Contract

```
contract Foo {
    string public text = "Hello";

    function bla() {
        text = "test";
    }
}
```

kraken

Solana

Smart-contracts

Smart
Contract

```
contract Foo {
    string public text = "Hello";

    function bla() {
        text = "test";
    }
}
```

Solana

Smart-contracts

Smart
Contract

```
contract Foo {
    string public text = "Hello";

    function bla() {
        text = "test";
    }
}
```

Solana

Smart-contracts

Smart
Contract

kraken

Solana

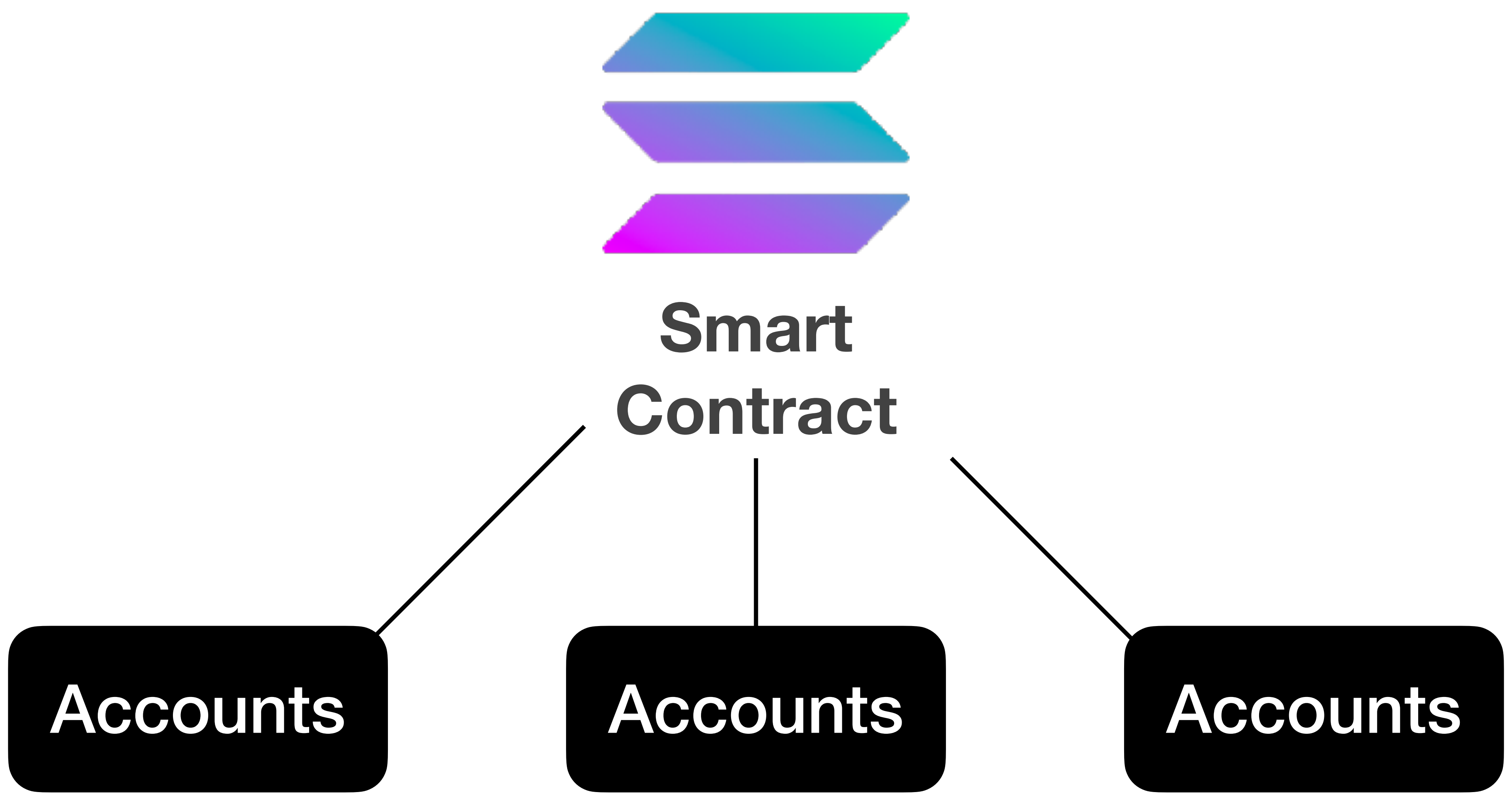
Smart-contracts

Smart
Contract

Stateless

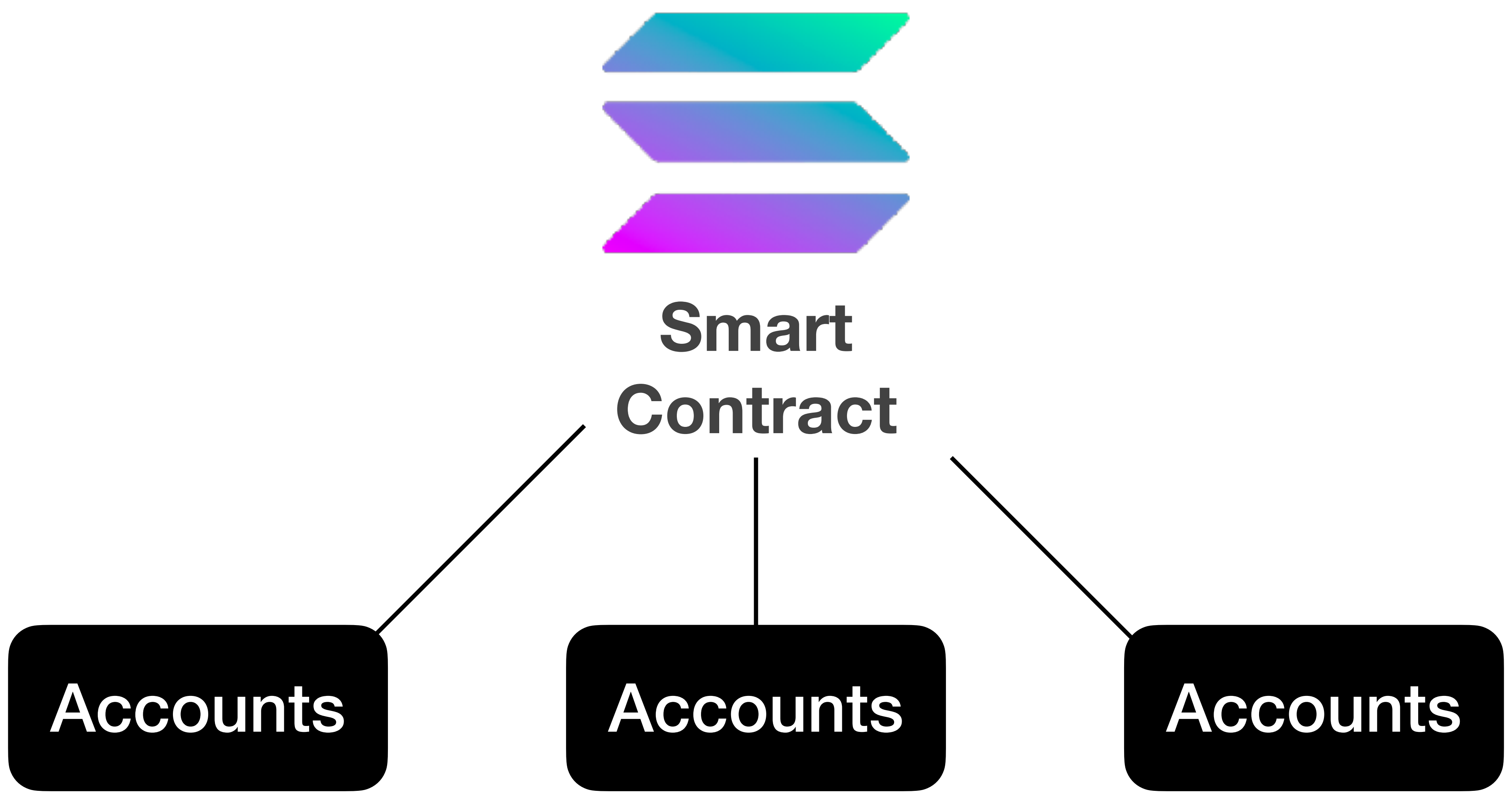
Solana

Smart-contracts



Solana

Smart-contracts



Accounts need to pay rent to not get deleted

Solana

Virtual machine

eBPF ELF

Solana

Virtual machine

eBPF ELF

Shared object

Solana
Virtual machine

eBPF ELF

Shared object
No writable global variables

Solana

Virtual machine

eBPF ELF

Program code loaded at 0x100000000
Stack starts at 0x200000000
Heap starts at 0x300000000
Program input at 0x400000000

Solana

Virtual machine

eBPF ELF

Program code loaded at 0x100000000
Stack starts at 0x200000000
Heap starts at 0x300000000
Program input at 0x400000000

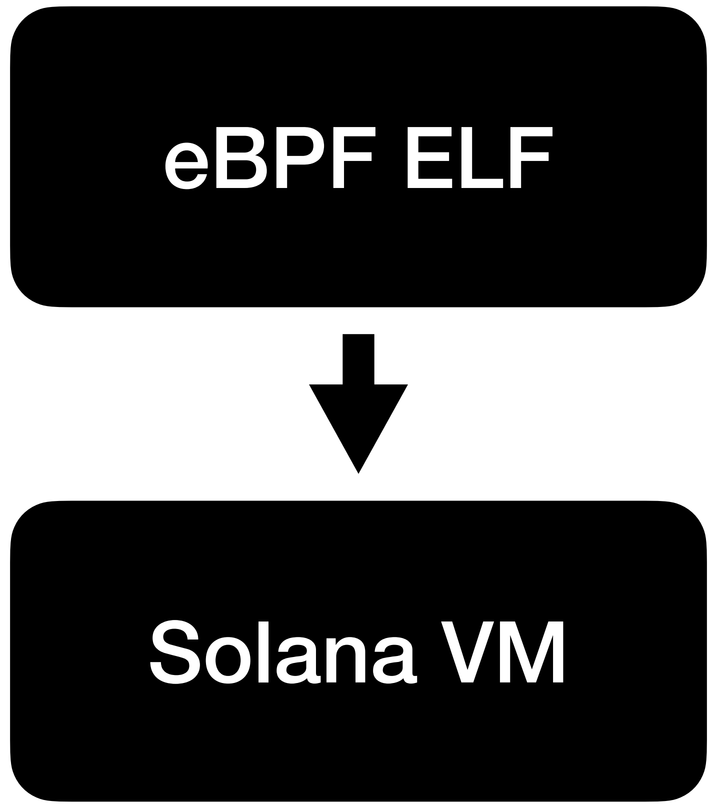
(No ASLR)

Solana
Virtual machine

eBPF ELF

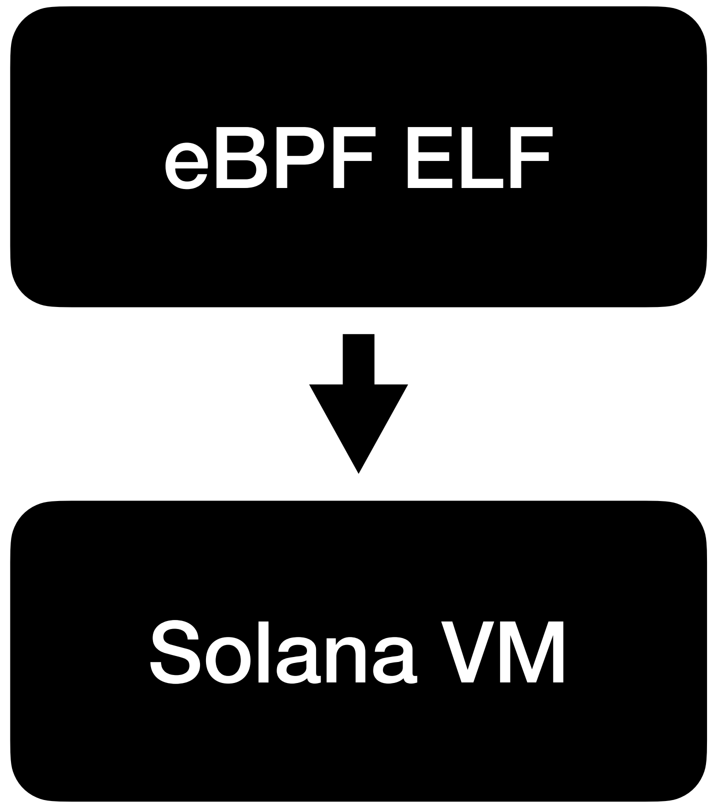
Solana

Virtual machine



Solana

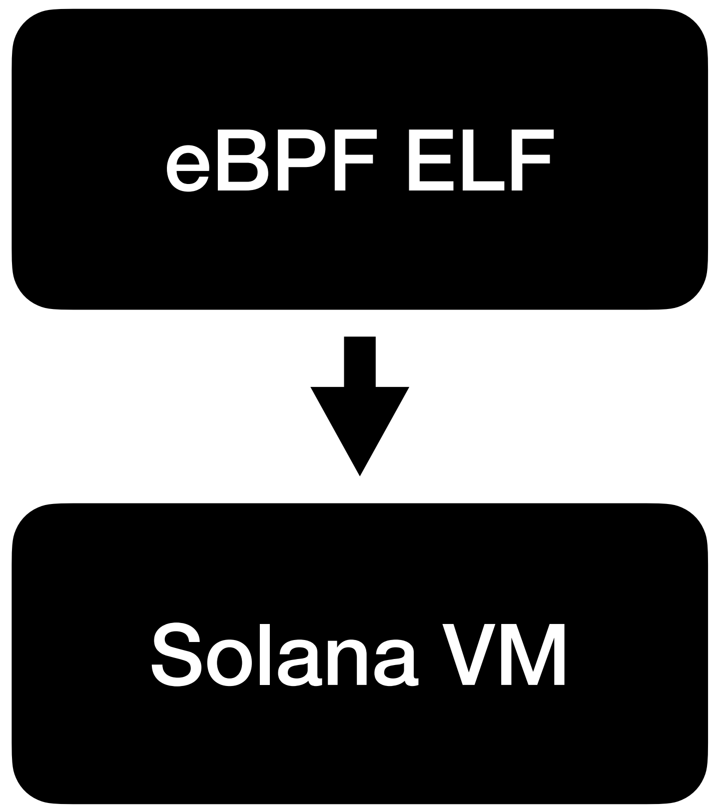
Virtual machine



Interpret the bytecode

Solana

Virtual machine

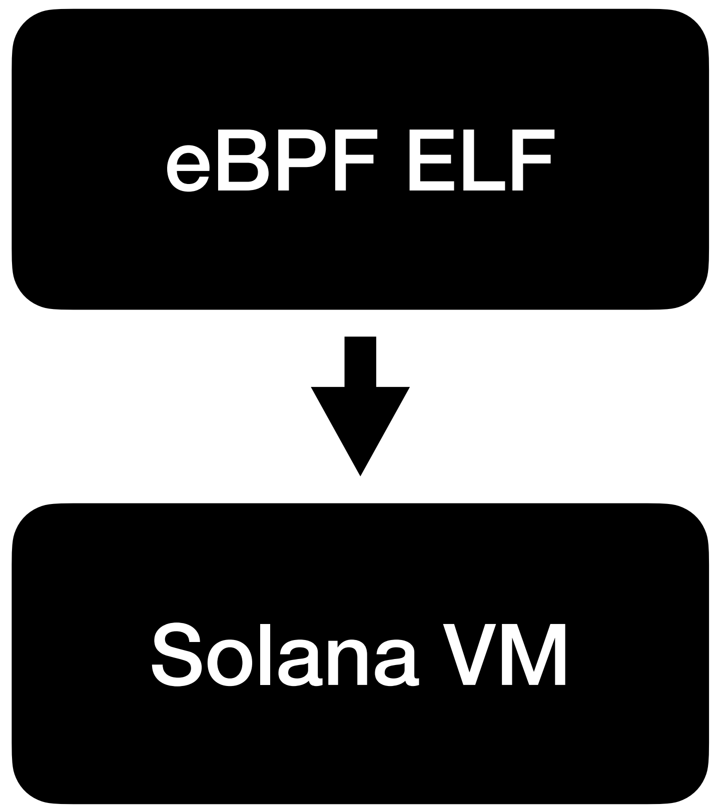


Interpret the bytecode

Or

Solana

Virtual machine



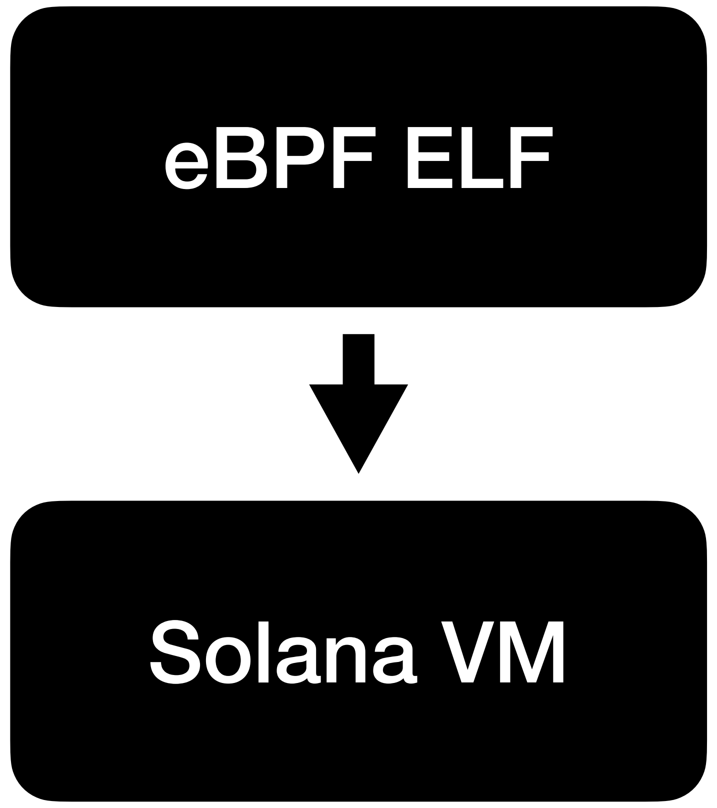
Interpret the bytecode

Or

Just-in-time compilation

Solana

Virtual machine



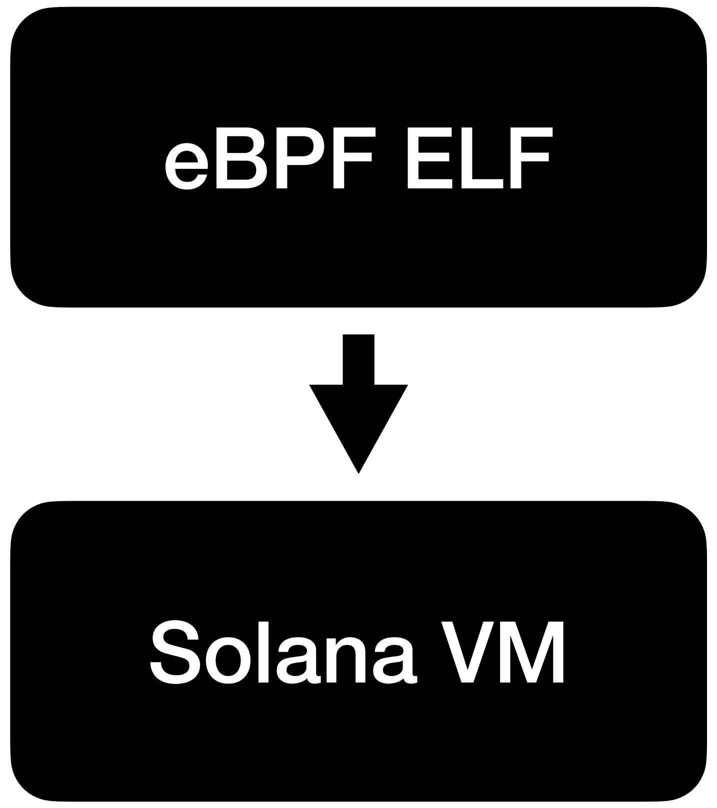
Interpret the bytecode

Or

Just-in-time compilation

kraken

Solana
Virtual machine



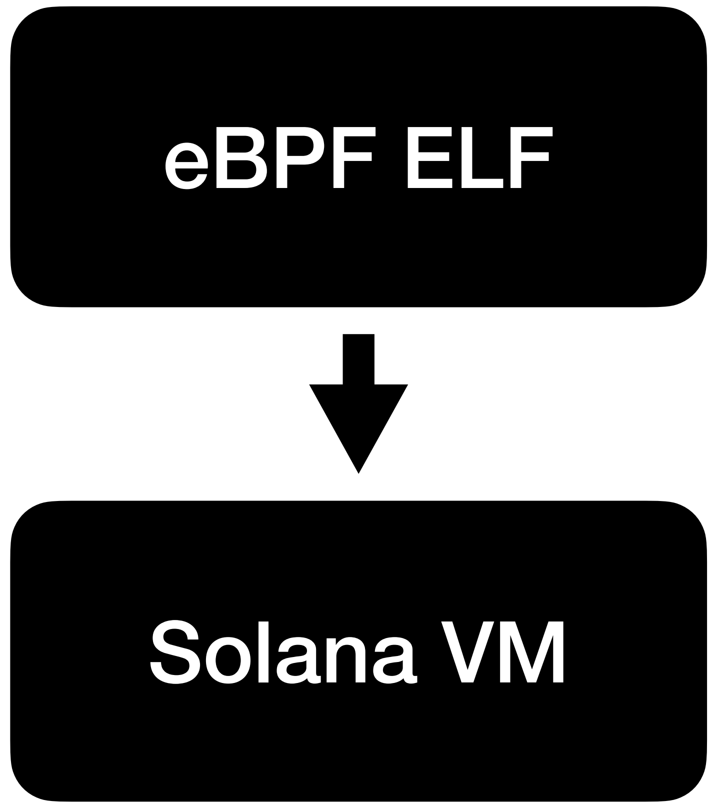
Interpret the bytecode

Or

Just-in-time compilation
(Compile the bytecode to
native machine code at runtime)

Solana

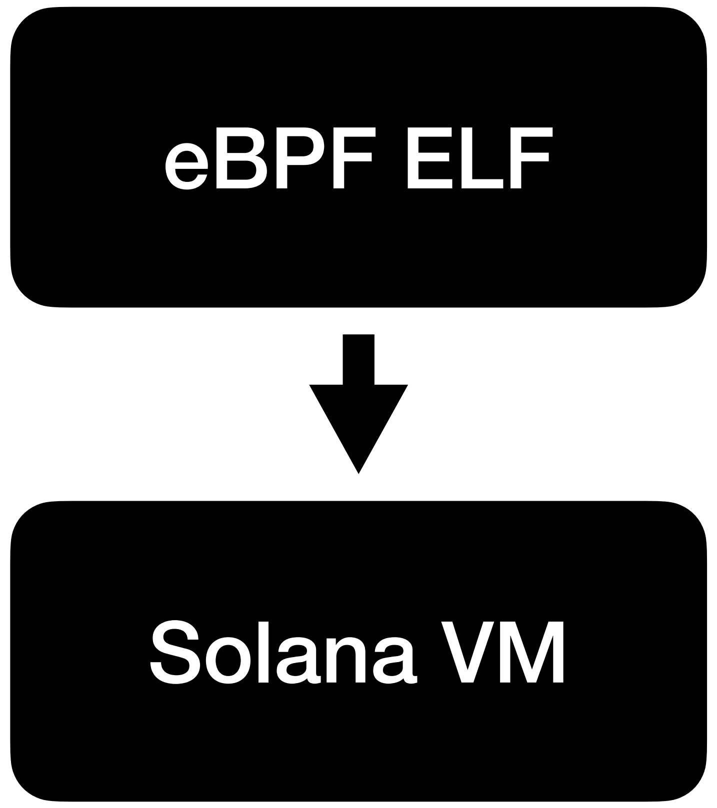
Virtual machine



Just-in-time compilation

Solana

Virtual machine

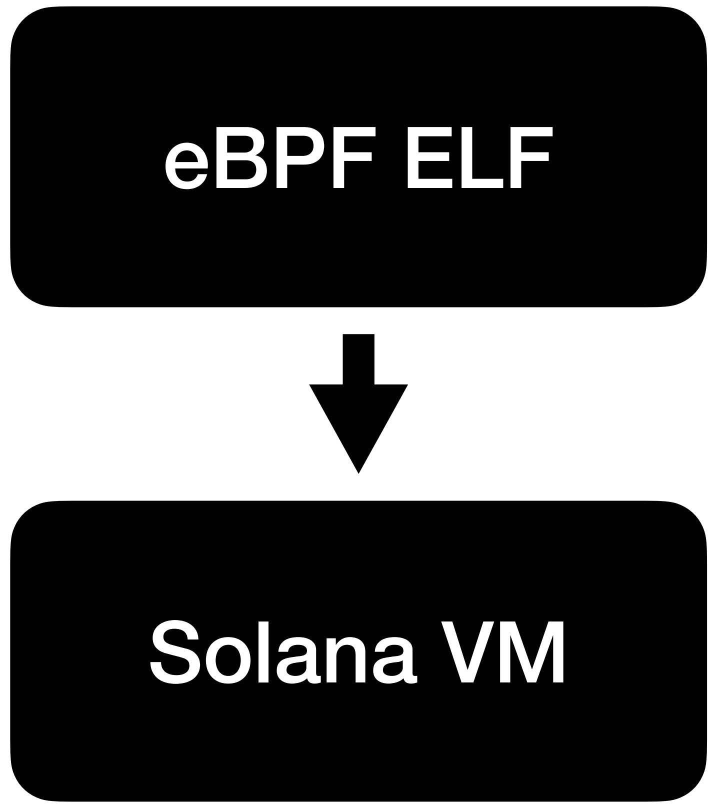


Just-in-time compilation

Requires writable & (eventually) executable pages

Solana

Virtual machine



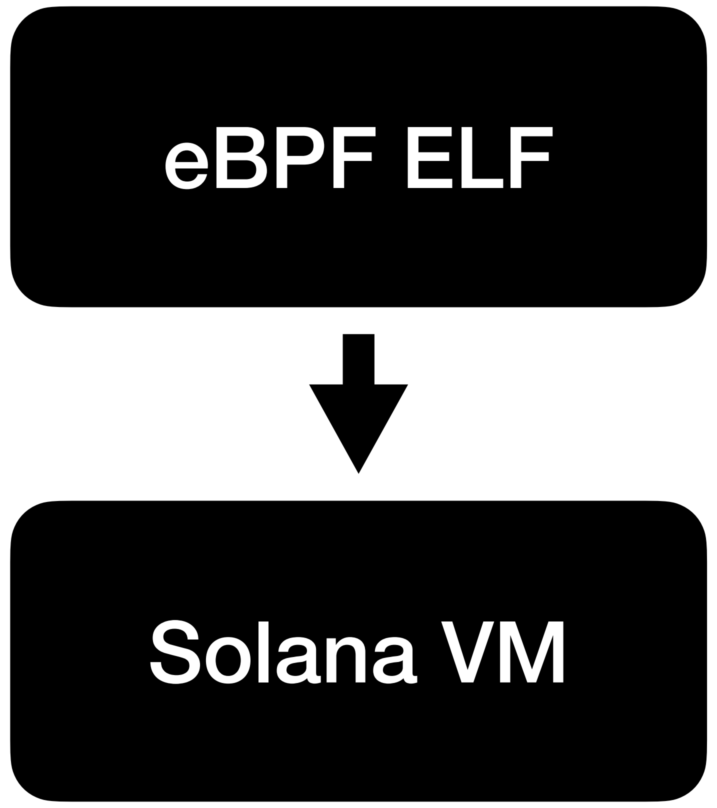
Just-in-time compilation

Requires writable & (eventually) executable pages

Needs to bring its own memory safety

Solana

Virtual machine



Just-in-time compilation

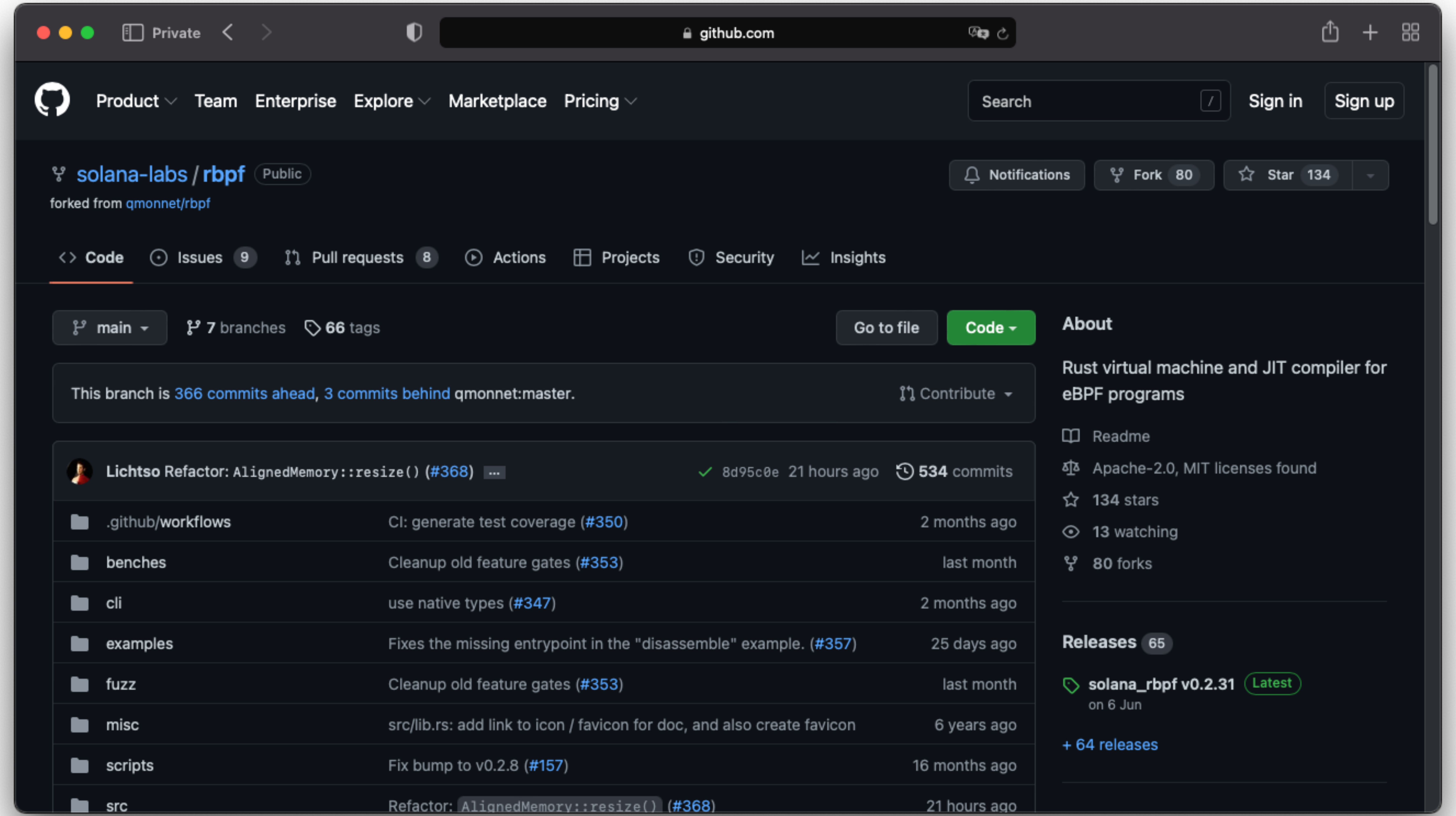
Requires writable & (eventually) executable pages

Needs to bring its own memory safety

Fairly difficult to implement securely

Solana

Virtual machine



Solana

Virtual machine

Private

github.com

README.md

solana_rbpf

Rust (user-space) virtual machine for eBPF

build passing crates.io v0.2.31

Description

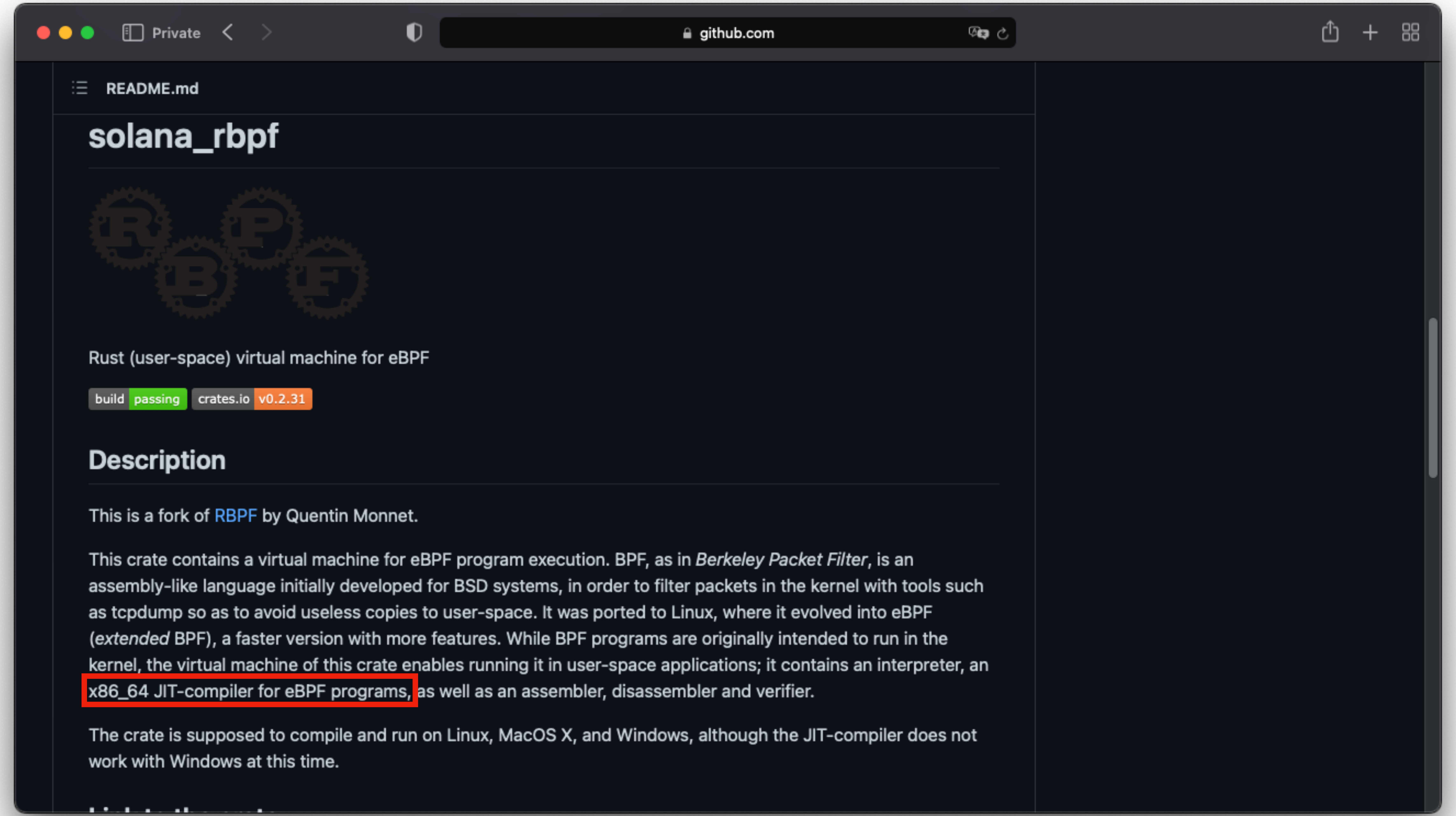
This is a fork of RBPF by Quentin Monnet.

This crate contains a virtual machine for eBPF program execution. BPF, as in *Berkeley Packet Filter*, is an assembly-like language initially developed for BSD systems, in order to filter packets in the kernel with tools such as tcpdump so as to avoid useless copies to user-space. It was ported to Linux, where it evolved into eBPF (*extended* BPF), a faster version with more features. While BPF programs are originally intended to run in the kernel, the virtual machine of this crate enables running it in user-space applications; it contains an interpreter, an x86_64 JIT-compiler for eBPF programs, as well as an assembler, disassembler and verifier.

The crate is supposed to compile and run on Linux, MacOS X, and Windows, although the JIT-compiler does not work with Windows at this time.

Solana

Virtual machine



github.com

README.md

solana_rbpf

Rust (user-space) virtual machine for eBPF

build passing crates.io v0.2.31

Description

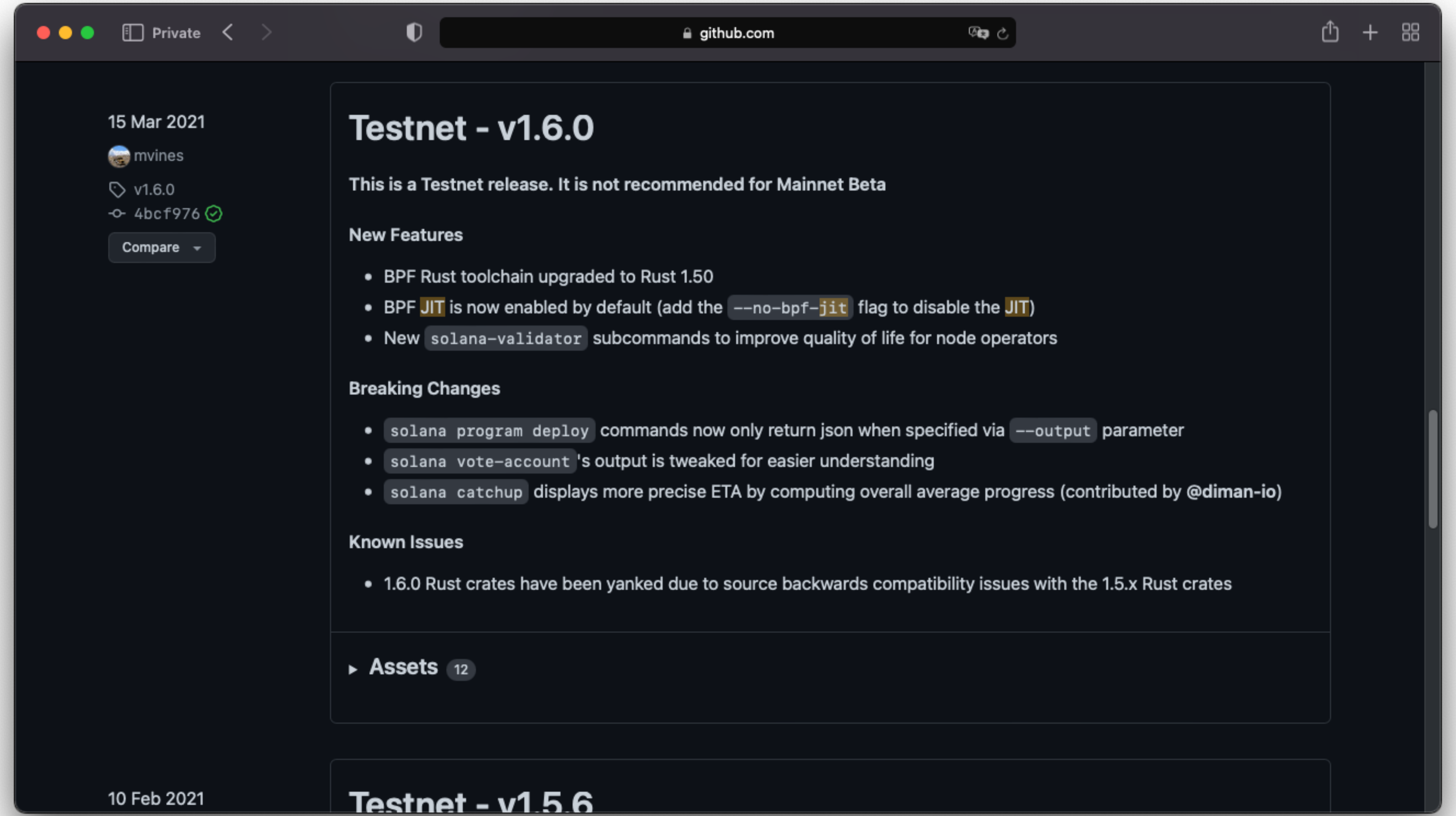
This is a fork of RBPF by Quentin Monnet.

This crate contains a virtual machine for eBPF program execution. BPF, as in *Berkeley Packet Filter*, is an assembly-like language initially developed for BSD systems, in order to filter packets in the kernel with tools such as tcpdump so as to avoid useless copies to user-space. It was ported to Linux, where it evolved into eBPF (*extended* BPF), a faster version with more features. While BPF programs are originally intended to run in the kernel, the virtual machine of this crate enables running it in user-space applications; it contains an interpreter, an $\boxed{\text{x86\_64 JIT-compiler for eBPF programs,}}$ as well as an assembler, disassembler and verifier.

The crate is supposed to compile and run on Linux, MacOS X, and Windows, although the JIT-compiler does not work with Windows at this time.

Solana

Virtual machine



Testnet - v1.6.0

This is a Testnet release. It is not recommended for Mainnet Beta

New Features

- BPF Rust toolchain upgraded to Rust 1.50
- BPF JIT is now enabled by default (add the `--no-bpf-jit` flag to disable the JIT)
- New `solana-validator` subcommands to improve quality of life for node operators

Breaking Changes

- `solana program deploy` commands now only return json when specified via `--output` parameter
- `solana vote-account`'s output is tweaked for easier understanding
- `solana catchup` displays more precise ETA by computing overall average progress (contributed by **@diman-io**)

Known Issues

- 1.6.0 Rust crates have been yanked due to source backwards compatibility issues with the 1.5.x Rust crates

▸ **Assets** 12

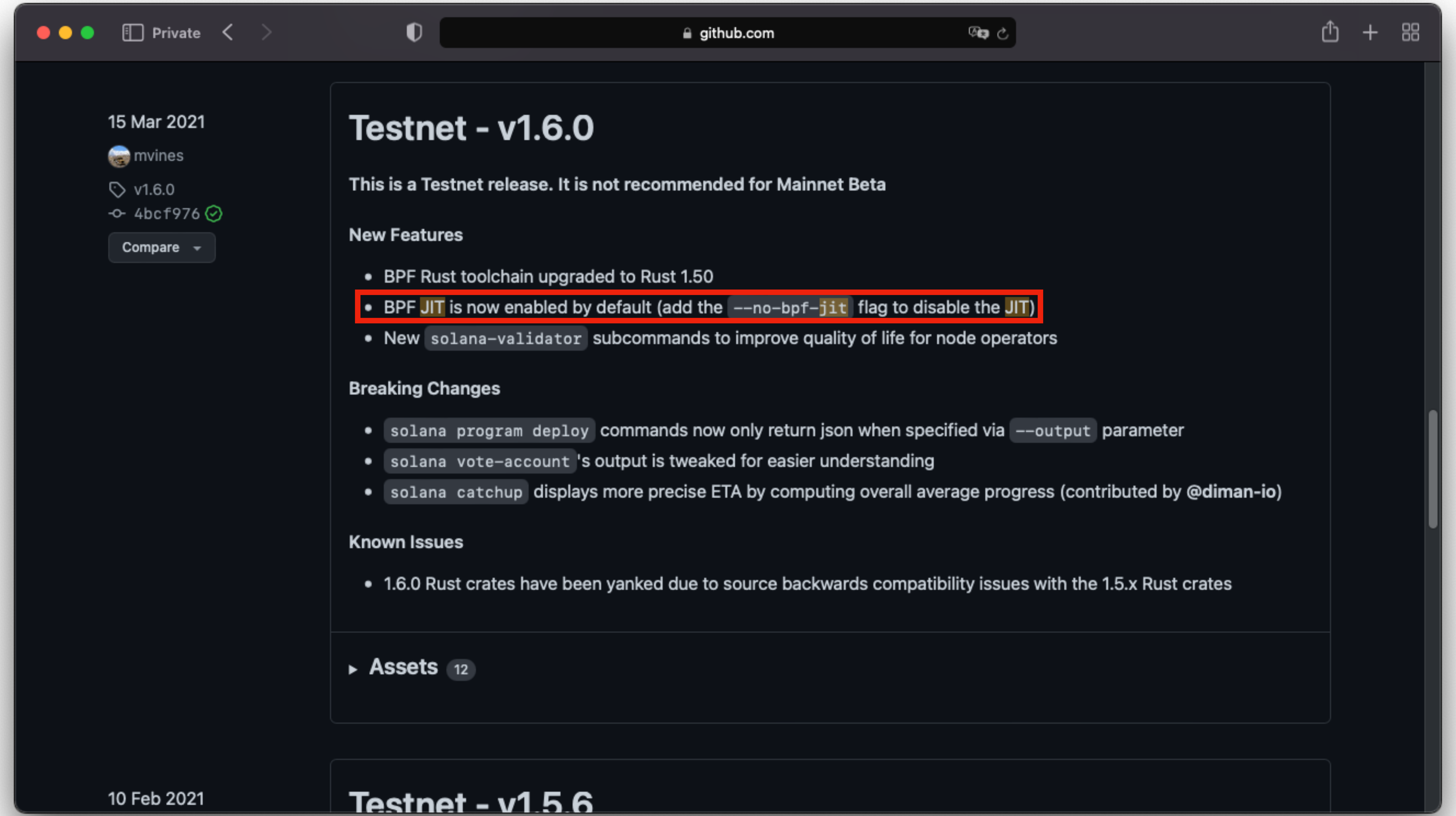
10 Feb 2021

Testnet - v1.5.6

kraken

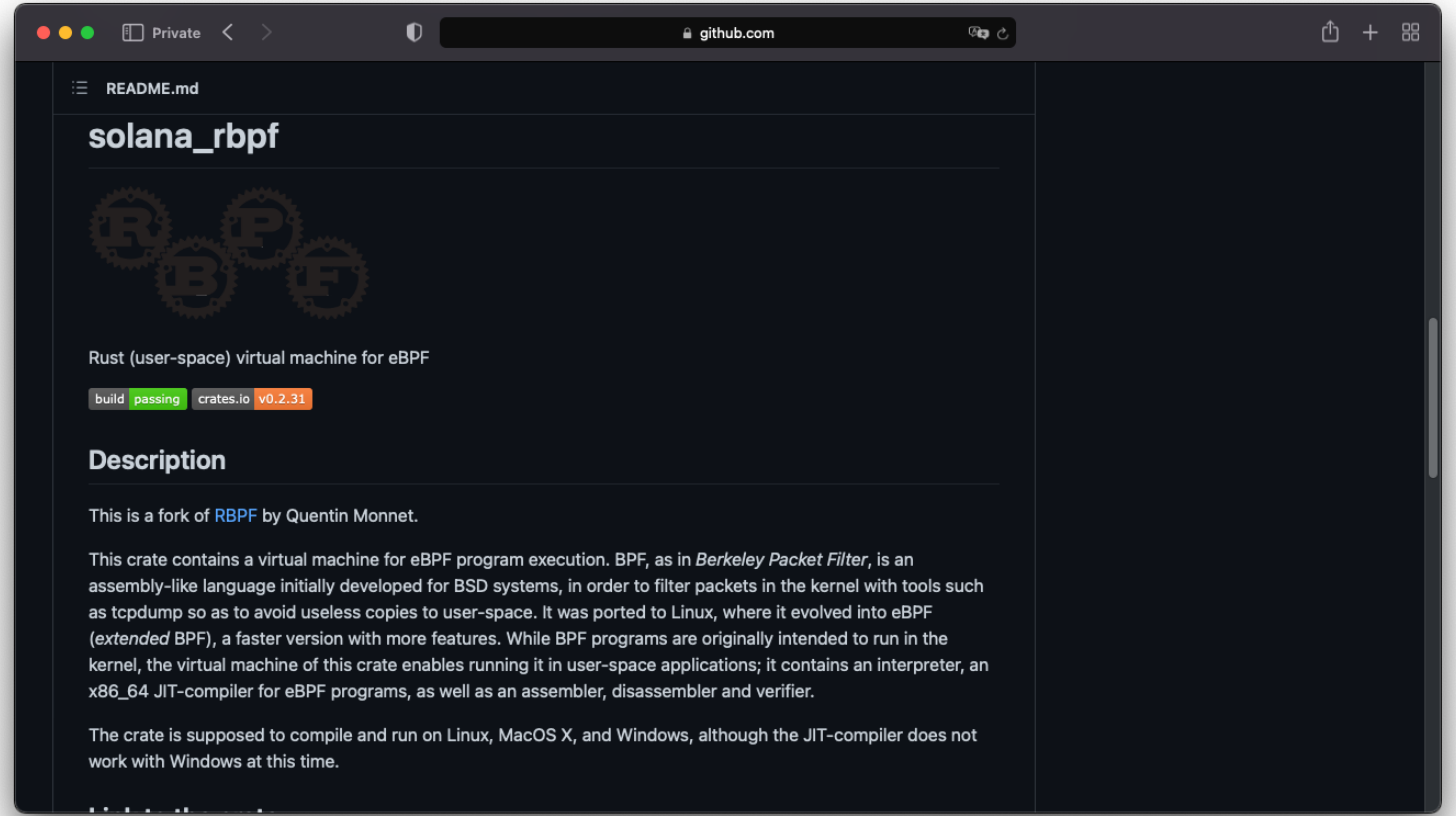
Solana

Virtual machine



Solana

Virtual machine


github.com

README.md

solana_rbpf

Rust (user-space) virtual machine for eBPF

build passing crates.io v0.2.31

Description

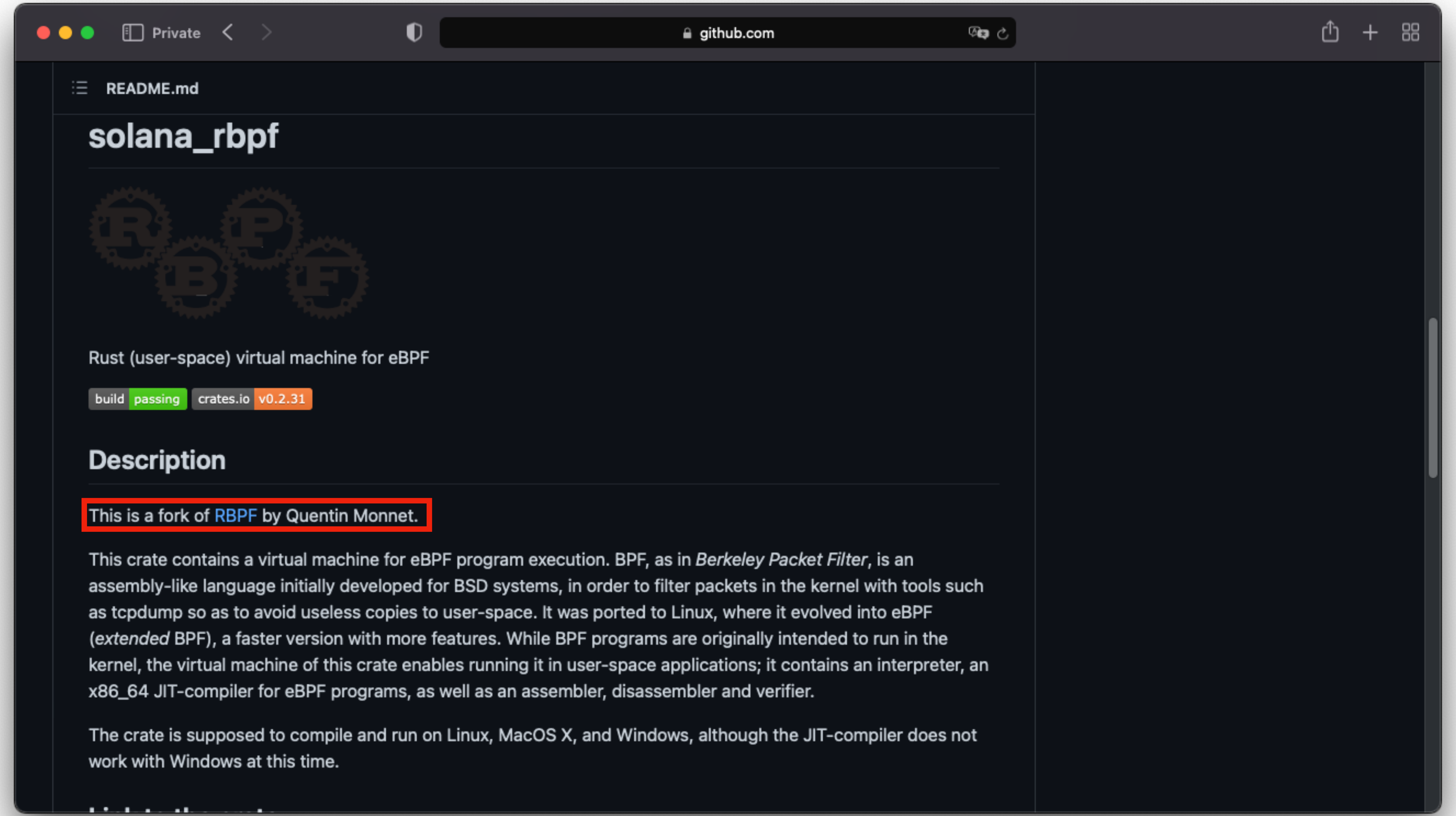
This is a fork of RBPF by Quentin Monnet.

This crate contains a virtual machine for eBPF program execution. BPF, as in *Berkeley Packet Filter*, is an assembly-like language initially developed for BSD systems, in order to filter packets in the kernel with tools such as tcpdump so as to avoid useless copies to user-space. It was ported to Linux, where it evolved into eBPF (*extended* BPF), a faster version with more features. While BPF programs are originally intended to run in the kernel, the virtual machine of this crate enables running it in user-space applications; it contains an interpreter, an x86_64 JIT-compiler for eBPF programs, as well as an assembler, disassembler and verifier.

The crate is supposed to compile and run on Linux, MacOS X, and Windows, although the JIT-compiler does not work with Windows at this time.

Solana

Virtual machine



github.com

README.md

solana_rbpf

Rust (user-space) virtual machine for eBPF

build passing crates.io v0.2.31

Description

This is a fork of RBPF by Quentin Monnet.

This crate contains a virtual machine for eBPF program execution. BPF, as in *Berkeley Packet Filter*, is an assembly-like language initially developed for BSD systems, in order to filter packets in the kernel with tools such as tcpdump so as to avoid useless copies to user-space. It was ported to Linux, where it evolved into eBPF (*extended* BPF), a faster version with more features. While BPF programs are originally intended to run in the kernel, the virtual machine of this crate enables running it in user-space applications; it contains an interpreter, an x86_64 JIT-compiler for eBPF programs, as well as an assembler, disassembler and verifier.

The crate is supposed to compile and run on Linux, MacOS X, and Windows, although the JIT-compiler does not work with Windows at this time.

Solana

Virtual machine

github.com

README.md

To do list

- Implement some traits (`Clone`, `Drop`, `Debug` are good candidates).
- Provide built-in support for user-space array and hash BPF maps.
- Improve safety of JIT-compiled programs with runtime memory checks.
- Add helpers (some of those supported in the kernel, such as checksum update, could be helpful).
- Improve verifier. Could we find a way to directly support programs compiled with clang?
- Maybe one day, tail calls?
- JIT-compilers for other architectures?
- …

License

Following the effort of the Rust language project itself in order to ease integration with other projects, the rbpf crate is distributed under the terms of both the MIT license and the Apache License (Version 2.0).

See LICENSE-APACHE and LICENSE-MIT for details.

Inspired by

- uBPF, a C user-space implementation of an eBPF virtual machine, with a JIT-compiler and disassembler (and also including the assembler from the human-readable form of the instructions, such as in `mov r0, 0x1337`), by Rich Lane for Big Switch Networks (2015)
- *Building a simple JIT in Rust*, by Jonathan Turner (2015)

Solana

Virtual machine

github.com

README.md

To do list

- Implement some traits (`Clone`, `Drop`, `Debug` are good candidates).
- Provide built-in support for user-space array and hash BPF maps.
- Improve safety of JIT-compiled programs with runtime memory checks.
- Add helpers (some of those supported in the kernel, such as checksum update, could be helpful).
- Improve verifier. Could we find a way to directly support programs compiled with clang?
- Maybe one day, tail calls?
- JIT-compilers for other architectures?
- ...

License

Following the effort of the Rust language project itself in order to ease integration with other projects, the rbpf crate is distributed under the terms of both the MIT license and the Apache License (Version 2.0).

See LICENSE-APACHE and LICENSE-MIT for details.

Inspired by

- uBPF, a C user-space implementation of an eBPF virtual machine, with a JIT-compiler and disassembler (and also including the assembler from the human-readable form of the instructions, such as in `mov r0, 0x1337`), by Rich Lane for Big Switch Networks (2015)
- *Building a simple JIT in Rust*, by Jonathan Turner (2015)

Solana

Virtual machine

github.com

README.md

What about program validation?

The "verifier" of this crate is very short and has nothing to do with the kernel verifier, which means that it accepts programs that may not be safe. On the other hand, you probably do not run this in a kernel here, so it will not crash your system. Implementing a verifier similar to the one in the kernel is not trivial, and we cannot "copy" it since it is under GPL license.

What about safety then?

Rust has a strong emphasis on safety. Yet to have the eBPF VM work, some `unsafe` blocks of code are used. The VM, taken as an eBPF interpreter, can return an error but should not crash. Please file an issue otherwise.

As for the JIT-compiler, it is a different story, since runtime memory checks are more complicated to implement in assembly. It *will* crash if your JIT-compiled program tries to perform unauthorized memory accesses. Usually, it could be a good idea to test your program with the interpreter first.

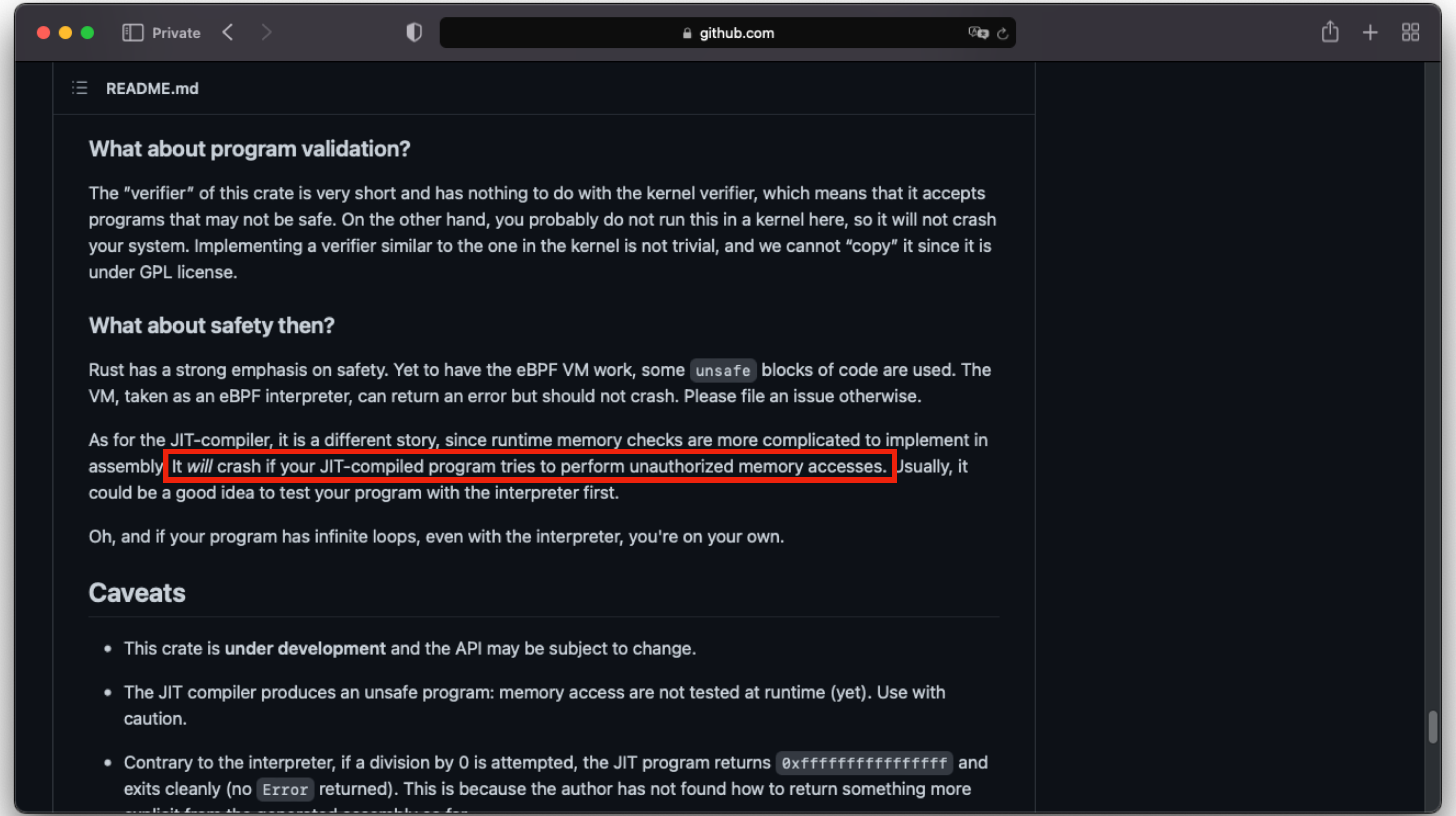
Oh, and if your program has infinite loops, even with the interpreter, you're on your own.

Caveats

- This crate is **under development** and the API may be subject to change.
- The JIT compiler produces an unsafe program: memory access are not tested at runtime (yet). Use with caution.
- Contrary to the interpreter, if a division by 0 is attempted, the JIT program returns `0xffffffffffffffff` and exits cleanly (no `Error` returned). This is because the author has not found how to return something more

Solana

Virtual machine


github.com

README.md

What about program validation?

The "verifier" of this crate is very short and has nothing to do with the kernel verifier, which means that it accepts programs that may not be safe. On the other hand, you probably do not run this in a kernel here, so it will not crash your system. Implementing a verifier similar to the one in the kernel is not trivial, and we cannot "copy" it since it is under GPL license.

What about safety then?

Rust has a strong emphasis on safety. Yet to have the eBPF VM work, some `unsafe` blocks of code are used. The VM, taken as an eBPF interpreter, can return an error but should not crash. Please file an issue otherwise.

As for the JIT-compiler, it is a different story, since runtime memory checks are more complicated to implement in assembly. $\boxed{\text{It } \textit{will} \text{ crash if your JIT-compiled program tries to perform unauthorized memory accesses.}}$ Usually, it could be a good idea to test your program with the interpreter first.

Oh, and if your program has infinite loops, even with the interpreter, you're on your own.

Caveats

- This crate is **under development** and the API may be subject to change.
- The JIT compiler produces an unsafe program: memory access are not tested at runtime (yet). Use with caution.
- Contrary to the interpreter, if a division by 0 is attempted, the JIT program returns `0xffffffffffffffff` and exits cleanly (no `Error` returned). This is because the author has not found how to return something more

Solana

Virtual machine

github.com

README.md

What about program validation?

The "verifier" of this crate is very short and has nothing to do with the kernel verifier, which means that it accepts programs that may not be safe. On the other hand, you probably do not run this in a kernel here, so it will not crash your system. Implementing a verifier similar to the one in the kernel is not trivial, and we cannot "copy" it since it is under GPL license.

What about safety then?

Rust has a strong emphasis on safety. Yet to have the eBPF VM work, some `unsafe` blocks of code are used. The VM, taken as an eBPF interpreter, can return an error but should not crash. Please file an issue otherwise.

As for the JIT-compiler, it is a different story, since runtime memory checks are more complicated to implement in assembly. It *will* crash if your JIT-compiled program tries to perform unauthorized memory accesses. Usually, it could be a good idea to test your program with the interpreter first.

Oh, and if your program has infinite loops, even with the interpreter, you're on your own.

Caveats

- This crate is **under development** and the API may be subject to change.
- The JIT compiler produces an unsafe program: memory access are not tested at runtime (yet). Use with caution.
- Contrary to the interpreter, if a division by 0 is attempted, the JIT program returns `0xffffffffffffffff` and exits cleanly (no `Error` returned). This is because the author has not found how to return something more

kraken

Let's get fuzzing!

Fuzzing the Solana VM

Simpe eBPF program

kraken

Fuzzing the Solana VM

Simpe eBPF program → Mutate it semi-randomly

Fuzzing the Solana VM

Simpe eBPF program → Mutate it semi-randomly → Execute it in the Solana VM

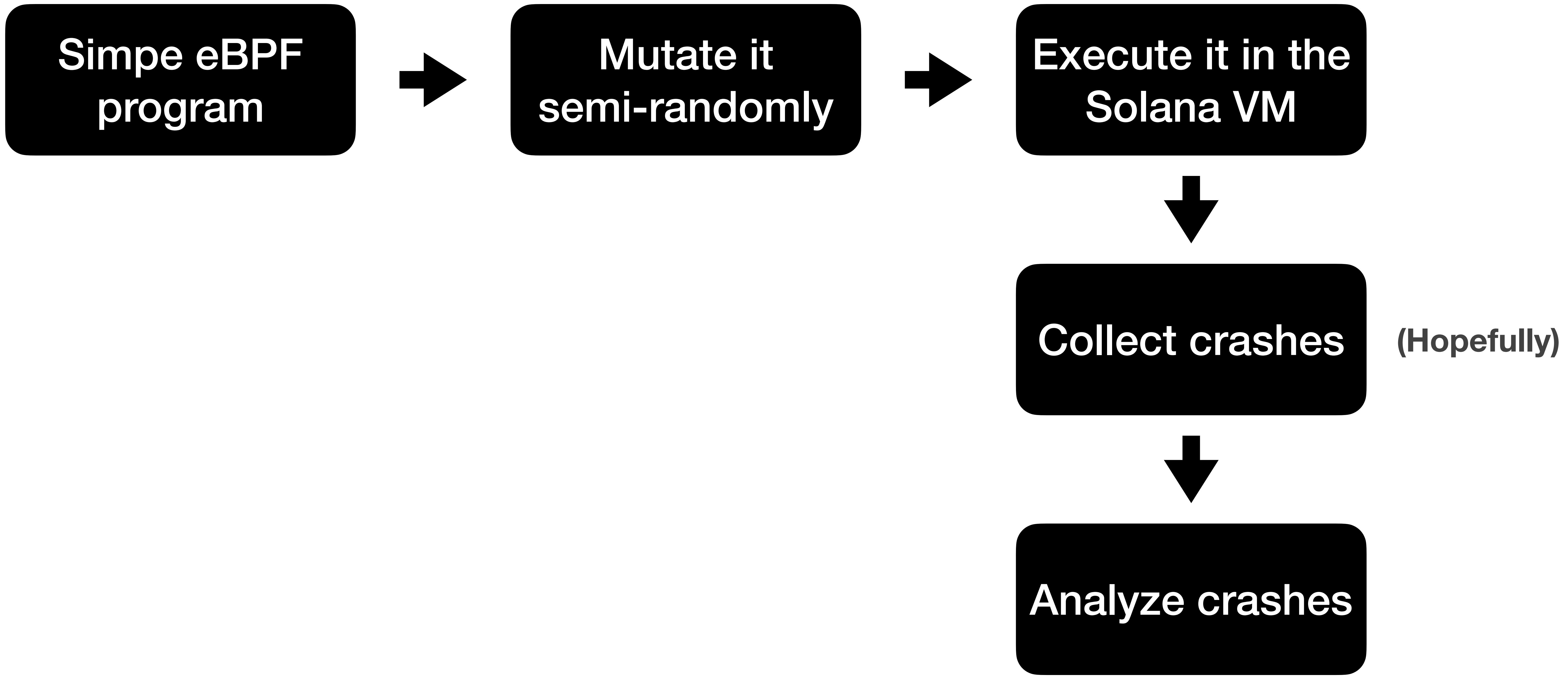
kraken

Fuzzing the Solana VM

Fuzzing the Solana VM

Simpe eBPF program → Mutate it semi-randomly → Execute it in the Solana VM → Collect crashes (Hopefully)

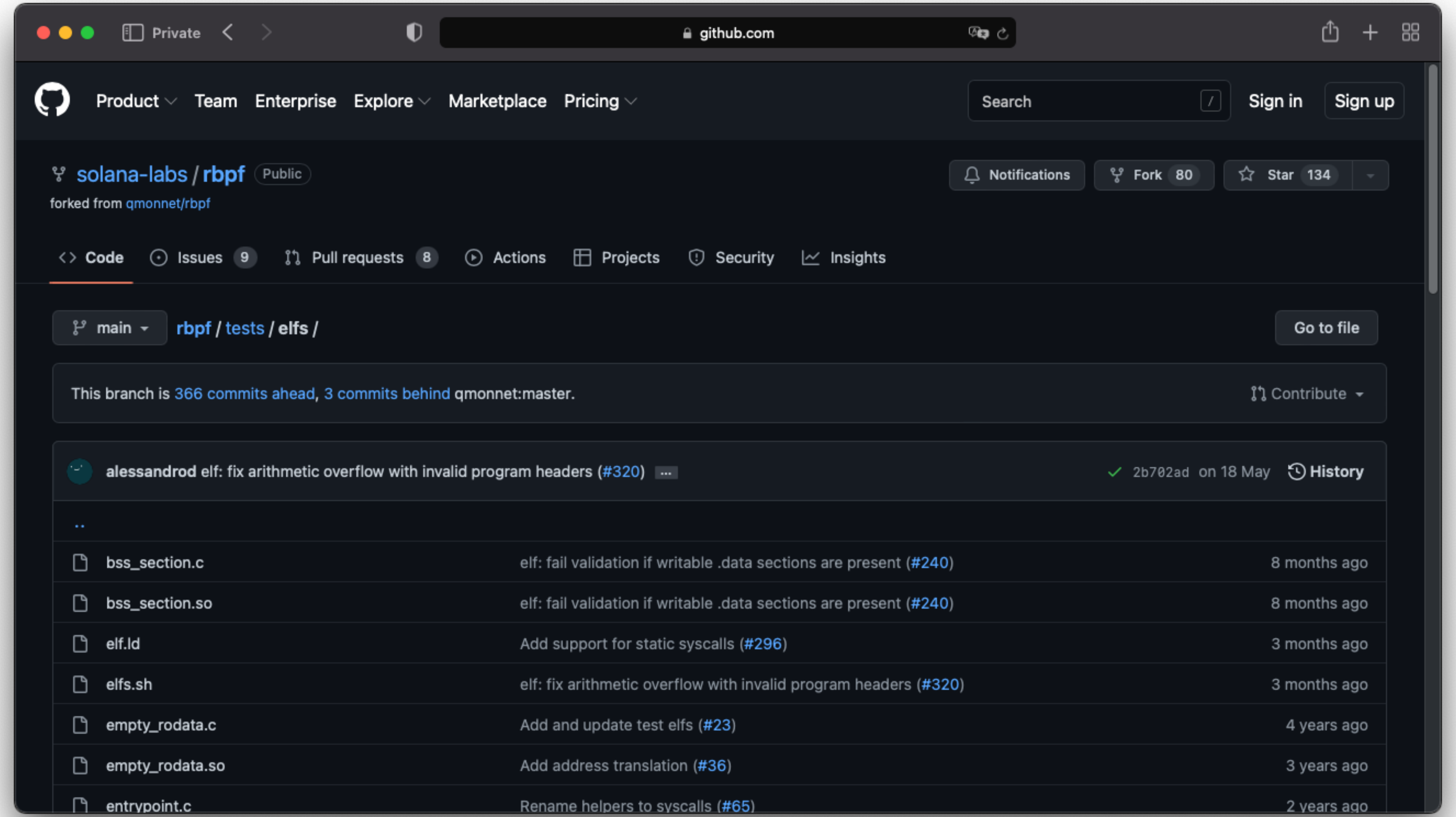
Fuzzing the Solana VM



Fuzzing the Solana VM

Simpe eBPF
program

Fuzzing the Solana VM



kraken

Fuzzing the Solana VM

Simpe eBPF program

Fuzzing the Solana VM

Simpe eBPF program

How can we analyze them?

Fuzzing the Solana VM

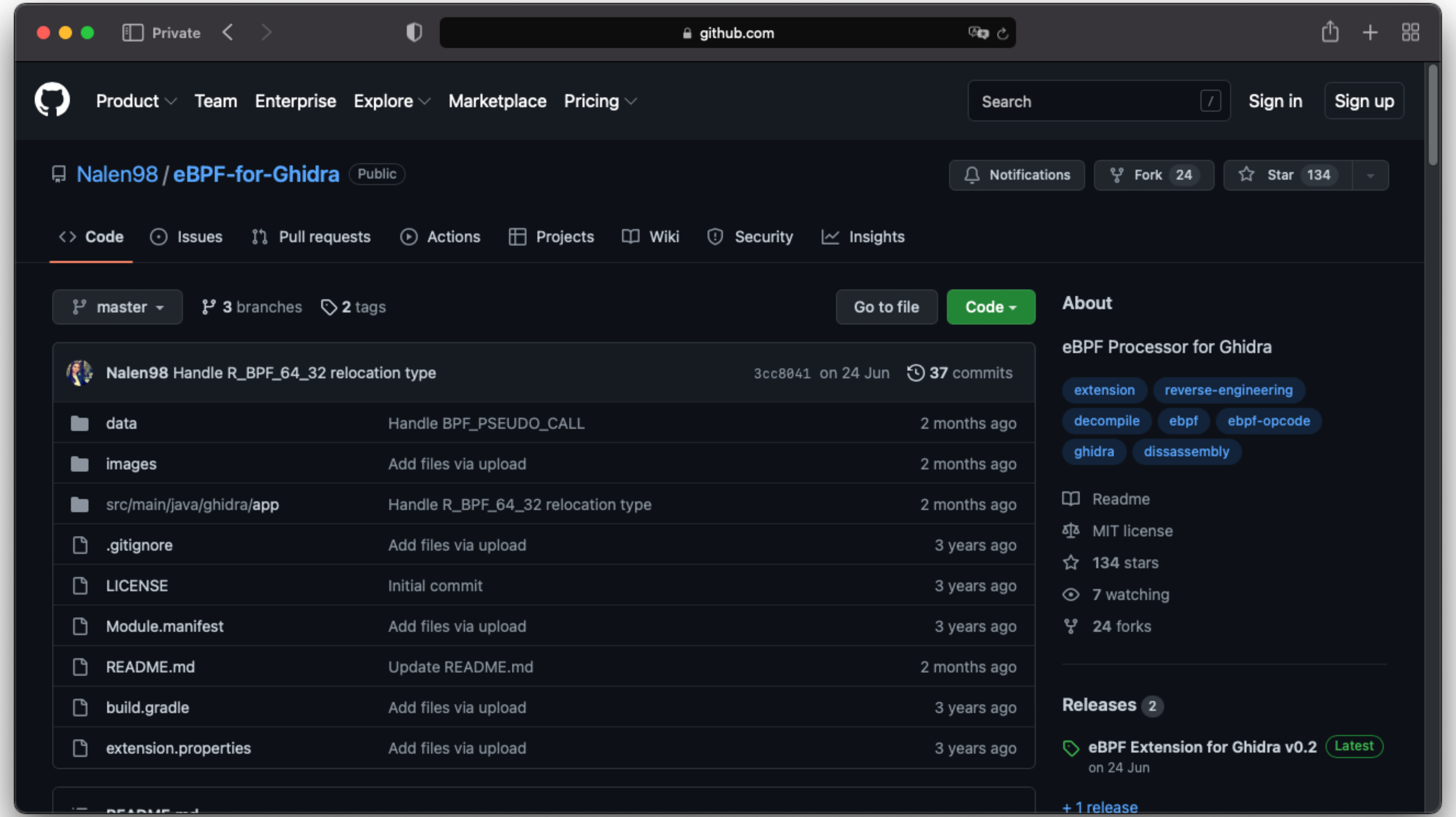
Simpe eBPF program

How can we analyze them?

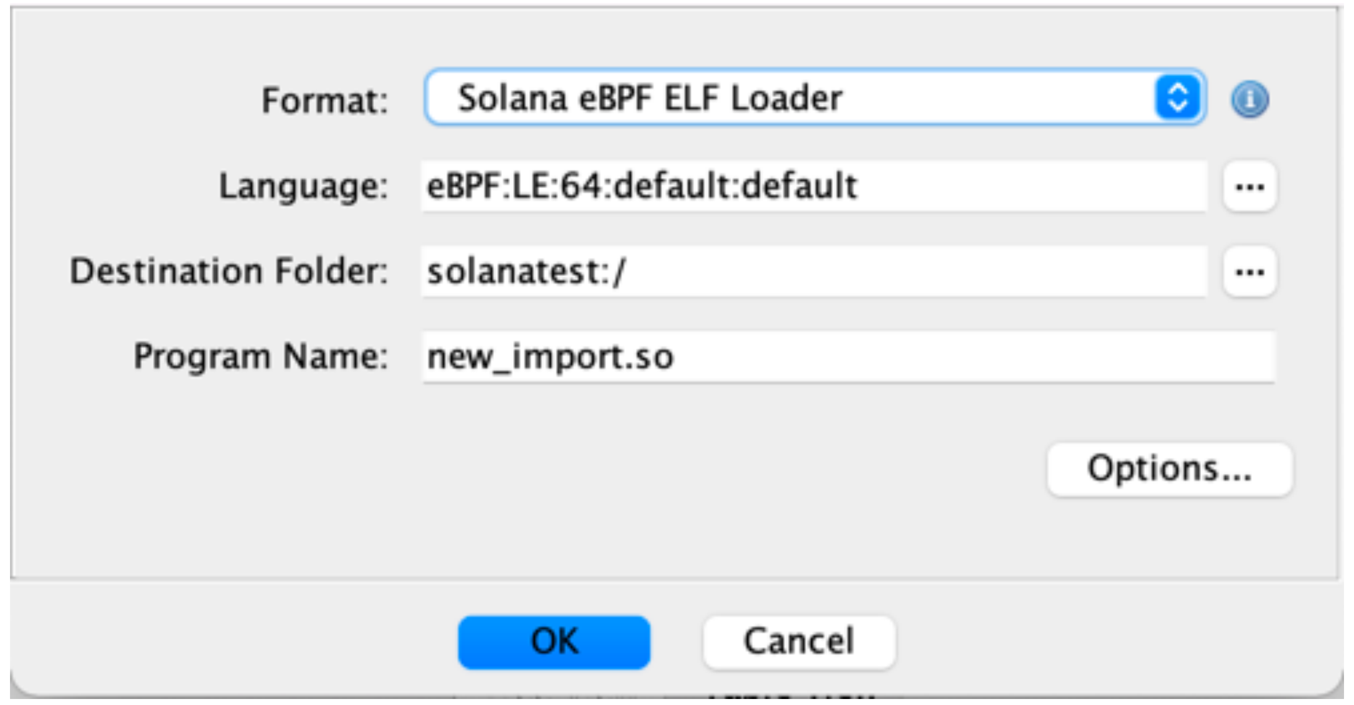
No good reverse-engineering tooling for Solana binaries...

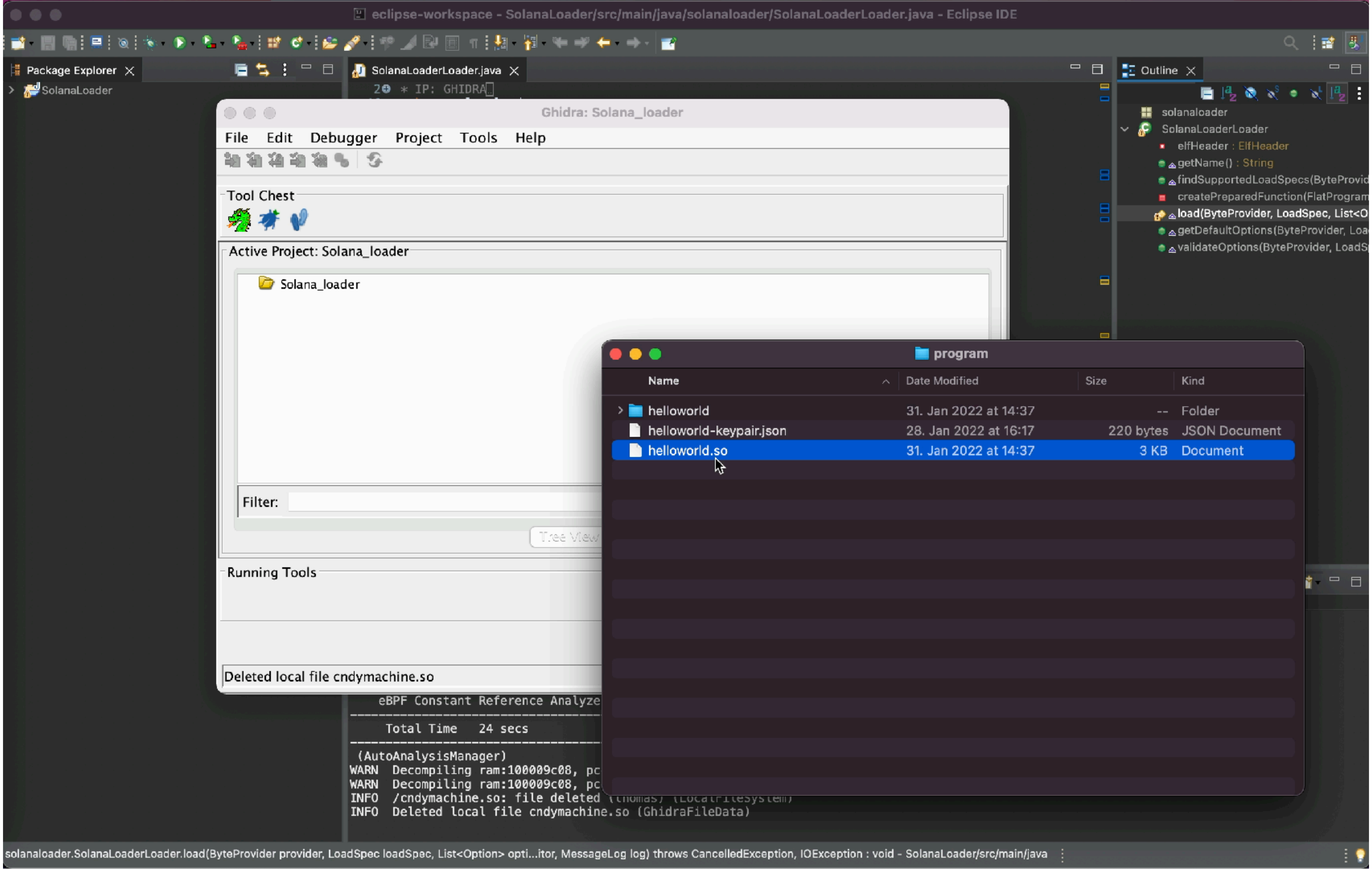
Analyzing Solana binaries

Analyzing Solana binaries

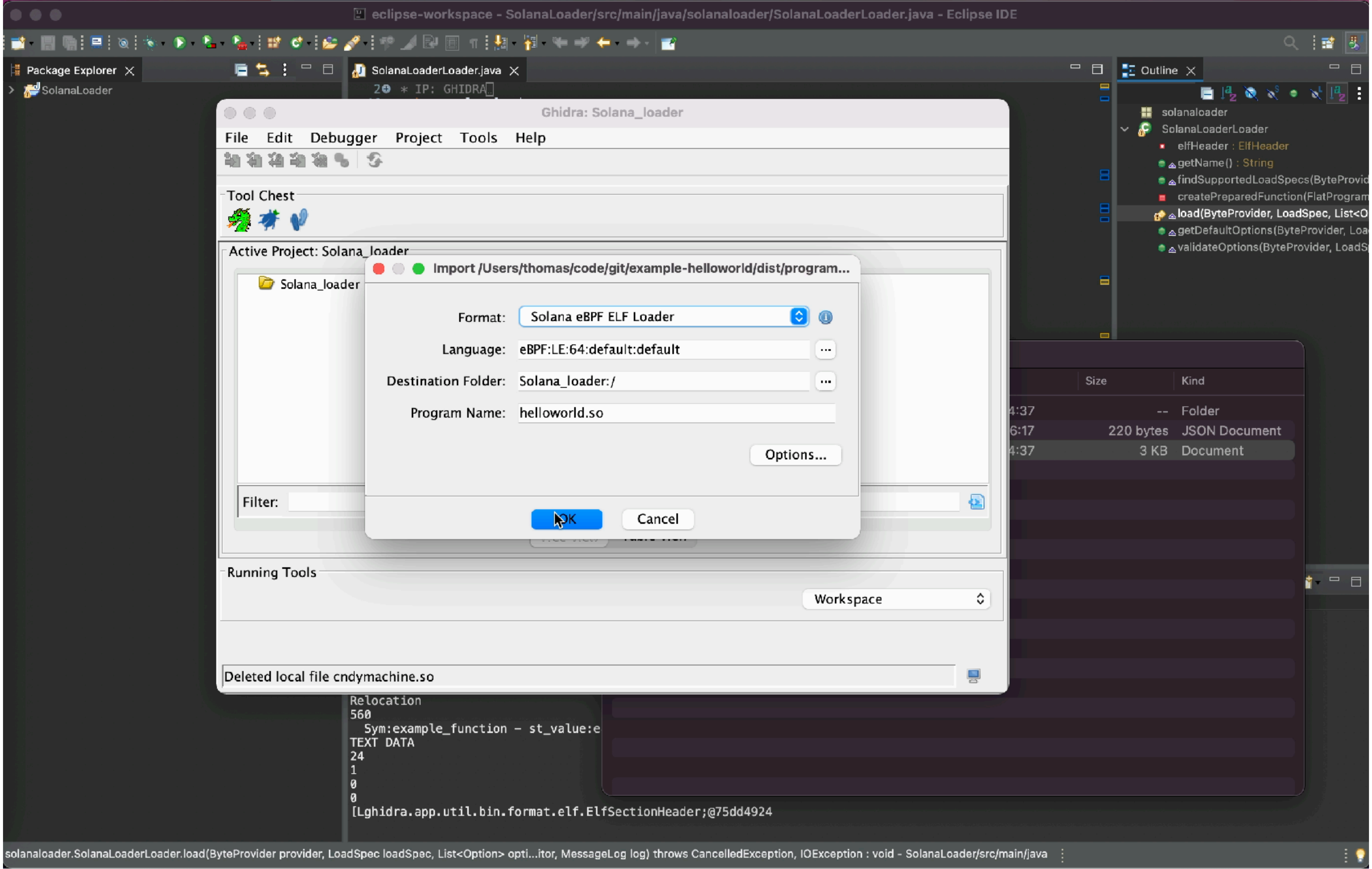


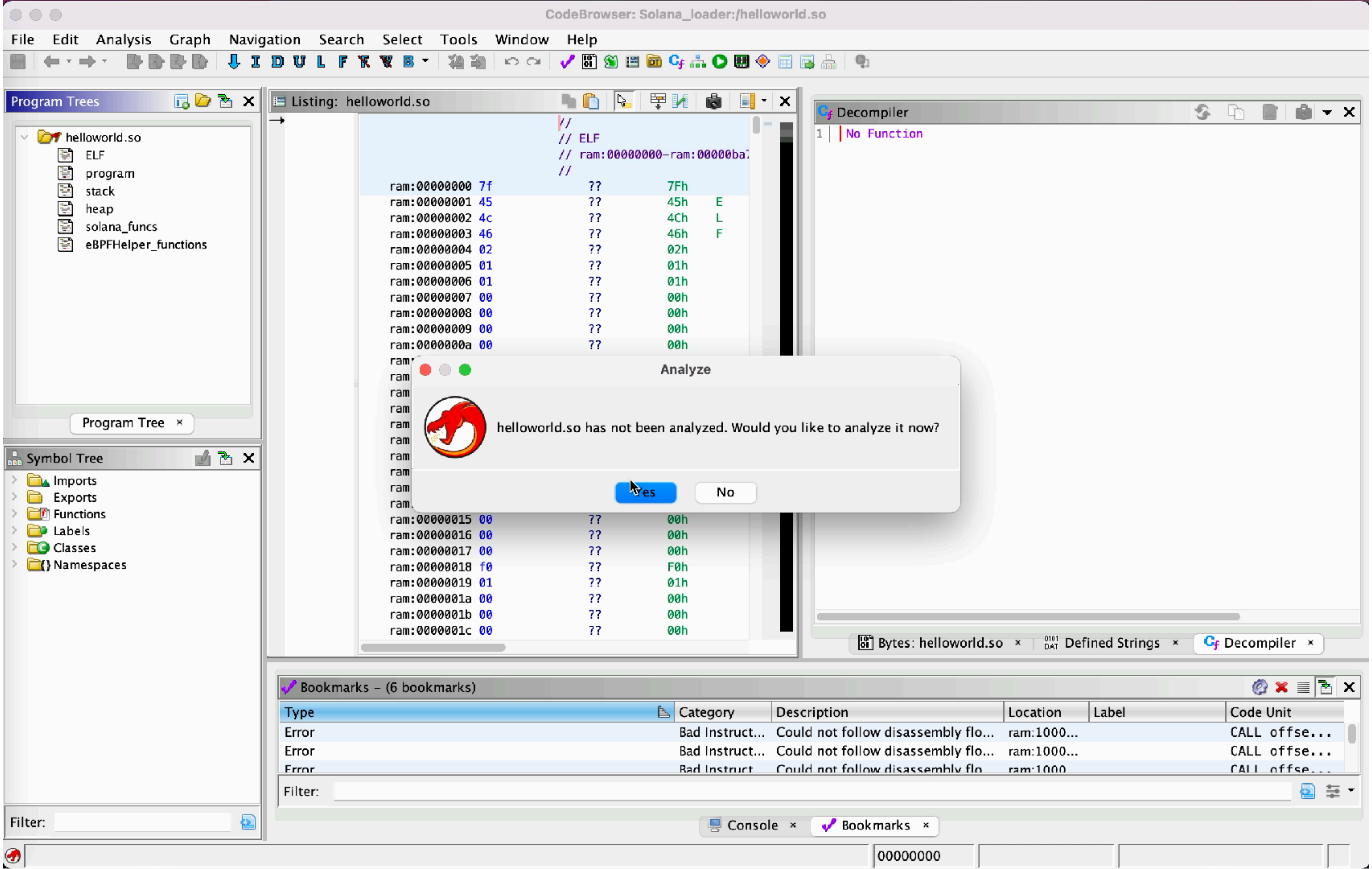
Analyzing Solana binaries





eclipse-workspace - SolanaLoader/src/main/java/solanaloader/SolanaLoaderLoader.java - Eclipse IDE
Package Explorer
SolanaLoader
SolanaLoaderLoader.java
Outline
solanaloader
SolanaLoaderLoader
elfHeader : ElfHeader
getName() : String
findSupportedLoadSpecs(ByteProvid
createPreparedFunction(FlatProgram
load(ByteProvider, LoadSpec, List<O
getDefaultOptions(ByteProvider, Loa
validateOptions(ByteProvider, LoadS
Ghidra: Solana_loader
File
Edit
Debugger
Project
Tools
Help
Tool Chest
Active Project: Solana_loader
Solana_loader
Filter:
Tree View
Running Tools
Deleted local file cndymachine.so
program
Name
Date Modified
Size
Kind
helloworld
31. Jan 2022 at 14:37
--
Folder
helloworld-keypair.json
28. Jan 2022 at 16:17
220 bytes
JSON Document
helloworld.so
31. Jan 2022 at 14:37
3 KB
Document
eBPF Constant Reference Analyze
Total Time 24 secs
(AutoAnalysisManager)
WARN Decompiling ram:100009c08, pc
WARN Decompiling ram:100009c08, pc
INFO /cndymachine.so: file deleted
INFO Deleted local file cndymachine.so (GhidraFileData)
solanaloader.SolanaLoaderLoader.load(ByteProvider provider, LoadSpec loadSpec, List<Option> opti...itor, MessageLog log) throws CancelledException, IOException : void - SolanaLoader/src/main/java

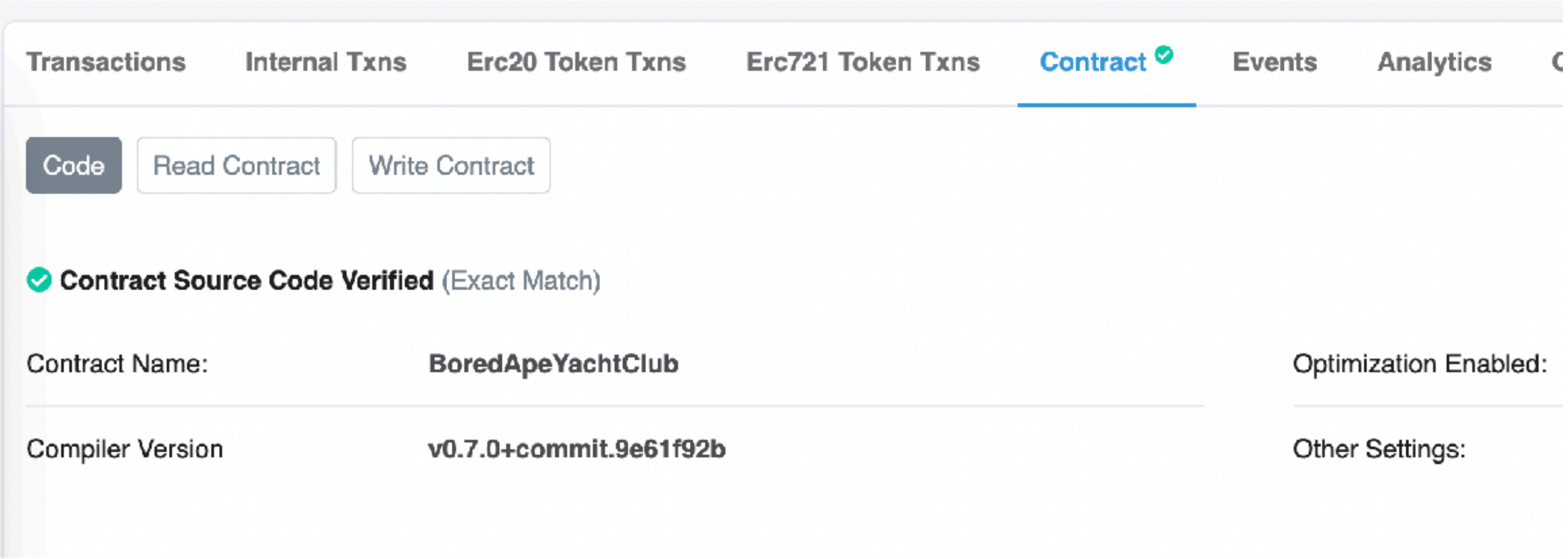

eclipse-workspace - SolanaLoader/src/main/java/solanaloader/SolanaLoaderLoader.java - Eclipse IDE
Package Explorer
SolanaLoader
SolanaLoaderLoader.java
Outline
solanaloader
SolanaLoaderLoader
elfHeader : ElfHeader
getName() : String
findSupportedLoadSpecs(ByteProvid
createPreparedFunction(FlatProgram
load(ByteProvider, LoadSpec, List<O
getDefaultOptions(ByteProvider, Loa
validateOptions(ByteProvider, LoadS
Ghidra: Solana_loader
File
Edit
Debugger
Project
Tools
Help
Tool Chest
Active Project: Solana_loader
Solana_loader
Filter:
Import /Users/thomas/code/git/example-helloworld/dist/program...
Format:
Solana eBPF ELF Loader
Language:
eBPF:LE:64:default:default
Destination Folder:
Solana_loader:/
Program Name:
helloworld.so
Options...
OK
Cancel
Running Tools
Workspace
Deleted local file cndymachine.so
Size
Kind
4:37
--
Folder
6:17
220 bytes
JSON Document
4:37
3 KB
Document
Relocation
560
Sym:example_function - st_value:e
TEXT DATA
24
1
0
0
[Lghidra.app.util.bin.format.elf.ElfSectionHeader;@75dd4924
solanaloader.SolanaLoaderLoader.load(ByteProvider provider, LoadSpec loadSpec, List<Option> opti...itor, MessageLog log) throws CancelledException, IOException : void - SolanaLoader/src/main/java


CodeBrowser: Solana_loader:/helloworld.so
File Edit Analysis Graph Navigation Search Select Tools Window Help
Program Trees
helloworld.so
ELF
program
stack
heap
solana_funcs
eBPFHelper_functions
Program Tree
Listing: helloworld.so
// ELF
// ram:00000000-ram:00000ba7
ram:00000000 7f ?? 7Fh
ram:00000001 45 ?? 45h E
ram:00000002 4c ?? 4Ch L
ram:00000003 46 ?? 46h F
ram:00000004 02 ?? 02h
ram:00000005 01 ?? 01h
ram:00000006 01 ?? 01h
ram:00000007 00 ?? 00h
ram:00000008 00 ?? 00h
ram:00000009 00 ?? 00h
ram:0000000a 00 ?? 00h
ram:00000015 00 ?? 00h
ram:00000016 00 ?? 00h
ram:00000017 00 ?? 00h
ram:00000018 f0 ?? F0h
ram:00000019 01 ?? 01h
ram:0000001a 00 ?? 00h
ram:0000001b 00 ?? 00h
ram:0000001c 00 ?? 00h
Decompiler
1 No Function
Analyze
helloworld.so has not been analyzed. Would you like to analyze it now?
Yes
No
Symbol Tree
Imports
Exports
Functions
Labels
Classes
Namespaces
Filter:
Bytes: helloworld.so
Defined Strings
Decompiler
Bookmarks - (6 bookmarks)
Type Category Description Location Label Code Unit
Error Bad Instruct... Could not follow disassembly flo... ram:1000... CALL offse...
Error Bad Instruct... Could not follow disassembly flo... ram:1000... CALL offse...
Filter:
Console
Bookmarks
00000000

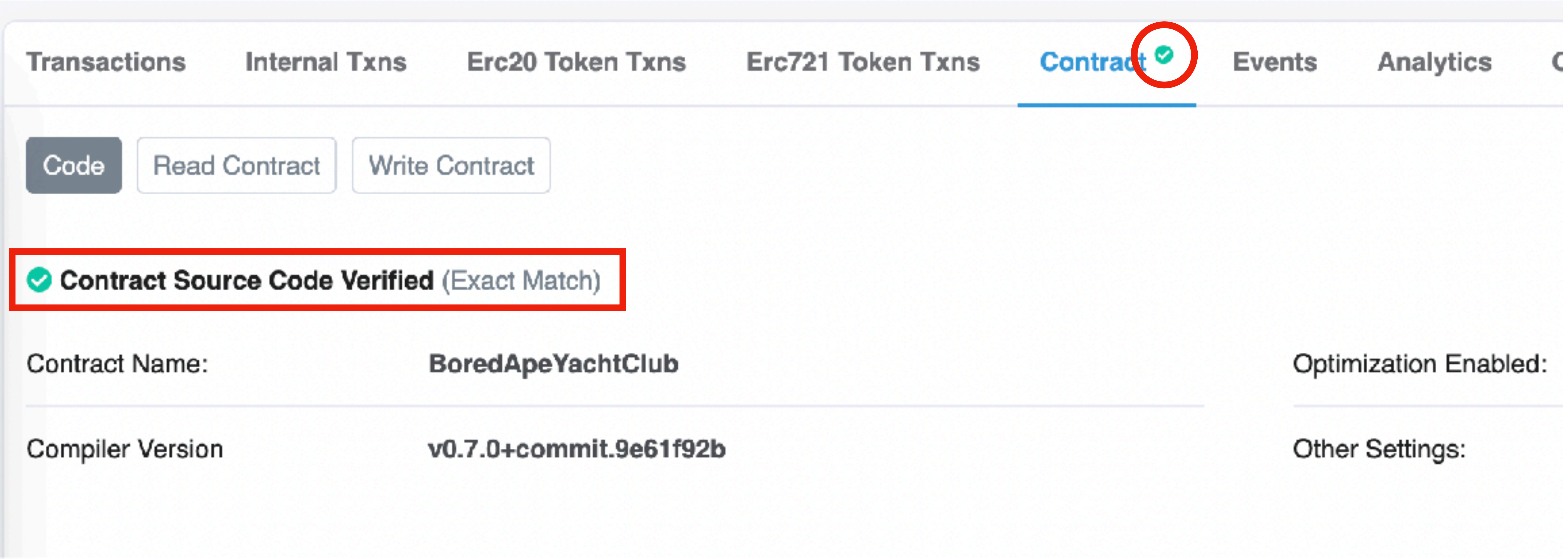
Solana
Smart-contracts

Verifiable Builds

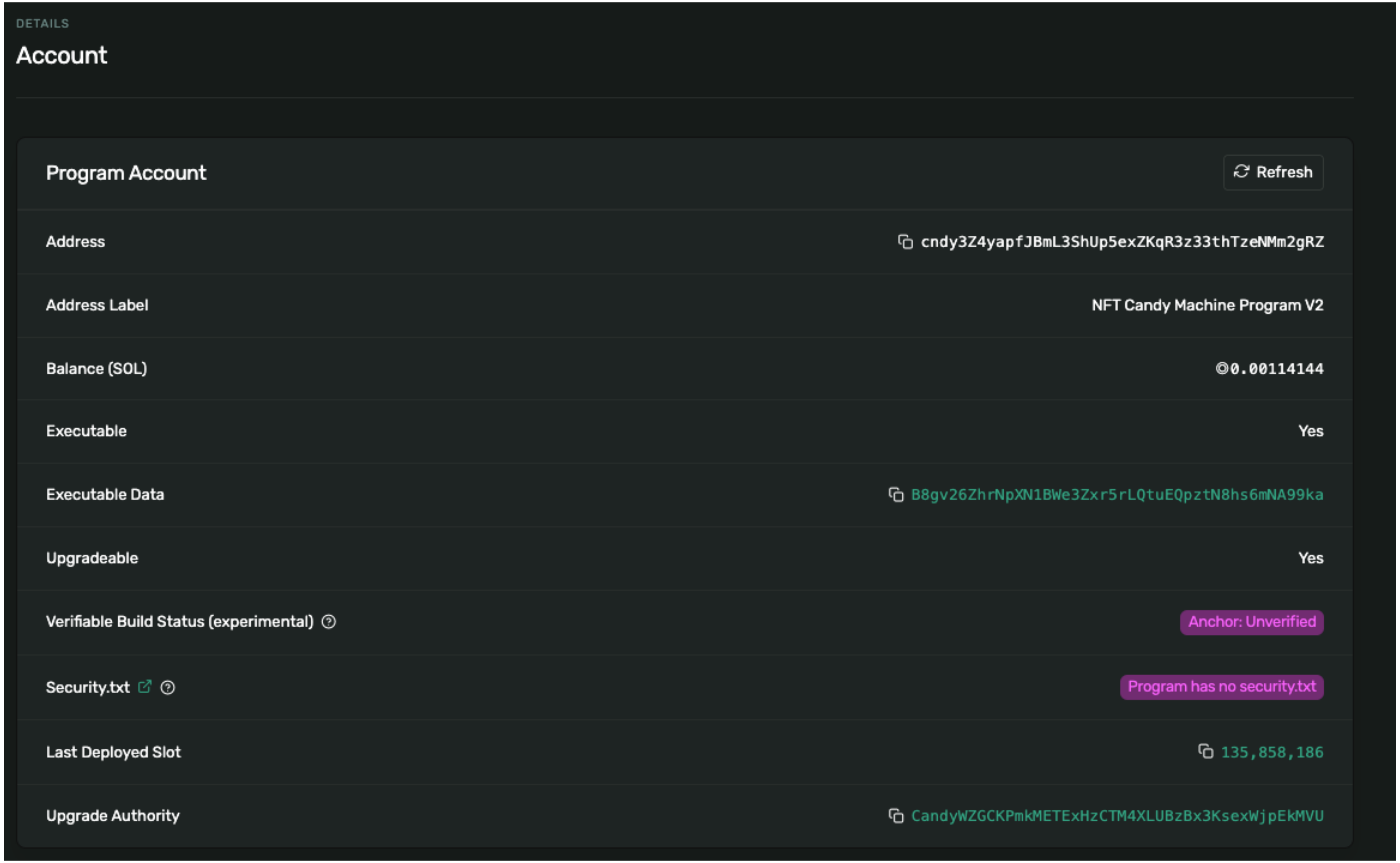
Verifiable builds



Verifiable builds

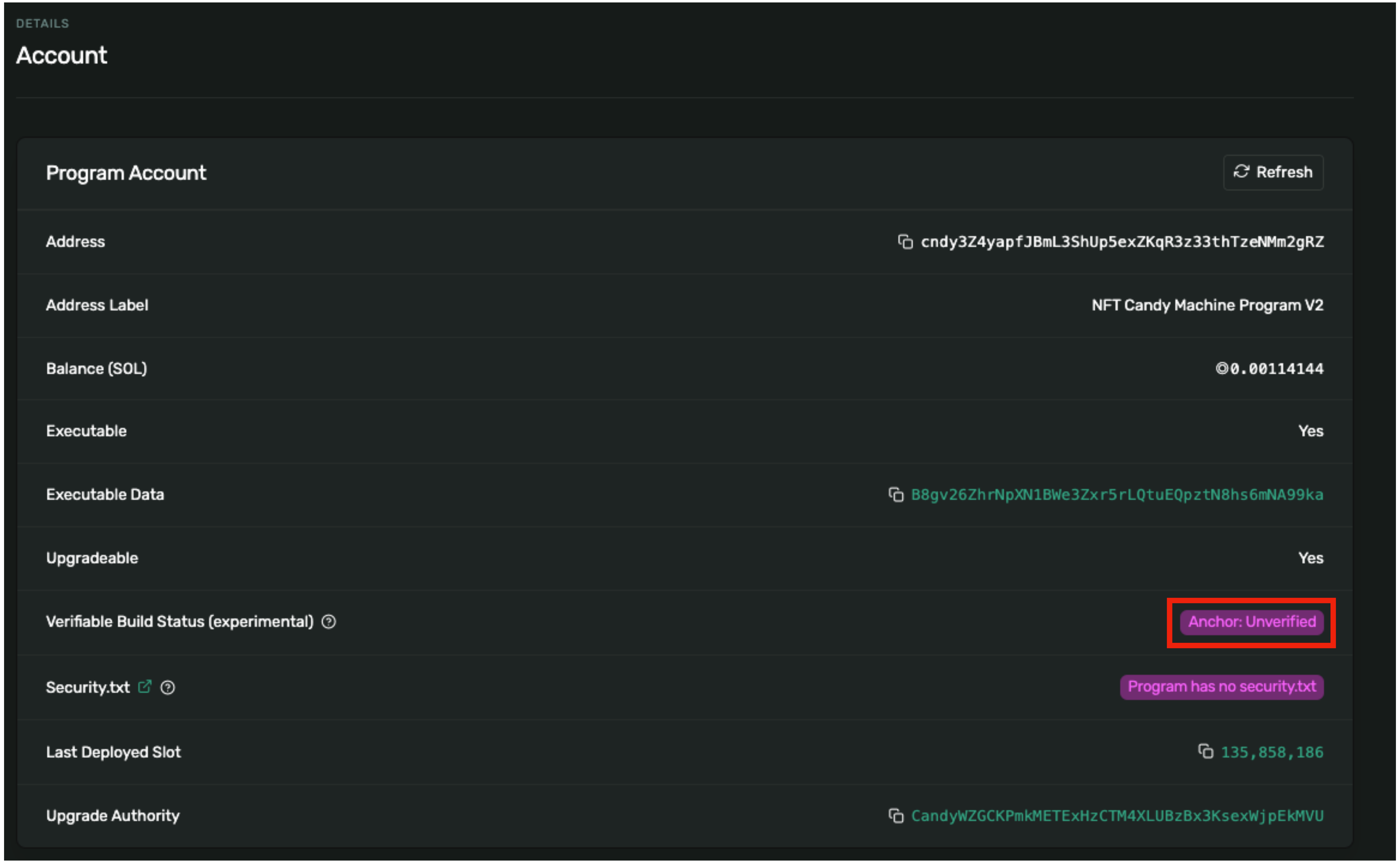


Verifiable builds

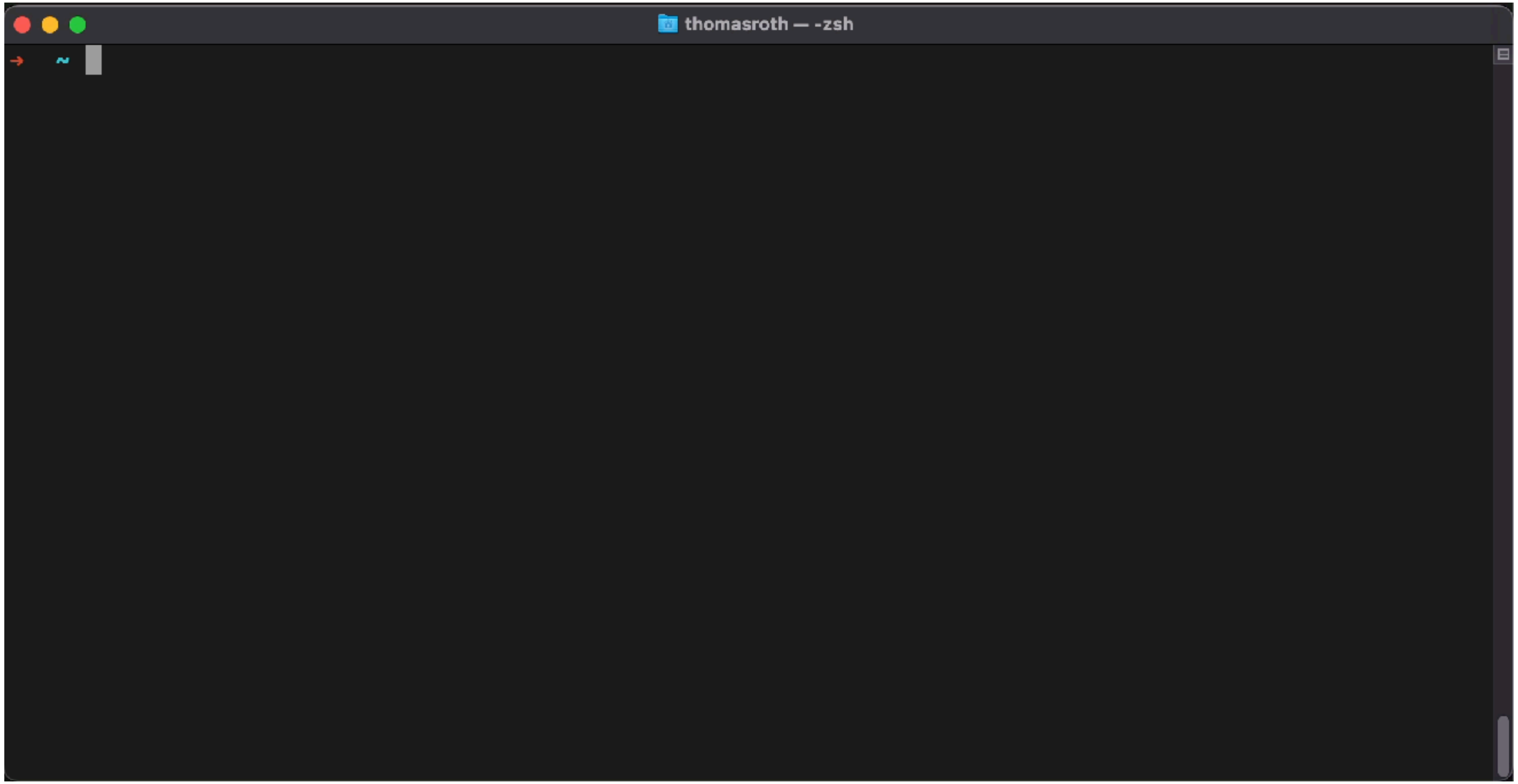


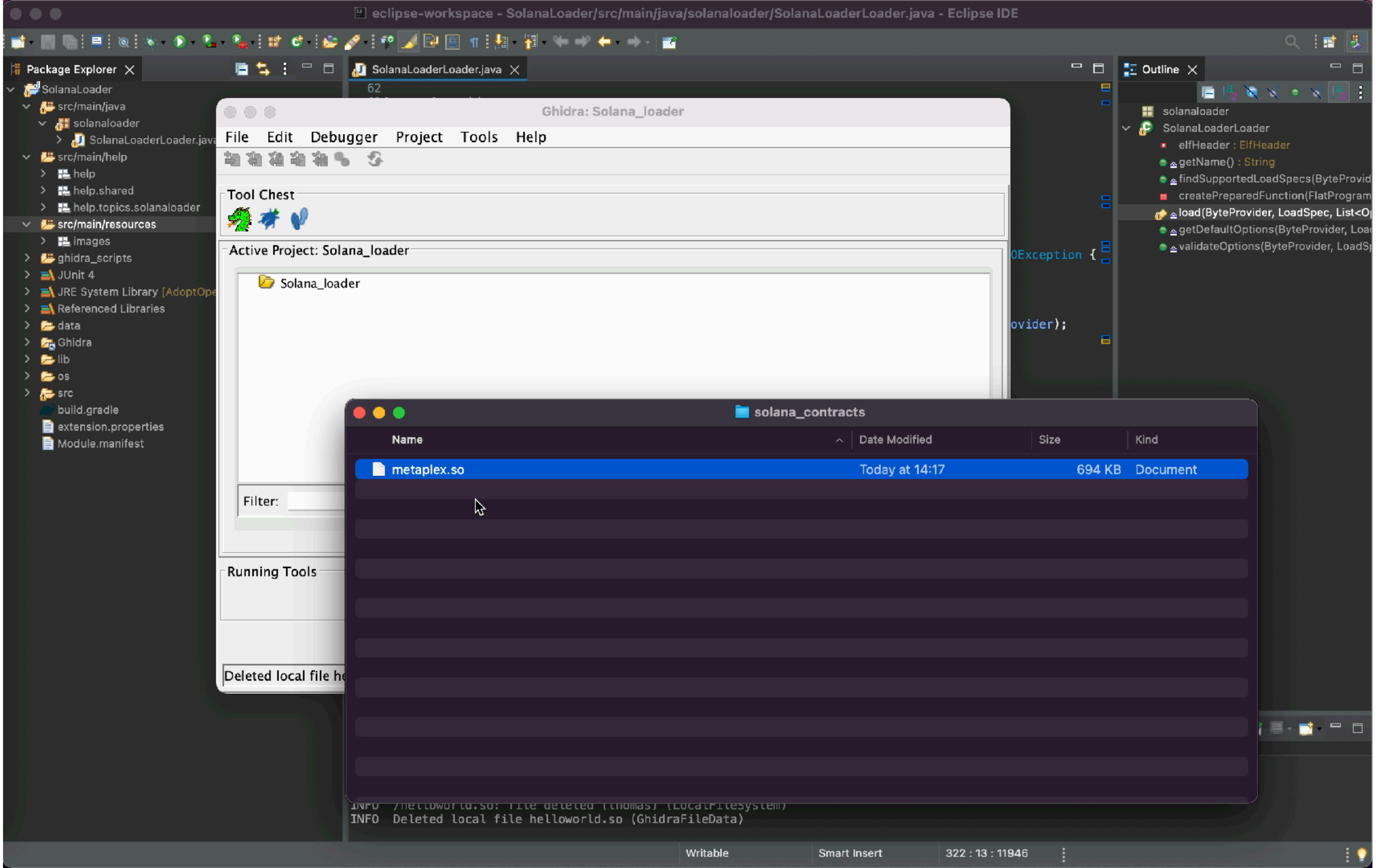
kraken

Verifiable builds

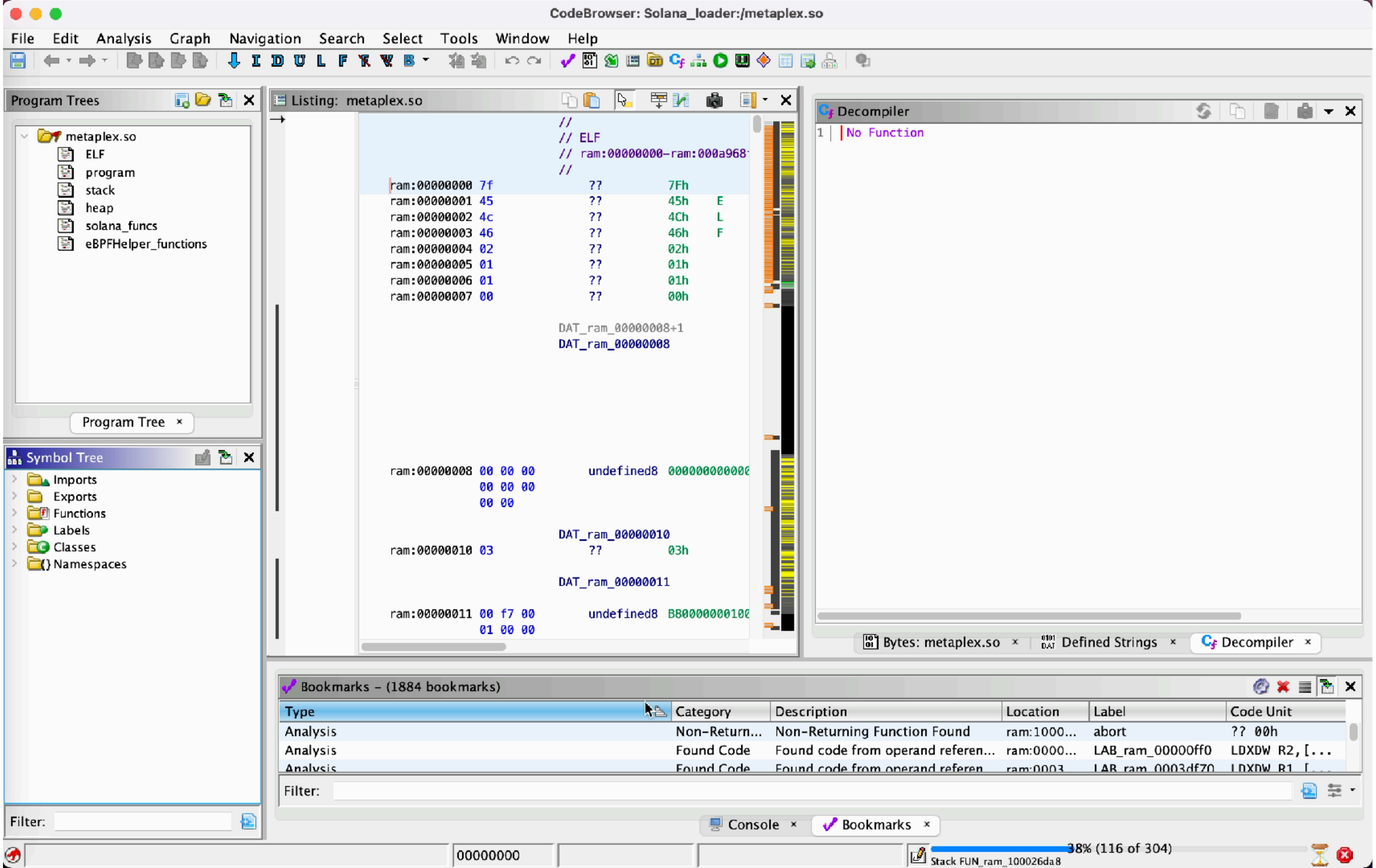


Verifiable builds





eclipse-workspace - SolanaLoader/src/main/java/solanaloader/SolanaLoaderLoader.java - Eclipse IDE
Package Explorer
SolanaLoader
src/main/java
solanaloader
SolanaLoaderLoader.java
src/main/help
help
help.shared
help.topics.solanaloader
src/main/resources
images
ghidra_scripts
JUnit 4
JRE System Library [AdoptOpe
Referenced Libraries
data
Ghidra
lib
os
src
build.gradle
extension.properties
Module.manifest
SolanaLoaderLoader.java
62
OException {
ovider);
Outline
solanaloader
SolanaLoaderLoader
elfHeader : ElfHeader
getName() : String
findSupportedLoadSpecs(ByteProvid
createPreparedFunction(FlatProgram
load(ByteProvider, LoadSpec, List<O
getDefaultOptions(ByteProvider, Loa
validateOptions(ByteProvider, LoadS
Ghidra: Solana_loader
File
Edit
Debugger
Project
Tools
Help
Tool Chest
Active Project: Solana_loader
Solana_loader
Filter:
Running Tools
Deleted local file he
solana_contracts
Name
Date Modified
Size
Kind
metaplex.so
Today at 14:17
694 KB
Document
INFO /helloworld.so: file deleted (thomas) (LocalFileSystem)
INFO Deleted local file helloworld.so (GhidraFileData)
Writable
Smart Insert
322 : 13 : 11946


CodeBrowser: Solana_loader:/metaplex.so
File Edit Analysis Graph Navigation Search Select Tools Window Help
Program Trees
metaplex.so
ELF
program
stack
heap
solana_funcs
eBPFHelper_functions
Program Tree
Symbol Tree
Imports
Exports
Functions
Labels
Classes
Namespaces
Filter:
Listing: metaplex.so
//
// ELF
// ram:00000000-ram:000a968
//
ram:00000000 7f ?? 7Fh
ram:00000001 45 ?? 45h E
ram:00000002 4c ?? 4Ch L
ram:00000003 46 ?? 46h F
ram:00000004 02 ?? 02h
ram:00000005 01 ?? 01h
ram:00000006 01 ?? 01h
ram:00000007 00 ?? 00h
DAT_ram_00000008+1
DAT_ram_00000008
ram:00000008 00 00 00 undefined8 00000000000
00 00 00
00 00
DAT_ram_00000010
ram:00000010 03 ?? 03h
DAT_ram_00000011
ram:00000011 00 f7 00 undefined8 BB000000010
01 00 00
Decompiler
1 No Function
Bytes: metaplex.so
Defined Strings
Decompiler
Bookmarks – (1884 bookmarks)
Type Category Description Location Label Code Unit
Analysis Non-Return... Non-Returning Function Found ram:1000... abort ?? 00h
Analysis Found Code Found code from operand referen... ram:0000... LAB_ram_00000ff0 LDXDW R2,[...
Analysis Found Code Found code from operand referen... ram:0003... LAB_ram_0003df70 LDXDW R1,[...
Filter:
Console
Bookmarks
00000000
38% (116 of 304)
Stack FUN_ram_100026da8

Fuzzing the Solana VM

Fuzzing the Solana VM

Simpe eBPF program → Mutate it semi-randomly → Execute it in the Solana VM

Fuzzing the Solana VM

Simpe eBPF program → Mutate it semi-randomly → Execute it in the Solana VM

```
fn main() {
```

```rust
fn main() {
        let args: Vec<String> = env::args().collect();
        let query = &args[1];
        let mut file = File::open(query).unwrap();
        let mut elf = Vec::new();
        file.read_to_end(&mut elf).unwrap();
        let data: &[u8] = &elf[..];
```

```rust
fn main() {
    let args: Vec<String> = env::args().collect();
    let query = &args[1];
    let mut file = File::open(query).unwrap();
    let mut elf = Vec::new();
    file.read_to_end(&mut elf).unwrap();
    let data: &[u8] = &elf[..];

    let mut executable = Executable::<UserError, TestInstructionMeter>::from_elf(
        &data,
        Config::default(),
        SyscallRegistry::default(),
    ).unwrap();
```

```rust
fn main() {
        let args: Vec<String> = env::args().collect();
        let query = &args[1];
        let mut file = File::open(query).unwrap();
        let mut elf = Vec::new();
        file.read_to_end(&mut elf).unwrap();
        let data: &[u8] = &elf[..];

        let mut executable = Executable::<UserError, TestInstructionMeter>::from_elf(
            &data,
            Config::default(),
            SyscallRegistry::default(),
        ).unwrap();

        Executable::<UserError, TestInstructionMeter>::jit_compile(&mut executable).unwrap();

```

```rust
fn main() {
        let args: Vec<String> = env::args().collect();
        let query = &args[1];
        let mut file = File::open(query).unwrap();
        let mut elf = Vec::new();
        file.read_to_end(&mut elf).unwrap();
        let data: &[u8] = &elf[..];

        let mut executable = Executable::<UserError, TestInstructionMeter>::from_elf(
            &data,
            Config::default(),
            SyscallRegistry::default(),
        ).unwrap();

        Executable::<UserError, TestInstructionMeter>::jit_compile(&mut executable).unwrap();

        let verified_executable = VerifiedExecutable
                        ::<EmptyVerified, UserError, TestInstructionMeter>
                        ::from_executable(executable).unwrap();
```

```rust
fn main() {
        let args: Vec<String> = env::args().collect();
        let query = &args[1];
        let mut file = File::open(query).unwrap();
        let mut elf = Vec::new();
        file.read_to_end(&mut elf).unwrap();
        let data: &[u8] = &elf[..];

        let mut executable = Executable::<UserError, TestInstructionMeter>::from_elf(
            &data,
            Config::default(),
            SyscallRegistry::default(),
        ).unwrap();

        Executable::<UserError, TestInstructionMeter>::jit_compile(&mut executable).unwrap();

        let verified_executable = VerifiedExecutable
                        ::<EmptyVerified, UserError, TestInstructionMeter>
                        ::from_executable(executable).unwrap();

        let mut vm = EbpfVm::new(&verified_executable, &mut [], vec![]).unwrap();
```

```rust
fn main() {
        let args: Vec<String> = env::args().collect();
        let query = &args[1];
        let mut file = File::open(query).unwrap();
        let mut elf = Vec::new();
        file.read_to_end(&mut elf).unwrap();
        let data: &[u8] = &elf[..];

        let mut executable = Executable::<UserError, TestInstructionMeter>::from_elf(
            &data,
            Config::default(),
            SyscallRegistry::default(),
        ).unwrap();

        Executable::<UserError, TestInstructionMeter>::jit_compile(&mut executable).unwrap();

        let verified_executable = VerifiedExecutable
                                ::<EmptyVerified, UserError, TestInstructionMeter>
                                ::from_executable(executable).unwrap();

        let mut vm = EbpfVm::new(&verified_executable, &mut [], vec![]).unwrap();

        vm.execute_program_jit(&mut TestInstructionMeter { remaining: 90000 });
}
```

Fuzzing the Solana VM

Simpe eBPF program → Mutate it semi-randomly → Execute it in the Solana VM

Fuzzing the Solana VM

Simpe eBPF program → Mutate it semi-randomly → Execute it in the Solana VM

Fuzzing the Solana VM

Fuzzing the Solana VM

Fuzzing the Solana VM

AFL++

- AFL++ is a fork of Google's AFL
    - Faster
    - Better instrumentation
    - Rust support!

AFL++

- Instruments the target binary
- Automatically mutates input to generate as much coverage as possible
- Comes with nice UI
- Integrated test-case minimization

```rust
fn main() {
    let args: Vec<String> = env::args().collect();
    let query = &args[1];
    let mut file = File::open(query).unwrap();
    let mut elf = Vec::new();
    file.read_to_end(&mut elf).unwrap();
    let data: &[u8] = &elf[..];

    let mut executable = Executable::<UserError, TestInstructionMeter>::from_elf(
        &data,
        Config::default(),
        SyscallRegistry::default(),
    ).unwrap();

    Executable::<UserError, TestInstructionMeter>::jit_compile(&mut executable).unwrap();

    let verified_executable = VerifiedExecutable
        ::<EmptyVerified, UserError, TestInstructionMeter>
        ::from_executable(executable).unwrap();

    let mut vm = EbpfVm::new(&verified_executable, &mut [], vec![]).unwrap();

    vm.execute_program_jit(&mut TestInstructionMeter { remaining: 90000 });
}
```

```rust
fn main() {
    fuzz!(|data: &[u8]| {






        let mut executable = Executable::<UserError, TestInstructionMeter>::from_elf(
            &data,
            Config::default(),
            SyscallRegistry::default(),
        ).unwrap();

        Executable::<UserError, TestInstructionMeter>::jit_compile(&mut executable).unwrap();
        let verified_executable = VerifiedExecutable
                        ::<EmptyVerifier, UserError, TestInstructionMeter>
                        ::from_executable(executable).unwrap();

        let mut vm = EbpfVm::new(&verified_executable, &mut [], vec![]).unwrap();

        vm.execute_program_jit(&mut TestInstructionMeter { remaining: 90000 });
    });
}
```

```rust
fn main() {
    fuzz!(|data: &[u8]| {
        panic::set_hook(Box::new(|_| {
            unsafe {
                libc::_exit(0);
            }
        }));

        let mut executable = Executable::<UserError, TestInstructionMeter>::from_elf(
            &data,
            Config::default(),
            SyscallRegistry::default(),
        ).unwrap();

        Executable::<UserError, TestInstructionMeter>::jit_compile(&mut executable).unwrap();
        let verified_executable = VerifiedExecutable
                            ::<EmptyVerifier, UserError, TestInstructionMeter>
                            ::from_executable(executable).unwrap();

        let mut vm = EbpfVm::new(&verified_executable, &mut [], vec![]).unwrap();

        vm.execute_program_jit(&mut TestInstructionMeter { remaining: 90000 });
    });
}
```

Fuzzing the Solana VM

Simpe eBPF program → → Execute it in the Solana VM

Fuzzing the Solana VM

Simpe eBPF program → s… y → Execute it in the Solana VM

Ready to fuzz!

kraken

root@localhost: ~

```
root@localhost:~/solana_fuzz#
```

root@localhost: ~

```
root@localhost:~/solana_fuzz# cargo afl fuzz
```

root@localhost: ~

```
root@localhost:~/solana_fuzz# cargo afl fuzz -i inputs
```

root@localhost: ~

```
root@localhost:~/solana_fuzz# cargo afl fuzz -i inputs -o out_demo
```

root@localhost: ~

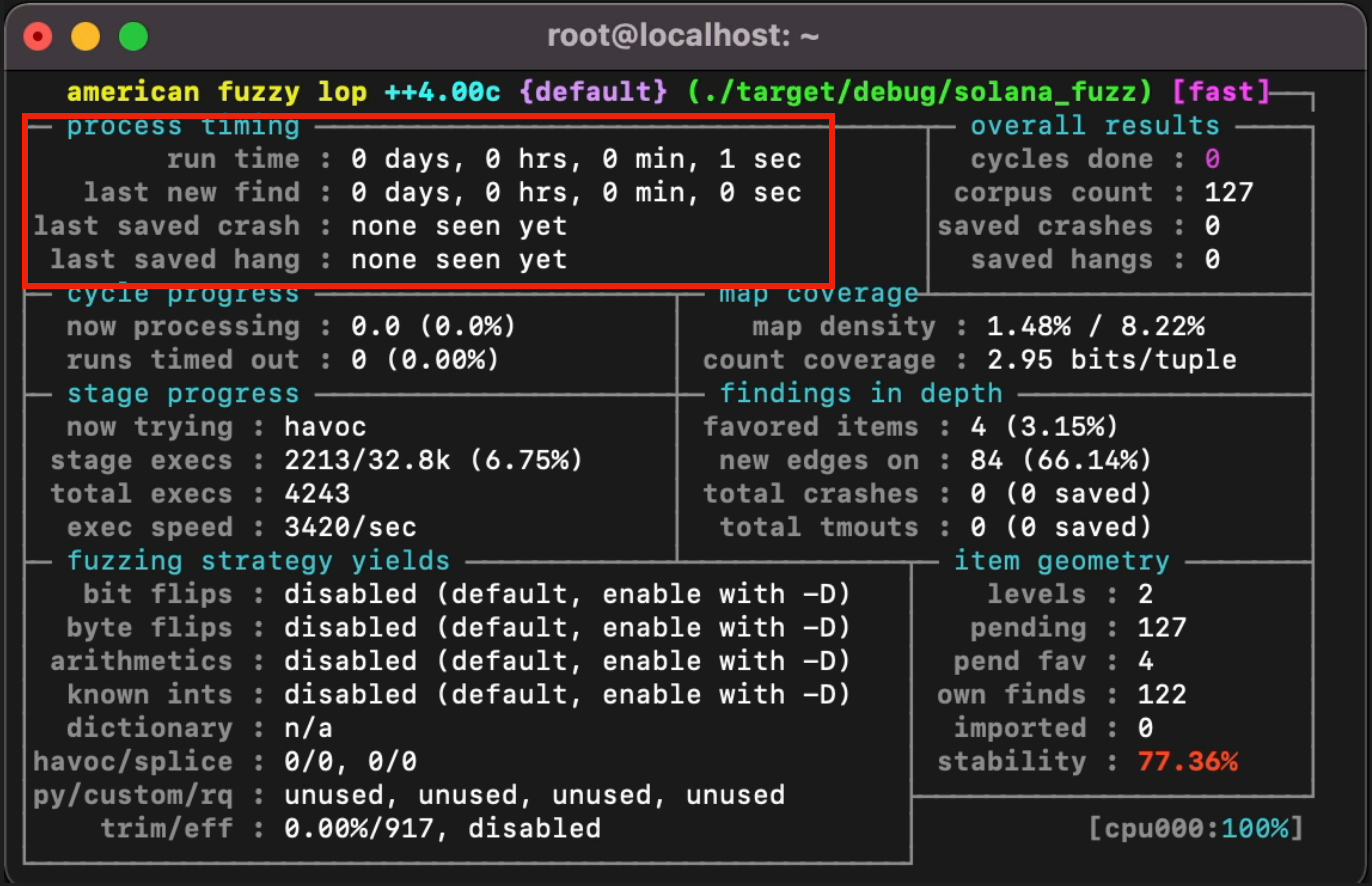
```
root@localhost:~/solana_fuzz# cargo afl fuzz -i inputs -o out_demo -- ./target/d
```

root@localhost: ~

```
root@localhost:~/solana_fuzz# cargo afl fuzz -i inputs -o out_demo -- ./target/d
ebug/solana_fuzz
```

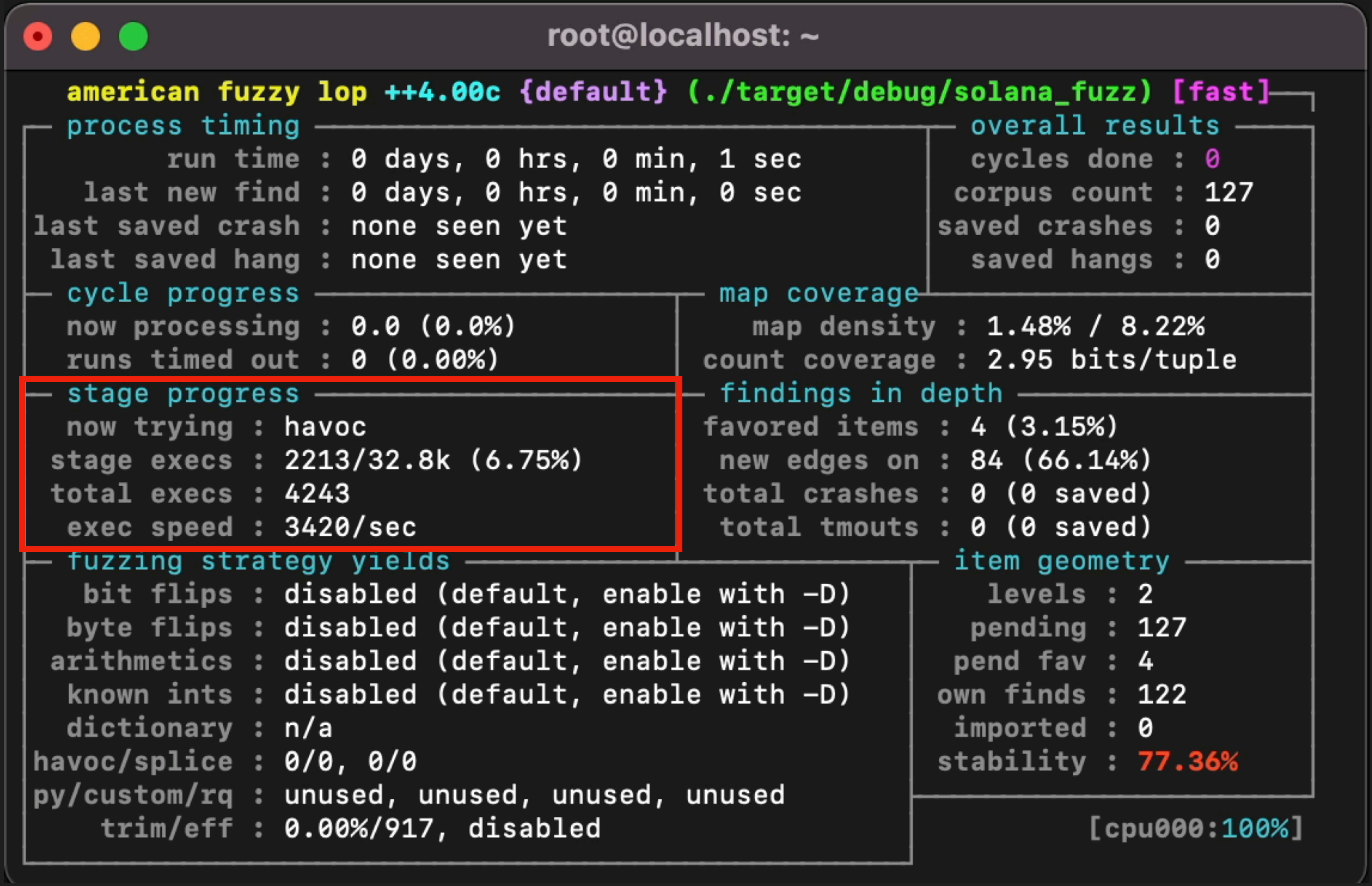
root@localhost: ~

```
root@localhost:~/solana_fuzz# cargo afl fuzz -i inputs -o out_demo -- ./target/d
ebug/solana_fuzz
```


root@localhost: ~
american fuzzy lop ++4.00c {default} (./target/debug/solana_fuzz) [fast]
process timing
run time : 0 days, 0 hrs, 0 min, 1 sec
last new find : 0 days, 0 hrs, 0 min, 0 sec
last saved crash : none seen yet
last saved hang : none seen yet
overall results
cycles done : 0
corpus count : 127
saved crashes : 0
saved hangs : 0
cycle progress
now processing : 0.0 (0.0%)
runs timed out : 0 (0.00%)
map coverage
map density : 1.48% / 8.22%
count coverage : 2.95 bits/tuple
stage progress
now trying : havoc
stage execs : 2213/32.8k (6.75%)
total execs : 4243
exec speed : 3420/sec
findings in depth
favored items : 4 (3.15%)
new edges on : 84 (66.14%)
total crashes : 0 (0 saved)
total tmouts : 0 (0 saved)
fuzzing strategy yields
bit flips : disabled (default, enable with -D)
byte flips : disabled (default, enable with -D)
arithmetics : disabled (default, enable with -D)
known ints : disabled (default, enable with -D)
dictionary : n/a
havoc/splice : 0/0, 0/0
py/custom/rq : unused, unused, unused, unused
trim/eff : 0.00%/917, disabled
item geometry
levels : 2
pending : 127
pend fav : 4
own finds : 122
imported : 0
stability : 77.36%
[cpu000:100%]

root@localhost: ~

```
   american fuzzy lop ++4.00c {default} (./target/debug/solana_fuzz) [fast]
┌─ process timing ────────────────────────────────────┬─ overall results ─┐
│        run time : 0 days, 0 hrs, 0 min, 1 sec       │  cycles done : 0  │
│   last new find : 0 days, 0 hrs, 0 min, 0 sec       │ corpus count : 127│
│last saved crash : none seen yet                     │saved crashes : 0  │
│ last saved hang : none seen yet                     │  saved hangs : 0  │
├─ cycle progress ─────────────────────┬─ map coverage┴───────────────────┤
│  now processing : 0.0 (0.0%)         │    map density : 1.48% / 8.22%  │
│  runs timed out : 0 (0.00%)          │ count coverage : 2.95 bits/tuple │
├─ stage progress ─────────────────────┼─ findings in depth ──────────────┤
│  now trying : havoc                  │ favored items : 4 (3.15%)        │
│ stage execs : 2213/32.8k (6.75%)     │  new edges on : 84 (66.14%)      │
│ total execs : 4243                   │ total crashes : 0 (0 saved)      │
│  exec speed : 3420/sec               │  total tmouts : 0 (0 saved)      │
├─ fuzzing strategy yields ────────────┴─────────────┬─ item geometry ────┤
│   bit flips : disabled (default, enable with -D)   │    levels : 2      │
│  byte flips : disabled (default, enable with -D)   │   pending : 127    │
│ arithmetics : disabled (default, enable with -D)   │  pend fav : 4      │
│  known ints : disabled (default, enable with -D)   │ own finds : 122    │
│  dictionary : n/a                                  │  imported : 0      │
│havoc/splice : 0/0, 0/0                             │ stability : 77.36% │
│py/custom/rq : unused, unused, unused, unused       ├────────────────────┘
│    trim/eff : 0.00%/917, disabled                  │          [cpu000:100%]
└────────────────────────────────────────────────────┘
```


root@localhost: ~
american fuzzy lop ++4.00c {default} (./target/debug/solana_fuzz) [fast]
process timing
run time : 0 days, 0 hrs, 0 min, 1 sec
last new find : 0 days, 0 hrs, 0 min, 0 sec
last saved crash : none seen yet
last saved hang : none seen yet
overall results
cycles done : 0
corpus count : 127
saved crashes : 0
saved hangs : 0
cycle progress
now processing : 0.0 (0.0%)
runs timed out : 0 (0.00%)
map coverage
map density : 1.48% / 8.22%
count coverage : 2.95 bits/tuple
stage progress
now trying : havoc
stage execs : 2213/32.8k (6.75%)
total execs : 4243
exec speed : 3420/sec
findings in depth
favored items : 4 (3.15%)
new edges on : 84 (66.14%)
total crashes : 0 (0 saved)
total tmouts : 0 (0 saved)
fuzzing strategy yields
bit flips : disabled (default, enable with -D)
byte flips : disabled (default, enable with -D)
arithmetics : disabled (default, enable with -D)
known ints : disabled (default, enable with -D)
dictionary : n/a
havoc/splice : 0/0, 0/0
py/custom/rq : unused, unused, unused, unused
trim/eff : 0.00%/917, disabled
item geometry
levels : 2
pending : 127
pend fav : 4
own finds : 122
imported : 0
stability : 77.36%
[cpu000:100%]

root@localhost: ~

```
  american fuzzy lop ++4.00c {default} (./target/debug/solana_fuzz) [fast]
┌─ process timing ────────────────────────────────────┬─ overall results ────┐
│        run time : 0 days, 0 hrs, 0 min, 1 sec      │  cycles done : 0     │
│   last new find : 0 days, 0 hrs, 0 min, 0 sec      │ corpus count : 127   │
│last saved crash : none seen yet                     │saved crashes : 0     │
│ last saved hang : none seen yet                     │  saved hangs : 0     │
├─ cycle progress ─────────────────────┬─ map coverage┴──────────────────────┤
│  now processing : 0.0 (0.0%)         │    map density : 1.48% / 8.22%      │
│  runs timed out : 0 (0.00%)          │ count coverage : 2.95 bits/tuple    │
├─ stage progress ─────────────────────┼─ findings in depth ─────────────────┤
│  now trying : havoc                  │ favored items : 4 (3.15%)           │
│ stage execs : 2213/32.8k (6.75%)     │  new edges on : 84 (66.14%)         │
│ total execs : 4243                   │ total crashes : 0 (0 saved)         │
│  exec speed : 3420/sec               │  total tmouts : 0 (0 saved)         │
├─ fuzzing strategy yields ────────────┴───────────────┬─ item geometry ───────┤
│   bit flips : disabled (default, enable with -D)    │    levels : 2         │
│  byte flips : disabled (default, enable with -D)    │   pending : 127       │
│ arithmetics : disabled (default, enable with -D)    │  pend fav : 4         │
│  known ints : disabled (default, enable with -D)    │ own finds : 122       │
│  dictionary : n/a                                   │  imported : 0         │
│havoc/splice : 0/0, 0/0                              │ stability : 77.36%    │
│py/custom/rq : unused, unused, unused, unused        ├───────────────────────┘
│    trim/eff : 0.00%/917, disabled                   │          [cpu000:100%]
└─────────────────────────────────────────────────────┘
```

root@localhost: ~

```
   american fuzzy lop ++4.00c {default} (./target/debug/solana_fuzz) [fast]
┌─ process timing ────────────────────────────────────┬─ overall results ────┐
│        run time : 0 days, 0 hrs, 1 min, 12 sec      │  cycles done : 0     │
│   last new find : 0 days, 0 hrs, 0 min, 0 sec       │ corpus count : 512   │
│last saved crash : 0 days, 0 hrs, 0 min, 4 sec       │saved crashes : 1     │
│ last saved hang : none seen yet                     │  saved hangs : 0     │
├─ cycle progress ───────────────────┬─ map coverage ─┴──────────────────────┤
│  now processing : 426.0 (83.2%)    │    map density : 5.77% / 13.55%      │
│  runs timed out : 0 (0.00%)        │ count coverage : 3.71 bits/tuple      │
├─ stage progress ───────────────────┼─ findings in depth ───────────────────┤
│  now trying : havoc                │ favored items : 237 (46.29%)          │
│ stage execs : 16.2k/32.8k (49.44%) │  new edges on : 328 (64.06%)          │
│ total execs : 262k                 │ total crashes : 1 (1 saved)           │
│  exec speed : 3605/sec             │  total tmouts : 0 (0 saved)           │
├─ fuzzing strategy yields ──────────┴─────────────┬─ item geometry ────────┤
│   bit flips : disabled (default, enable with -D) │    levels : 5          │
│  byte flips : disabled (default, enable with -D) │   pending : 477        │
│ arithmetics : disabled (default, enable with -D) │  pend fav : 208        │
│  known ints : disabled (default, enable with -D) │ own finds : 508        │
│  dictionary : n/a                                │  imported : 0          │
│havoc/splice : 482/191k, 9/16.7k                  │ stability : 67.93%     │
│py/custom/rq : unused, unused, unused, unused     ├────────────────────────┘
│    trim/eff : 1.95%/32.1k, disabled              │          [cpu000: 50%]
└──────────────────────────────────────────────────┘
```

root@localhost: ~

```
   american fuzzy lop ++4.00c {default} (./target/debug/solana_fuzz) [fast]
┌─ process timing ────────────────────────────────────┬─ overall results ────┐
│        run time : 0 days, 0 hrs, 1 min, 12 sec      │  cycles done : 0     │
│   last new find : 0 days, 0 hrs, 0 min, 0 sec       │ corpus count : 512   │
│last saved crash : 0 days, 0 hrs, 0 min, 4 sec       │saved crashes : 1     │
│ last saved hang : none seen yet                     │  saved hangs : 0     │
├─ cycle progress ─────────────────────┬─ map coverage┴──────────────────────┤
│  now processing : 426.0 (83.2%)      │    map density : 5.77% / 13.55%    │
│  runs timed out : 0 (0.00%)          │ count coverage : 3.71 bits/tuple   │
├─ stage progress ─────────────────────┼─ findings in depth ─────────────────┤
│  now trying : havoc                  │ favored items : 237 (46.29%)        │
│ stage execs : 16.2k/32.8k (49.44%)   │  new edges on : 328 (64.06%)        │
│ total execs : 262k                   │ total crashes : 1 (1 saved)         │
│  exec speed : 3605/sec               │  total tmouts : 0 (0 saved)         │
├─ fuzzing strategy yields ────────────┴───────────┬─ item geometry ─────────┤
│   bit flips : disabled (default, enable with -D) │    levels : 5           │
│  byte flips : disabled (default, enable with -D) │   pending : 477         │
│ arithmetics : disabled (default, enable with -D) │  pend fav : 208         │
│  known ints : disabled (default, enable with -D) │ own finds : 508         │
│  dictionary : n/a                                │  imported : 0           │
│havoc/splice : 482/191k, 9/16.7k                  │ stability : 67.93%      │
│py/custom/rq : unused, unused, unused, unused     ├─────────────────────────┘
│    trim/eff : 1.95%/32.1k, disabled              │          [cpu000: 50%]
└──────────────────────────────────────────────────┘
```

root@localhost: ~

```
  american fuzzy lop ++4.00c {default} (./target/debug/solana_fuzz) [fast]
┌─ process timing ────────────────────────────────────┬─ overall results ────┐
│        run time : 0 days, 0 hrs, 1 min, 12 sec      │  cycles done : 0     │
│   last new find : 0 days, 0 hrs, 0 min, 0 sec       │ corpus count : 512   │
│last saved crash : 0 days, 0 hrs, 0 min, 4 sec       │saved crashes : 54    │
│ last saved hang : none seen yet                     │  saved hangs : 0     │
├─ cycle progress ─────────────────────┬─ map coverage┴──────────────────────┤
│  now processing : 426.0 (83.2%)      │    map density : 5.77% / 13.55%    │
│  runs timed out : 0 (0.00%)          │ count coverage : 3.71 bits/tuple   │
├─ stage progress ─────────────────────┼─ findings in depth ─────────────────┤
│  now trying : havoc                  │ favored items : 237 (46.29%)        │
│ stage execs : 16.2k/32.8k (49.44%)   │  new edges on : 328 (64.06%)        │
│ total execs : 262k                   │ total crashes : 1 (1 saved)         │
│  exec speed : 3605/sec               │  total tmouts : 0 (0 saved)         │
├─ fuzzing strategy yields ────────────┴─────────────┬─ item geometry ───────┤
│   bit flips : disabled (default, enable with -D)   │    levels : 5         │
│  byte flips : disabled (default, enable with -D)   │   pending : 477       │
│ arithmetics : disabled (default, enable with -D)   │  pend fav : 208       │
│  known ints : disabled (default, enable with -D)   │ own finds : 508       │
│  dictionary : n/a                                  │  imported : 0         │
│havoc/splice : 482/191k, 9/16.7k                    │ stability : 67.93%    │
│py/custom/rq : unused, unused, unused, unused       ├───────────────────────┘
│    trim/eff : 1.95%/32.1k, disabled                │          [cpu000: 50%]
└────────────────────────────────────────────────────┘
```

```
root@localhost:~/solana_fuzz# ls out/fuzzer01/crashes/
README.txt
id:000000,sig:08,src:000001,time:159670,op:flip1,pos:248
id:000001,sig:08,src:000001,time:159671,op:flip1,pos:248
id:000002,sig:08,src:000001,time:163277,op:flip2,pos:248
id:000003,sig:08,src:000001,time:201255,op:arith16,pos:247,val:-1
id:000004,sig:08,src:000001,time:201256,op:arith16,pos:247,val:-2
id:000005,sig:08,src:000001,time:201260,op:arith16,pos:247,val:-9
id:000006,sig:08,src:000001,time:201263,op:arith16,pos:247,val:-13
id:000007,sig:08,src:000001,time:201266,op:arith16,pos:247,val:-18
id:000008,sig:08,src:000001,time:226148,op:arith32,pos:245,val:-1
id:000009,sig:08,src:000001,time:226171,op:arith32,pos:246,val:-4
id:000010,sig:08,src:000056,time:1905518,op:flip2,pos:608
id:000011,sig:08,src:000056,time:1923383,op:arith8,pos:608,val:+5
id:000012,sig:08,src:000056,time:2016965,op:ext_AO,pos:252
id:000013,sig:08,src:000058,time:2041379,op:flip2,pos:608
id:000014,sig:11,src:000061,time:2355363,op:arith16,pos:250,val:-15
id:000015,sig:08,src:000065,time:2581301,op:flip1,pos:264
id:000016,sig:08,src:000068,time:2988743,op:flip1,pos:620
id:000017,sig:08,src:000068,time:2991948,op:flip2,pos:256
id:000018,sig:08,src:000068,time:2991948,op:flip2,pos:256
id:000019,sig:08,src:000068,time:2992215,op:flip2,pos:376
id:000020,sig:08,src:000068,time:2996028,op:flip4,pos:256
id:000021,sig:08,src:000068,time:2996029,op:flip4,pos:256
id:000022,sig:08,src:000068,time:2996030,op:flip4,pos:256
id:000023,sig:08,src:000068,time:2996030,op:flip4,pos:256
id:000024,sig:08,src:000068,time:2996239,op:flip4,pos:352
id:000025,sig:08,src:000068,time:2999551,op:flip8,pos:256
id:000026,sig:08,src:000068,time:3004869,op:arith8,pos:256,val:-6
id:000027,sig:08,src:000068,time:3004870,op:arith8,pos:256,val:-7
id:000028,sig:08,src:000068,time:3004872,op:arith8,pos:256,val:+10
id:000029,sig:08,src:000068,time:3004874,op:arith8,pos:256,val:-12
id:000030,sig:08,src:000068,time:3004876,op:arith8,pos:256,val:-14
id:000031,sig:08,src:000068,time:3004877,op:arith8,pos:256,val:-15
id:000032,sig:08,src:000068,time:3004878,op:arith8,pos:256,val:+17
id:000033,sig:08,src:000068,time:3004879,op:arith8,pos:256,val:+18
id:000034,sig:08,src:000068,time:3004881,op:arith8,pos:256,val:+20
id:000035,sig:08,src:000068,time:3004881,op:arith8,pos:256,val:-20
id:000036,sig:08,src:000068,time:3004883,op:arith8,pos:256,val:-22
id:000037,sig:08,src:000068,time:3004884,op:arith8,pos:256,val:-23
id:000038,sig:08,src:000068,time:3004885,op:arith8,pos:256,val:+25
id:000039,sig:08,src:000068,time:3004887,op:arith8,pos:256,val:+28
id:000040,sig:08,src:000068,time:3004888,op:arith8,pos:256,val:-28
id:000041,sig:08,src:000068,time:3004890,op:arith8,pos:256,val:-30
id:000042,sig:08,src:000068,time:3004891,op:arith8,pos:256,val:-31
id:000043,sig:08,src:000068,time:3004892,op:arith8,pos:256,val:+33
id:000044,sig:08,src:000068,time:3004893,op:arith8,pos:256,val:+34
id:000045,sig:08,src:000068,time:3083111,op:int16,pos:255,val:+1024
id:000046,sig:08,src:000068,time:3089878,op:int32,pos:253,val:+100663045
id:000047,sig:08,src:000068,time:3103126,op:ext_AO,pos:236
id:000048,sig:08,src:000068,time:3103838,op:ext_AO,pos:256
id:000049,sig:08,src:000068,time:3104830,op:ext_AO,pos:345
id:000050,sig:08,src:000068,time:3104919,op:ext_AO,pos:356
id:000051,sig:11,src:000226,time:10488566,op:arith16,pos:258,val:-16
id:000052,sig:11,src:000272,time:12327385,op:arith16,pos:250,val:-15
id:000053,sig:08,src:000290,time:14322431,op:flip2,pos:256
```

```
root@localhost:~/solana_fuzz# ls out/fuzzer01/crashes/
README.txt
id:000000,sig:08,src:000001,time:159670,op:flip1,pos:248
id:000001,sig:08,src:000001,time:159671,op:flip1,pos:248
id:000002,sig:08,src:000001,time:163277,op:flip2,pos:248
id:000003,sig:08,src:000001,time:201255,op:arith16,pos:247,val:-1
id:000004,sig:08,src:000001,time:201256,op:arith16,pos:247,val:-2
id:000005,sig:08,src:000001,time:201260,op:arith16,pos:247,val:-9
id:000006,sig:08,src:000001,time:201263,op:arith16,pos:247,val:-13
id:000007,sig:08,src:000001,time:201266,op:arith16,pos:247,val:-18
id:000008,sig:08,src:000001,time:226148,op:arith32,pos:245,val:-1
id:000009,sig:08,src:000001,time:226171,op:arith32,pos:246,val:-4
id:000010,sig:08,src:000056,time:1905518,op:flip2,pos:608
id:000011,sig:08,src:000056,time:1923383,op:arith8,pos:608,val:+5
id:000012,sig:08,src:000056,time:2016965,op:ext_AO,pos:252
id:000013,sig:08,src:000058,time:2041379,op:flip2,pos:608
id:000014,sig:11,src:000061,time:2355363,op:arith16,pos:250,val:-15
id:000015,sig:08,src:000065,time:2581301,op:flip1,pos:264
id:000016,sig:08,src:000068,time:2988743,op:flip1,pos:620
id:000017,sig:08,src:000068,time:2991948,op:flip2,pos:256
id:000018,sig:08,src:000068,time:2991948,op:flip2,pos:256
id:000019,sig:08,src:000068,time:2992215,op:flip2,pos:376
id:000020,sig:08,src:000068,time:2996028,op:flip4,pos:256
id:000021,sig:08,src:000068,time:2996029,op:flip4,pos:256
id:000022,sig:08,src:000068,time:2996030,op:flip4,pos:256
id:000023,sig:08,src:000068,time:2996030,op:flip4,pos:256
id:000024,sig:08,src:000068,time:2996239,op:flip4,pos:352
id:000025,sig:08,src:000068,time:2999551,op:flip8,pos:256
id:000026,sig:08,src:000068,time:3004869,op:arith8,pos:256,val:-6
id:000027,sig:08,src:000068,time:3004870,op:arith8,pos:256,val:-7
id:000028,sig:08,src:000068,time:3004872,op:arith8,pos:256,val:+10
id:000029,sig:08,src:000068,time:3004874,op:arith8,pos:256,val:-12
id:000030,sig:08,src:000068,time:3004876,op:arith8,pos:256,val:-14
id:000031,sig:08,src:000068,time:3004877,op:arith8,pos:256,val:-15
id:000032,sig:08,src:000068,time:3004878,op:arith8,pos:256,val:+17
id:000033,sig:08,src:000068,time:3004879,op:arith8,pos:256,val:+18
id:000034,sig:08,src:000068,time:3004881,op:arith8,pos:256,val:+20
id:000035,sig:08,src:000068,time:3004881,op:arith8,pos:256,val:-20
id:000036,sig:08,src:000068,time:3004883,op:arith8,pos:256,val:-22
id:000037,sig:08,src:000068,time:3004884,op:arith8,pos:256,val:-23
id:000038,sig:08,src:000068,time:3004885,op:arith8,pos:256,val:+25
id:000039,sig:08,src:000068,time:3004887,op:arith8,pos:256,val:+28
id:000040,sig:08,src:000068,time:3004888,op:arith8,pos:256,val:-28
id:000041,sig:08,src:000068,time:3004890,op:arith8,pos:256,val:-30
id:000042,sig:08,src:000068,time:3004891,op:arith8,pos:256,val:-31
id:000043,sig:08,src:000068,time:3004892,op:arith8,pos:256,val:+33
id:000044,sig:08,src:000068,time:3004893,op:arith8,pos:256,val:+34
id:000045,sig:08,src:000068,time:3083111,op:int16,pos:255,val:+1024
id:000046,sig:08,src:000068,time:3089878,op:int32,pos:253,val:+100663045
id:000047,sig:08,src:000068,time:3103126,op:ext_AO,pos:236
id:000048,sig:08,src:000068,time:3103838,op:ext_AO,pos:256
id:000049,sig:08,src:000068,time:3104830,op:ext_AO,pos:345
id:000050,sig:08,src:000068,time:3104919,op:ext_AO,pos:356
id:000051,sig:11,src:000226,time:10488566,op:arith16,pos:258,val:-16
id:000052,sig:11,src:000272,time:12327385,op:arith16,pos:250,val:-15
id:000053,sig:08,src:000290,time:14322431,op:flip2,pos:256
```

```
root@localhost:~/solana_fuzz# ls out/fuzzer01/crashes/
README.txt
id:000000,sig:08,src:000001,time:159670,op:flip1,pos:248
id:000001,sig:08,src:000001,time:159671,op:flip1,pos:248
id:000002,sig:08,src:000001,time:163277,op:flip2,pos:248
id:000003,sig:08,src:000001,time:201255,op:arith16,pos:247,val:-1
id:000004,sig:08,src:000001,time:201256,op:arith16,pos:247,val:-2
id:000005,sig:08,src:000001,time:201260,op:arith16,pos:247,val:-9
id:000006,sig:08,src:000001,time:201263,op:arith16,pos:247,val:-13
id:000007,sig:08,src:000001,time:201266,op:arith16,pos:247,val:-18
id:000008,sig:08,src:000001,time:226148,op:arith32,pos:245,val:-1
id:000009,sig:08,src:000001,time:226171,op:arith32,pos:246,val:-4
id:000010,sig:08,src:000056,time:1905518,op:flip2,pos:608
id:000011,sig:08,src:000056,time:1923383,op:arith8,pos:608,val:+5
id:000012,sig:08,src:000056,time:2016965,op:ext_AO,pos:252
id:000013,sig:08,src:000058,time:2041379,op:flip2,pos:608
id:000014,sig:11,src:000061,time:2355363,op:arith16,pos:250,val:-15
id:000015,sig:08,src:000065,time:2581301,op:flip1,pos:264
id:000016,sig:08,src:000068,time:2988743,op:flip1,pos:620
id:000017,sig:08,src:000068,time:2991948,op:flip2,pos:256
id:000018,sig:08,src:000068,time:2991948,op:flip2,pos:256
id:000019,sig:08,src:000068,time:2992215,op:flip2,pos:376
id:000020,sig:08,src:000068,time:2996028,op:flip4,pos:256
id:000021,sig:08,src:000068,time:2996029,op:flip4,pos:256
id:000022,sig:08,src:000068,time:2996030,op:flip4,pos:256
id:000023,sig:08,src:000068,time:2996030,op:flip4,pos:256
id:000024,sig:08,src:000068,time:2996239,op:flip4,pos:352
id:000025,sig:08,src:000068,time:2999551,op:flip8,pos:256
id:000026,sig:08,src:000068,time:3004869,op:arith8,pos:256,val:-6
id:000027,sig:08,src:000068,time:3004870,op:arith8,pos:256,val:-7
id:000028,sig:08,src:000068,time:3004872,op:arith8,pos:256,val:+10
id:000029,sig:08,src:000068,time:3004874,op:arith8,pos:256,val:-12
id:000030,sig:08,src:000068,time:3004876,op:arith8,pos:256,val:-14
id:000031,sig:08,src:000068,time:3004877,op:arith8,pos:256,val:-15
id:000032,sig:08,src:000068,time:3004878,op:arith8,pos:256,val:+17
id:000033,sig:08,src:000068,time:3004879,op:arith8,pos:256,val:+18
id:000034,sig:08,src:000068,time:3004881,op:arith8,pos:256,val:+20
id:000035,sig:08,src:000068,time:3004881,op:arith8,pos:256,val:-20
id:000036,sig:08,src:000068,time:3004883,op:arith8,pos:256,val:-22
id:000037,sig:08,src:000068,time:3004884,op:arith8,pos:256,val:-23
id:000038,sig:08,src:000068,time:3004885,op:arith8,pos:256,val:+25
id:000039,sig:08,src:000068,time:3004887,op:arith8,pos:256,val:+28
id:000040,sig:08,src:000068,time:3004888,op:arith8,pos:256,val:-28
id:000041,sig:08,src:000068,time:3004890,op:arith8,pos:256,val:-30
id:000042,sig:08,src:000068,time:3004891,op:arith8,pos:256,val:-31
id:000043,sig:08,src:000068,time:3004892,op:arith8,pos:256,val:+33
id:000044,sig:08,src:000068,time:3004893,op:arith8,pos:256,val:+34
id:000045,sig:08,src:000068,time:3083111,op:int16,pos:255,val:+1024
id:000046,sig:08,src:000068,time:3089878,op:int32,pos:253,val:+100663045
id:000047,sig:08,src:000068,time:3103126,op:ext_AO,pos:236
id:000048,sig:08,src:000068,time:3103838,op:ext_AO,pos:256
id:000049,sig:08,src:000068,time:3104830,op:ext_AO,pos:345
id:000050,sig:08,src:000068,time:3104919,op:ext_AO,pos:356
id:000051,sig:11,src:000226,time:10488566,op:arith16,pos:258,val:-16
id:000052,sig:11,src:000272,time:12327385,op:arith16,pos:250,val:-15
id:000053,sig:08,src:000290,time:14322431,op:flip2,pos:256
```

Mostly floating-point exceptions :(

```
root@localhost:~/solana_fuzz# ls out/fuzzer01/crashes/
README.txt
id:000000,sig:08,src:000001,time:159670,op:flip1,pos:248
id:000001,sig:08,src:000001,time:159671,op:flip1,pos:248
id:000002,sig:08,src:000001,time:163277,op:flip2,pos:248
id:000003,sig:08,src:000001,time:201255,op:arith16,pos:247,val:-1
id:000004,sig:08,src:000001,time:201256,op:arith16,pos:247,val:-2
id:000005,sig:08,src:000001,time:201260,op:arith16,pos:247,val:-9
id:000006,sig:08,src:000001,time:201263,op:arith16,pos:247,val:-13
id:000007,sig:08,src:000001,time:201266,op:arith16,pos:247,val:-18
id:000008,sig:08,src:000001,time:226148,op:arith32,pos:245,val:-1
id:000009,sig:08,src:000001,time:226171,op:arith32,pos:246,val:-4
id:000010,sig:08,src:000056,time:1905518,op:flip2,pos:608
id:000011,sig:08,src:000056,time:1923383,op:arith8,pos:608,val:+5
id:000012,sig:08,src:000056,time:2016965,op:ext_AO,pos:252
id:000013,sig:08,src:000058,time:2041379,op:flip2,pos:608
id:000014,sig:11,src:000061,time:2355363,op:arith16,pos:250,val:-15
id:000015,sig:08,src:000065,time:2581301,op:flip1,pos:264
id:000016,sig:08,src:000068,time:2988743,op:flip1,pos:620
id:000017,sig:08,src:000068,time:2991948,op:flip2,pos:256
id:000018,sig:08,src:000068,time:2991948,op:flip2,pos:256
id:000019,sig:08,src:000068,time:2992215,op:flip2,pos:376
id:000020,sig:08,src:000068,time:2996028,op:flip4,pos:256
id:000021,sig:08,src:000068,time:2996029,op:flip4,pos:256
id:000022,sig:08,src:000068,time:2996030,op:flip4,pos:256
id:000023,sig:08,src:000068,time:2996030,op:flip4,pos:256
id:000024,sig:08,src:000068,time:2996239,op:flip4,pos:352
id:000025,sig:08,src:000068,time:2999551,op:flip8,pos:256
id:000026,sig:08,src:000068,time:3004869,op:arith8,pos:256,val:-6
id:000027,sig:08,src:000068,time:3004870,op:arith8,pos:256,val:-7
id:000028,sig:08,src:000068,time:3004872,op:arith8,pos:256,val:+10
id:000029,sig:08,src:000068,time:3004874,op:arith8,pos:256,val:-12
id:000030,sig:08,src:000068,time:3004876,op:arith8,pos:256,val:-14
id:000031,sig:08,src:000068,time:3004877,op:arith8,pos:256,val:-15
id:000032,sig:08,src:000068,time:3004878,op:arith8,pos:256,val:+17
id:000033,sig:08,src:000068,time:3004879,op:arith8,pos:256,val:+18
id:000034,sig:08,src:000068,time:3004881,op:arith8,pos:256,val:+20
id:000035,sig:08,src:000068,time:3004881,op:arith8,pos:256,val:-20
id:000036,sig:08,src:000068,time:3004883,op:arith8,pos:256,val:-22
id:000037,sig:08,src:000068,time:3004884,op:arith8,pos:256,val:-23
id:000038,sig:08,src:000068,time:3004885,op:arith8,pos:256,val:+25
id:000039,sig:08,src:000068,time:3004887,op:arith8,pos:256,val:+28
id:000040,sig:08,src:000068,time:3004888,op:arith8,pos:256,val:-28
id:000041,sig:08,src:000068,time:3004890,op:arith8,pos:256,val:-30
id:000042,sig:08,src:000068,time:3004891,op:arith8,pos:256,val:-31
id:000043,sig:08,src:000068,time:3004892,op:arith8,pos:256,val:+33
id:000044,sig:08,src:000068,time:3004893,op:arith8,pos:256,val:+34
id:000045,sig:08,src:000068,time:3083111,op:int16,pos:255,val:+1024
id:000046,sig:08,src:000068,time:3089878,op:int32,pos:253,val:+100663045
id:000047,sig:08,src:000068,time:3103126,op:ext_AO,pos:236
id:000048,sig:08,src:000068,time:3103838,op:ext_AO,pos:256
id:000049,sig:08,src:000068,time:3104830,op:ext_AO,pos:345
id:000050,sig:08,src:000068,time:3104919,op:ext_AO,pos:356
id:000051,sig:11,src:000226,time:10488566,op:arith16,pos:258,val:-16
id:000052,sig:11,src:000272,time:12327385,op:arith16,pos:250,val:-15
id:000053,sig:08,src:000290,time:14322431,op:flip2,pos:256
```

```
root@localhost:~/solana_fuzz# ls out/fuzzer01/crashes/
README.txt
id:000000,sig:08,src:000001,time:159670,op:flip1,pos:248
id:000001,sig:08,src:000001,time:159671,op:flip1,pos:248
id:000002,sig:08,src:000001,time:163277,op:flip2,pos:248
id:000003,sig:08,src:000001,time:201255,op:arith16,pos:247,val:-1
id:000004,sig:08,src:000001,time:201256,op:arith16,pos:247,val:-2
id:000005,sig:08,src:000001,time:201260,op:arith16,pos:247,val:-9
id:000006,sig:08,src:000001,time:201263,op:arith16,pos:247,val:-13
id:000007,sig:08,src:000001,time:201266,op:arith16,pos:247,val:-18
id:000008,sig:08,src:000001,time:226148,op:arith32,pos:245,val:-1
id:000009,sig:08,src:000001,time:226171,op:arith32,pos:246,val:-4
id:000010,sig:08,src:000056,time:1905518,op:flip2,pos:608
id:000011,sig:08,src:000056,time:1923383,op:arith8,pos:608,val:+5
id:000012,sig:08,src:000056,time:2016965,op:ext_AO,pos:252
id:000013,sig:08,src:000058,time:2041379,op:flip2,pos:608
id:000014,sig:11,src:000061,time:2355363,op:arith16,pos:250,val:-15
id:000015,sig:08,src:000065,time:2581301,op:flip1,pos:264
id:000016,sig:08,src:000068,time:2988743,op:flip1,pos:620
id:000017,sig:08,src:000068,time:2991948,op:flip2,pos:256
id:000018,sig:08,src:000068,time:2991948,op:flip2,pos:256
id:000019,sig:08,src:000068,time:2992215,op:flip2,pos:376
id:000020,sig:08,src:000068,time:2996028,op:flip4,pos:256
id:000021,sig:08,src:000068,time:2996029,op:flip4,pos:256
id:000022,sig:08,src:000068,time:2996030,op:flip4,pos:256
id:000023,sig:08,src:000068,time:2996030,op:flip4,pos:256
id:000024,sig:08,src:000068,time:2996239,op:flip4,pos:352
id:000025,sig:08,src:000068,time:2999551,op:flip8,pos:256
id:000026,sig:08,src:000068,time:3004869,op:arith8,pos:256,val:-6
id:000027,sig:08,src:000068,time:3004870,op:arith8,pos:256,val:-7
id:000028,sig:08,src:000068,time:3004872,op:arith8,pos:256,val:+10
id:000029,sig:08,src:000068,time:3004874,op:arith8,pos:256,val:-12
id:000030,sig:08,src:000068,time:3004876,op:arith8,pos:256,val:-14
id:000031,sig:08,src:000068,time:3004877,op:arith8,pos:256,val:-15
id:000032,sig:08,src:000068,time:3004878,op:arith8,pos:256,val:+17
id:000033,sig:08,src:000068,time:3004879,op:arith8,pos:256,val:+18
id:000034,sig:08,src:000068,time:3004881,op:arith8,pos:256,val:+20
id:000035,sig:08,src:000068,time:3004881,op:arith8,pos:256,val:-20
id:000036,sig:08,src:000068,time:3004883,op:arith8,pos:256,val:-22
id:000037,sig:08,src:000068,time:3004884,op:arith8,pos:256,val:-23
id:000038,sig:08,src:000068,time:3004885,op:arith8,pos:256,val:+25
id:000039,sig:08,src:000068,time:3004887,op:arith8,pos:256,val:+28
id:000040,sig:08,src:000068,time:3004888,op:arith8,pos:256,val:-28
id:000041,sig:08,src:000068,time:3004890,op:arith8,pos:256,val:-30
id:000042,sig:08,src:000068,time:3004891,op:arith8,pos:256,val:-31
id:000043,sig:08,src:000068,time:3004892,op:arith8,pos:256,val:+33
id:000044,sig:08,src:000068,time:3004893,op:arith8,pos:256,val:+34
id:000045,sig:08,src:000068,time:3083111,op:int16,pos:255,val:+1024
id:000046,sig:08,src:000068,time:3089878,op:int32,pos:253,val:+100663045
id:000047,sig:08,src:000068,time:3103126,op:ext_AO,pos:236
id:000048,sig:08,src:000068,time:3103838,op:ext_AO,pos:256
id:000049,sig:08,src:000068,time:3104830,op:ext_AO,pos:345
id:000050,sig:08,src:000068,time:3104919,op:ext_AO,pos:356
id:000051,sig:11,src:000226,time:10488566,op:arith16,pos:258,val:-16
id:000052,sig:11,src:000272,time:12327385,op:arith16,pos:250,val:-15
id:000053,sig:08,src:000290,time:14322431,op:flip2,pos:256
```

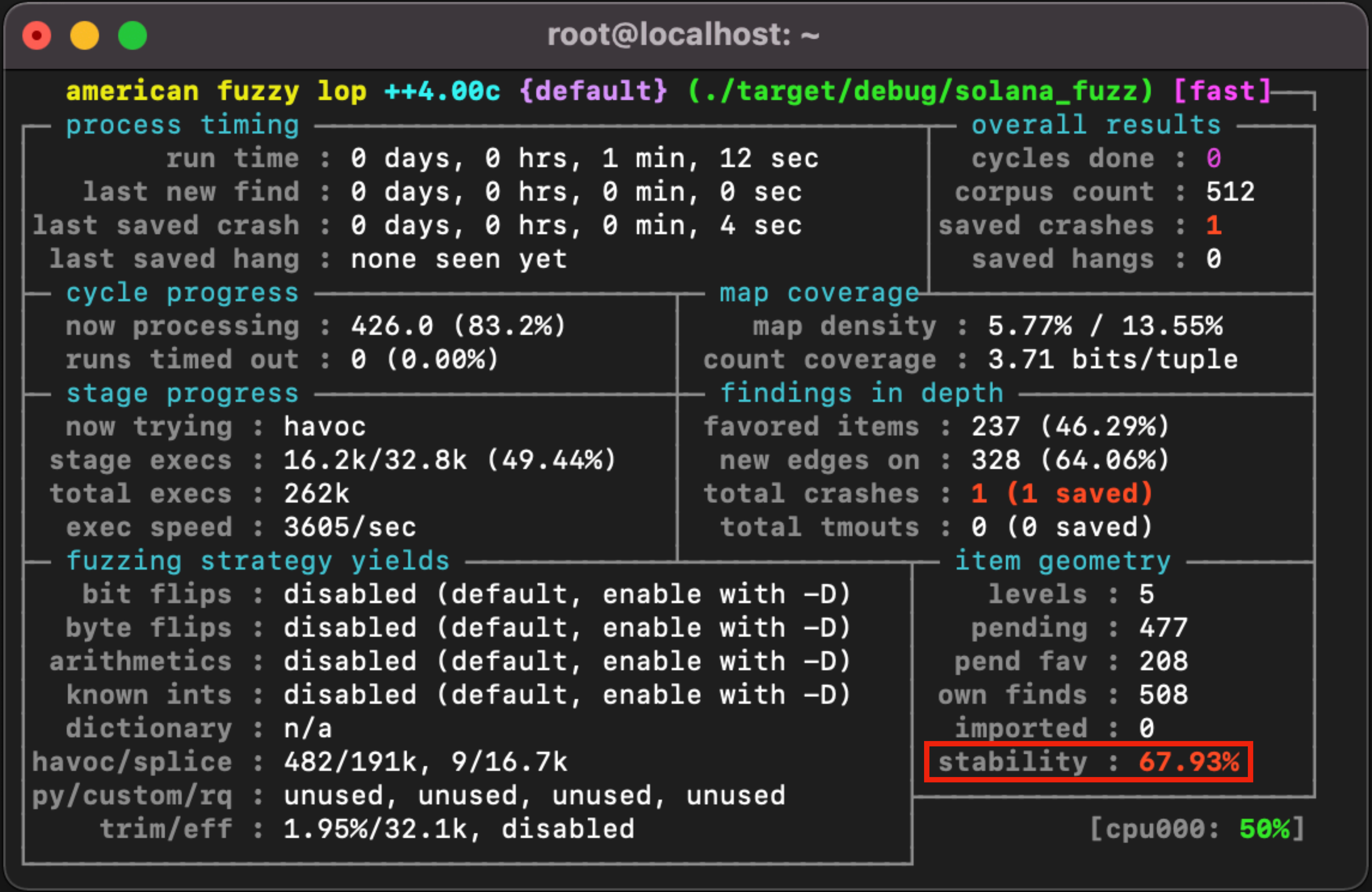
Segmentation faults 🥳

root@localhost: ~/solana_test

```
root@localhost:~/solana_test# ./solana_test crash.so
root@localhost:~/solana_test# ./solana_test crash.so
root@localhost:~/solana_test# ./solana_test crash.so
root@localhost:~/solana_test# ./solana_test crash.so
root@localhost:~/solana_test# ./solana_test crash.so
root@localhost:~/solana_test# ./solana_test crash.so
root@localhost:~/solana_test# ./solana_test crash.so
root@localhost:~/solana_test# ./solana_test crash.so
root@localhost:~/solana_test# ./solana_test crash.so
root@localhost:~/solana_test# ./solana_test crash.so
root@localhost:~/solana_test# ./solana_test crash.so
root@localhost:~/solana_test# ./solana_test crash.so
root@localhost:~/solana_test# ./solana_test crash.so
root@localhost:~/solana_test# ./solana_test crash.so
root@localhost:~/solana_test# ./solana_test crash.so
root@localhost:~/solana_test# ./solana_test crash.so
root@localhost:~/solana_test# ./solana_test crash.so
root@localhost:~/solana_test# ./solana_test crash.so
root@localhost:~/solana_test# ./solana_test crash.so
root@localhost:~/solana_test# ./solana_test crash.so
root@localhost:~/solana_test# ./solana_test crash.so
root@localhost:~/solana_test# ./solana_test crash.so
root@localhost:~/solana_test# ./solana_test crash.so
root@localhost:~/solana_test#
```

root@localhost: ~

```
   american fuzzy lop ++4.00c {default} (./target/debug/solana_fuzz) [fast]
┌─ process timing ────────────────────────────────────┬─ overall results ────┐
│         run time : 0 days, 0 hrs, 1 min, 12 sec      │  cycles done : 0     │
│    last new find : 0 days, 0 hrs, 0 min, 0 sec       │ corpus count : 512   │
│last saved crash : 0 days, 0 hrs, 0 min, 4 sec        │saved crashes : 1     │
│ last saved hang : none seen yet                      │  saved hangs : 0     │
├─ cycle progress ─────────────────────┬─ map coverage─┴──────────────────────┤
│  now processing : 426.0 (83.2%)      │    map density : 5.77% / 13.55%      │
│  runs timed out : 0 (0.00%)          │ count coverage : 3.71 bits/tuple     │
├─ stage progress ─────────────────────┼─ findings in depth ──────────────────┤
│  now trying : havoc                  │ favored items : 237 (46.29%)         │
│ stage execs : 16.2k/32.8k (49.44%)   │  new edges on : 328 (64.06%)         │
│ total execs : 262k                   │ total crashes : 1 (1 saved)          │
│  exec speed : 3605/sec               │  total tmouts : 0 (0 saved)          │
├─ fuzzing strategy yields ────────────┴───────────────┬─ item geometry ───────┤
│   bit flips : disabled (default, enable with -D)     │    levels : 5         │
│  byte flips : disabled (default, enable with -D)     │   pending : 477       │
│ arithmetics : disabled (default, enable with -D)     │  pend fav : 208       │
│  known ints : disabled (default, enable with -D)     │ own finds : 508       │
│  dictionary : n/a                                    │  imported : 0         │
│havoc/splice : 482/191k, 9/16.7k                      │ stability : 67.93%    │
│py/custom/rq : unused, unused, unused, unused         ├───────────────────────┘
│    trim/eff : 1.95%/32.1k, disabled                  │          [cpu000: 50%]
└──────────────────────────────────────────────────────┘
```


root@localhost: ~
american fuzzy lop ++4.00c {default} (./target/debug/solana_fuzz) [fast]
process timing
run time : 0 days, 0 hrs, 1 min, 12 sec
last new find : 0 days, 0 hrs, 0 min, 0 sec
last saved crash : 0 days, 0 hrs, 0 min, 4 sec
last saved hang : none seen yet
overall results
cycles done : 0
corpus count : 512
saved crashes : 1
saved hangs : 0
cycle progress
now processing : 426.0 (83.2%)
runs timed out : 0 (0.00%)
map coverage
map density : 5.77% / 13.55%
count coverage : 3.71 bits/tuple
stage progress
now trying : havoc
stage execs : 16.2k/32.8k (49.44%)
total execs : 262k
exec speed : 3605/sec
findings in depth
favored items : 237 (46.29%)
new edges on : 328 (64.06%)
total crashes : 1 (1 saved)
total tmouts : 0 (0 saved)
fuzzing strategy yields
bit flips : disabled (default, enable with -D)
byte flips : disabled (default, enable with -D)
arithmetics : disabled (default, enable with -D)
known ints : disabled (default, enable with -D)
dictionary : n/a
havoc/splice : 482/191k, 9/16.7k
py/custom/rq : unused, unused, unused, unused
trim/eff : 1.95%/32.1k, disabled
item geometry
levels : 5
pending : 477
pend fav : 208
own finds : 508
imported : 0
stability : 67.93%
[cpu000: 50%]

root@localhost: ~

```
  american fuzzy lop ++4.00c {default} (./target/debug/solana_fuzz) [fast]
┌─ process timing ────────────────────────────────────┬─ overall results ────┐
│        run time : 0 days, 0 hrs, 1 min, 12 sec      │  cycles done : 0     │
│   last new find : 0 days, 0 hrs, 0 min, 0 sec       │ corpus count : 512   │
│last saved crash : 0 days, 0 hrs, 0 min, 4 sec       │saved crashes : 1     │
│ last saved hang : none seen yet                     │  saved hangs : 0     │
├─ cycle progress ─────────────────────┬─ map coverage┴──────────────────────┤
│  now processing : 426.0 (83.2%)      │    map density : 5.77% / 13.55%     │
│  runs timed out : 0 (0.00%)          │ count coverage : 3.71 bits/tuple    │
├─ stage progress ─────────────────────┼─ findings in depth ─────────────────┤
│  now trying : havoc                  │ favored items : 237 (46.29%)        │
│ stage execs : 16.2k/32.8k (49.44%)   │  new edges on : 328 (64.06%)        │
│ total execs : 262k                   │ total crashes : 1 (1 saved)         │
│  exec speed : 3605/sec               │  total tmouts : 0 (0 saved)         │
├─ fuzzing strategy yields ────────────┴───────────────┬─ item geometry ──────┤
│   bit flips : disabled (default, enable with -D)     │    levels : 5        │
│  byte flips : disabled (default, enable with -D)     │   pending : 477      │
│ arithmetics : disabled (default, enable with -D)     │  pend fav : 208      │
│  known ints : disabled (default, enable with -D)     │ own finds : 508      │
│  dictionary : n/a                                    │  imported : 0        │
│havoc/splice : 482/191k, 9/16.7k                      │ stability : 67.93%   │
│py/custom/rq : unused, unused, unused, unused         ├──────────────────────┘
│    trim/eff : 1.95%/32.1k, disabled                  │          [cpu000: 50%]
└──────────────────────────────────────────────────────┘
```

It's not deterministic?!

```
/// Ratio of native host instructions per random no-op in JIT (0 = OFF)
pub noop_instruction_rate: u32,
```

root@localhost: ~

```
  american fuzzy lop ++4.00c {default} (./target/debug/solana_fuzz) [fast]
┌─ process timing ────────────────────────────────────┬─ overall results ───┐
│        run time : 0 days, 0 hrs, 0 min, 3 sec       │  cycles done : 0    │
│   last new find : 0 days, 0 hrs, 0 min, 0 sec       │ corpus count : 201  │
│last saved crash : none seen yet                     │saved crashes : 0    │
│ last saved hang : none seen yet                     │  saved hangs : 0    │
├─ cycle progress ─────────────────────┬─ map coverage┴─────────────────────┤
│  now processing : 2.0 (1.0%)         │    map density : 5.60% / 7.98%     │
│  runs timed out : 0 (0.00%)          │ count coverage : 1.67 bits/tuple   │
├─ stage progress ─────────────────────┼─ findings in depth ────────────────┤
│  now trying : havoc                  │ favored items : 4 (1.99%)          │
│ stage execs : 9180/24.6k (37.35%)    │  new edges on : 120 (59.70%)       │
│ total execs : 12.4k                  │ total crashes : 0 (0 saved)        │
│  exec speed : 3422/sec               │  total tmouts : 0 (0 saved)        │
├─ fuzzing strategy yields ────────────┴───────────┬─ item geometry ────────┤
│   bit flips : disabled (default, enable with -D) │    levels : 2          │
│  byte flips : disabled (default, enable with -D) │   pending : 201        │
│ arithmetics : disabled (default, enable with -D) │  pend fav : 4          │
│  known ints : disabled (default, enable with -D) │ own finds : 197        │
│  dictionary : n/a                                │  imported : 0          │
│havoc/splice : 0/0, 0/0                           │ stability : 98.09%     │
│py/custom/rq : unused, unused, unused, unused     ├────────────────────────┘
│    trim/eff : 0.00%/908, disabled                │          [cpu000: 75%]
└──────────────────────────────────────────────────┘
```

51 segmentation faults!

Triage

kraken

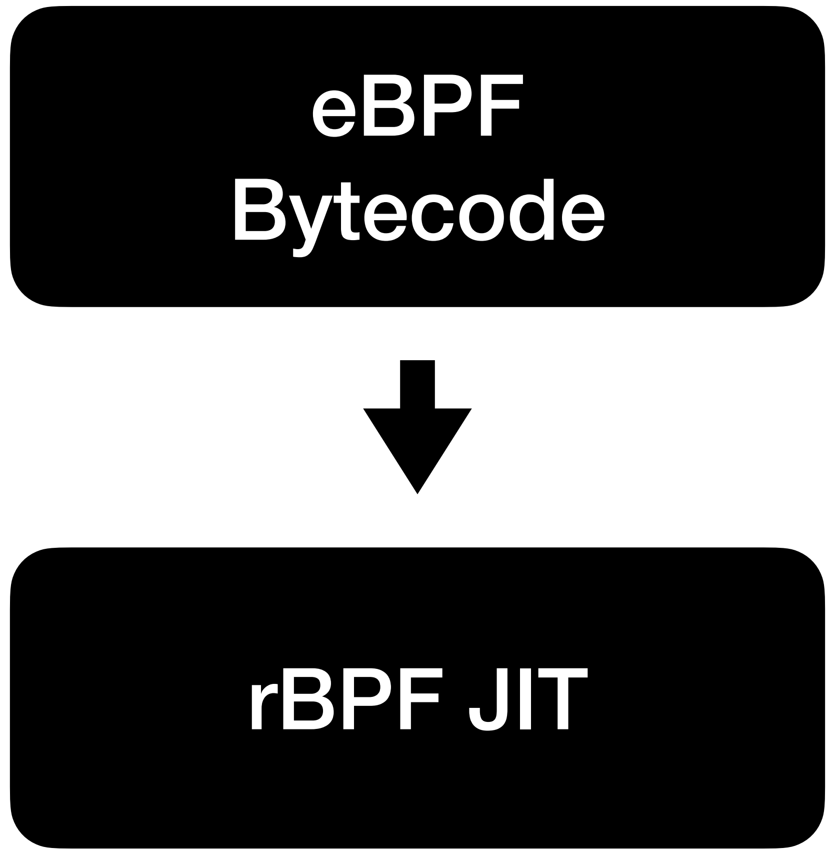
Triage

So many indirections...

eBPF
Bytecode

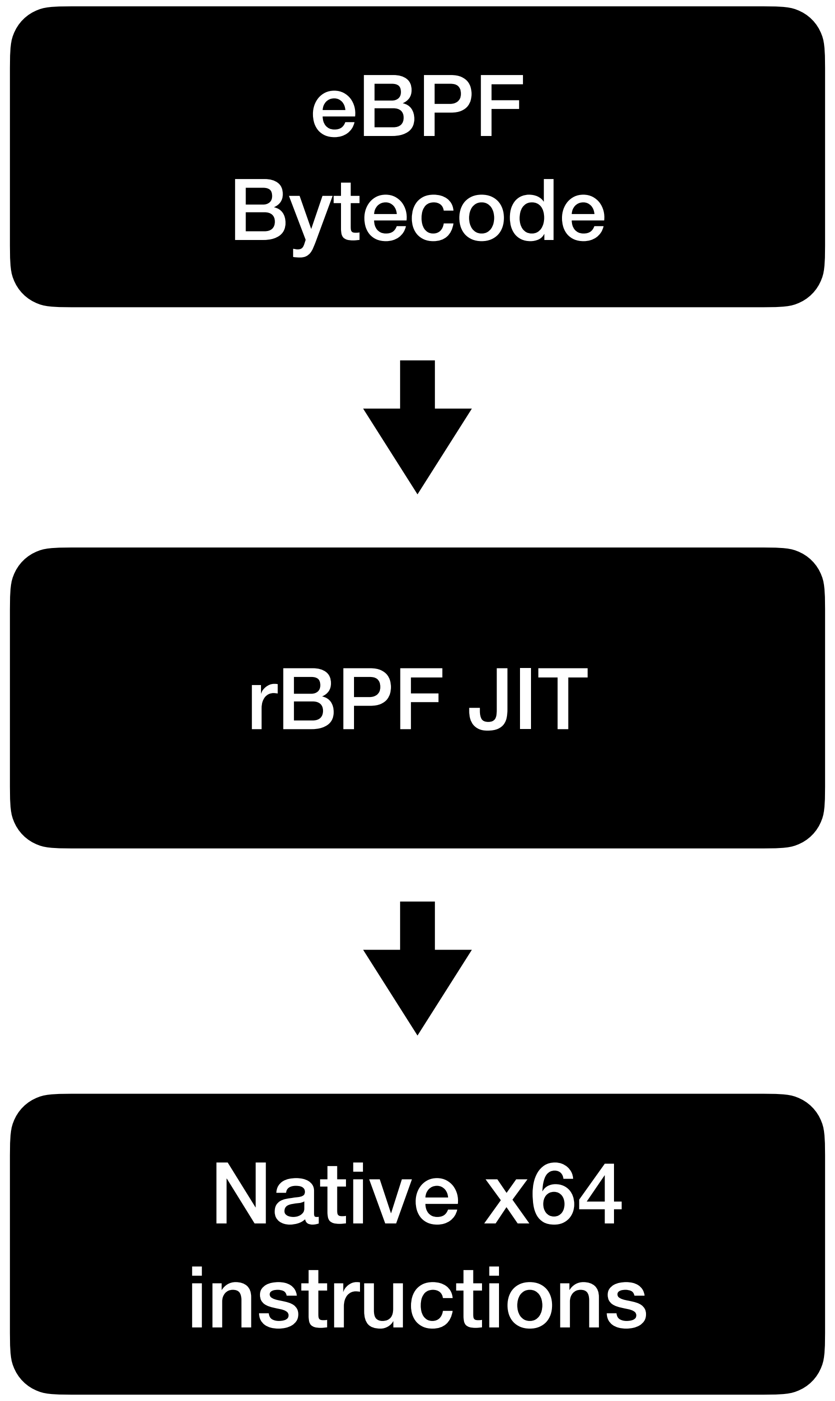
Triage

So many indirections...



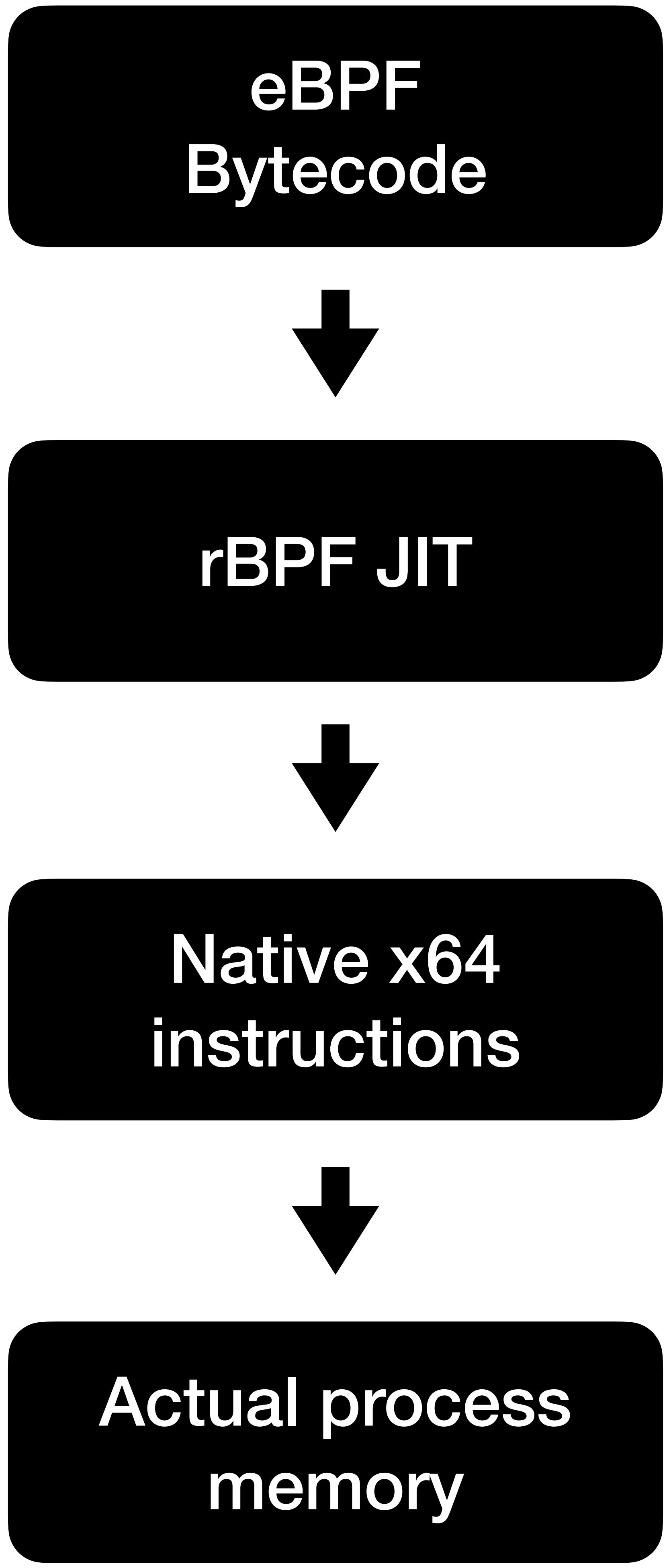
Triage

So many indirections...



Triage

So many indirections…



root@localhost: ~/solana_triage

```
root@localhost:~/solana_triage#
```

root@localhost: ~/solana_triage

```
(lldb)
```

```
(lldb) run segfaults/id:000140,sig:11,src:000773,time:96858858,op:arith16,pos:250,val:-15
There is a running process, kill it and restart?: [Y/n] y
Process 124037 exited with status = 9 (0x00000009)
Process 172729 launched: '/root/solana_triage/target/debug/solana_run_ebpf' (x86_64)
Process 172729 stopped
* thread #1, name = 'solana_run_ebpf', stop reason = signal SIGSEGV: address access protected (fault address: 0x7f43c03208f5)
    frame #0: 0x00007f43c03209bd
->  0x7f43c03209bd: movq   $0x1, (%r10)
    0x7f43c03209c5: addq   $0x1c, %r11
    0x7f43c03209cc: movq   %r11, 0x10(%r10)
    0x7f43c03209d0: jmp    0x7f43c0320b1b
(lldb) run segfaults/id:000140,sig:11,src:000773,time:96858858,op:arith16,pos:250,val:-15
There is a running process, kill it and restart?: [Y/n] y
Process 172729 exited with status = 9 (0x00000009)
Process 179590 launched: '/root/solana_triage/target/debug/solana_run_ebpf' (x86_64)
Process 179590 stopped
* thread #1, name = 'solana_run_ebpf', stop reason = signal SIGSEGV: address access protected (fault address: 0x7fa6678b88f5)
    frame #0: 0x00007fa6678b89bd
->  0x7fa6678b89bd: movq   $0x1, (%r10)
    0x7fa6678b89c5: addq   $0x1c, %r11
    0x7fa6678b89cc: movq   %r11, 0x10(%r10)
    0x7fa6678b89d0: jmp    0x7fa6678b8b1b
(lldb) run segfaults/id:000140,sig:11,src:000773,time:96858858,op:arith16,pos:250,val:-15
There is a running process, kill it and restart?: [Y/n] y
Process 179590 exited with status = 9 (0x00000009)
Process 187809 launched: '/root/solana_triage/target/debug/solana_run_ebpf' (x86_64)
Process 187809 stopped
* thread #1, name = 'solana_run_ebpf', stop reason = signal SIGSEGV: address access protected (fault address: 0x7ff5de1dd8f5)
    frame #0: 0x00007ff5de1dd9bd
->  0x7ff5de1dd9bd: movq   $0x1, (%r10)
    0x7ff5de1dd9c5: addq   $0x1c, %r11
    0x7ff5de1dd9cc: movq   %r11, 0x10(%r10)
    0x7ff5de1dd9d0: jmp    0x7ff5de1ddb1b
(lldb)
```

```
(lldb) run segfaults/id:000140,sig:11,src:000773,time:96858858,op:arith16,pos:250,val:-15
There is a running process, kill it and restart?: [Y/n] y
Process 124037 exited with status = 9 (0x00000009)
Process 172729 launched: '/root/solana_triage/target/debug/solana_run_ebpf' (x86_64)
Process 172729 stopped
* thread #1, name = 'solana_run_ebpf', stop reason = signal SIGSEGV: address access protected (fault address: 0x7f43c03208f5)
    frame #0: 0x00007f43c03209bd
->  0x7f43c03209bd: movq   $0x1, (%r10)
    0x7f43c03209c5: addq   $0x1c, %r11
    0x7f43c03209cc: movq   %r11, 0x10(%r10)
    0x7f43c03209d0: jmp    0x7f43c0320b1b
(lldb) run segfaults/id:000140,sig:11,src:000773,time:96858858,op:arith16,pos:250,val:-15
There is a running process, kill it and restart?: [Y/n] y
Process 172729 exited with status = 9 (0x00000009)
Process 179590 launched: '/root/solana_triage/target/debug/solana_run_ebpf' (x86_64)
Process 179590 stopped
* thread #1, name = 'solana_run_ebpf', stop reason = signal SIGSEGV: address access protected (fault address: 0x7fa6678b88f5)
    frame #0: 0x00007fa6678b89bd
->  0x7fa6678b89bd: movq   $0x1, (%r10)
    0x7fa6678b89c5: addq   $0x1c, %r11
    0x7fa6678b89cc: movq   %r11, 0x10(%r10)
    0x7fa6678b89d0: jmp    0x7fa6678b8b1b
(lldb) run segfaults/id:000140,sig:11,src:000773,time:96858858,op:arith16,pos:250,val:-15
There is a running process, kill it and restart?: [Y/n] y
Process 179590 exited with status = 9 (0x00000009)
Process 187809 launched: '/root/solana_triage/target/debug/solana_run_ebpf' (x86_64)
Process 187809 stopped
* thread #1, name = 'solana_run_ebpf', stop reason = signal SIGSEGV: address access protected (fault address: 0x7ff5de1dd8f5)
    frame #0: 0x00007ff5de1dd9bd
->  0x7ff5de1dd9bd: movq   $0x1, (%r10)
    0x7ff5de1dd9c5: addq   $0x1c, %r11
    0x7ff5de1dd9cc: movq   %r11, 0x10(%r10)
    0x7ff5de1dd9d0: jmp    0x7ff5de1ddb1b
(lldb)
```

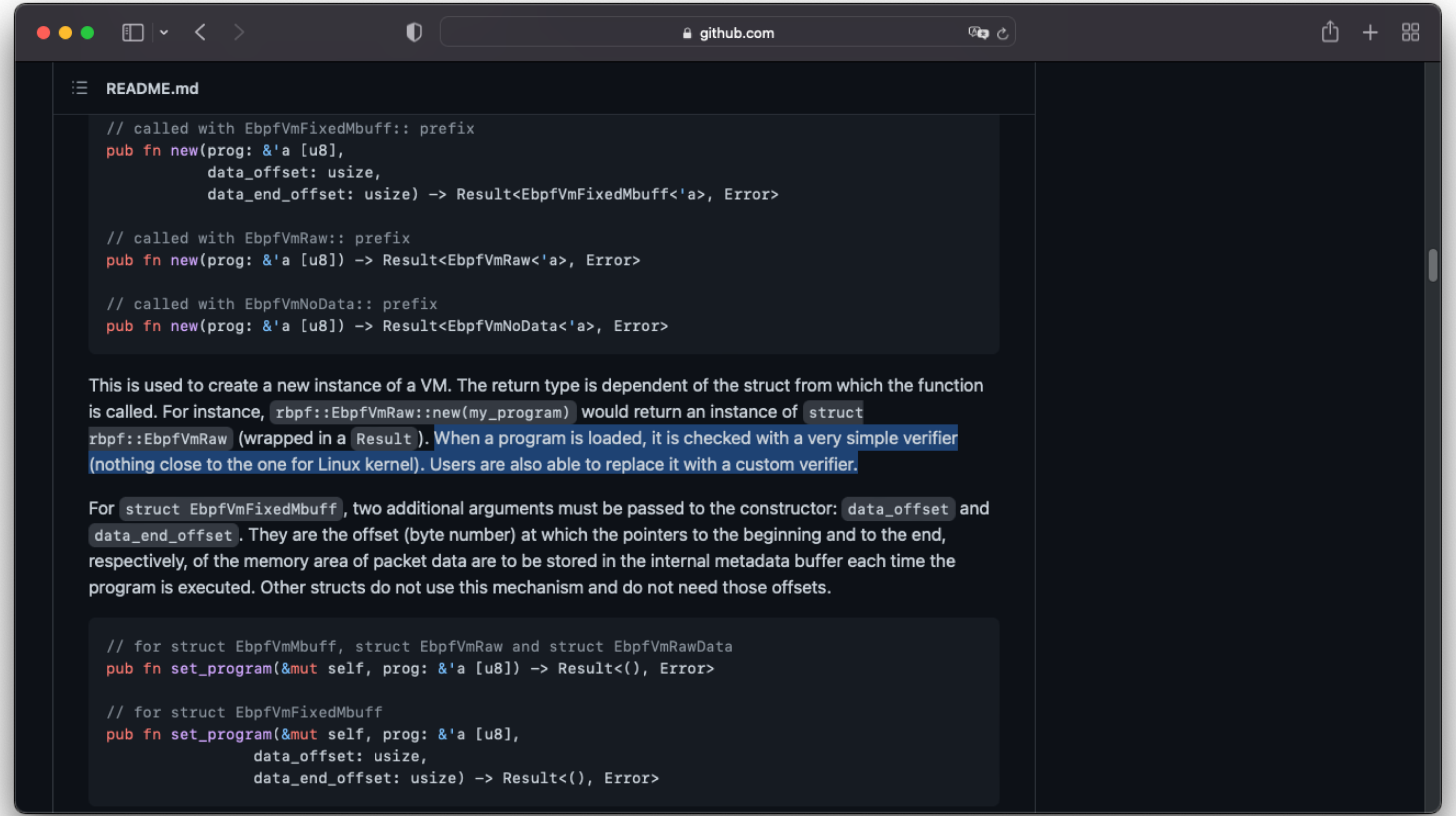
Triage

- Customized JIT to emit raw binary + map of eBPF instructions
- Spent hours analyzing

- Findings:
  - Multiple out-of-bound reads
  - Multiple out-of-bound writes

Affected in the wild?

The program verifier...




```rust
// called with EbpfVmFixedMbuff:: prefix
pub fn new(prog: &'a [u8],
           data_offset: usize,
           data_end_offset: usize) -> Result<EbpfVmFixedMbuff<'a>, Error>

// called with EbpfVmRaw:: prefix
pub fn new(prog: &'a [u8]) -> Result<EbpfVmRaw<'a>, Error>

// called with EbpfVmNoData:: prefix
pub fn new(prog: &'a [u8]) -> Result<EbpfVmNoData<'a>, Error>
```

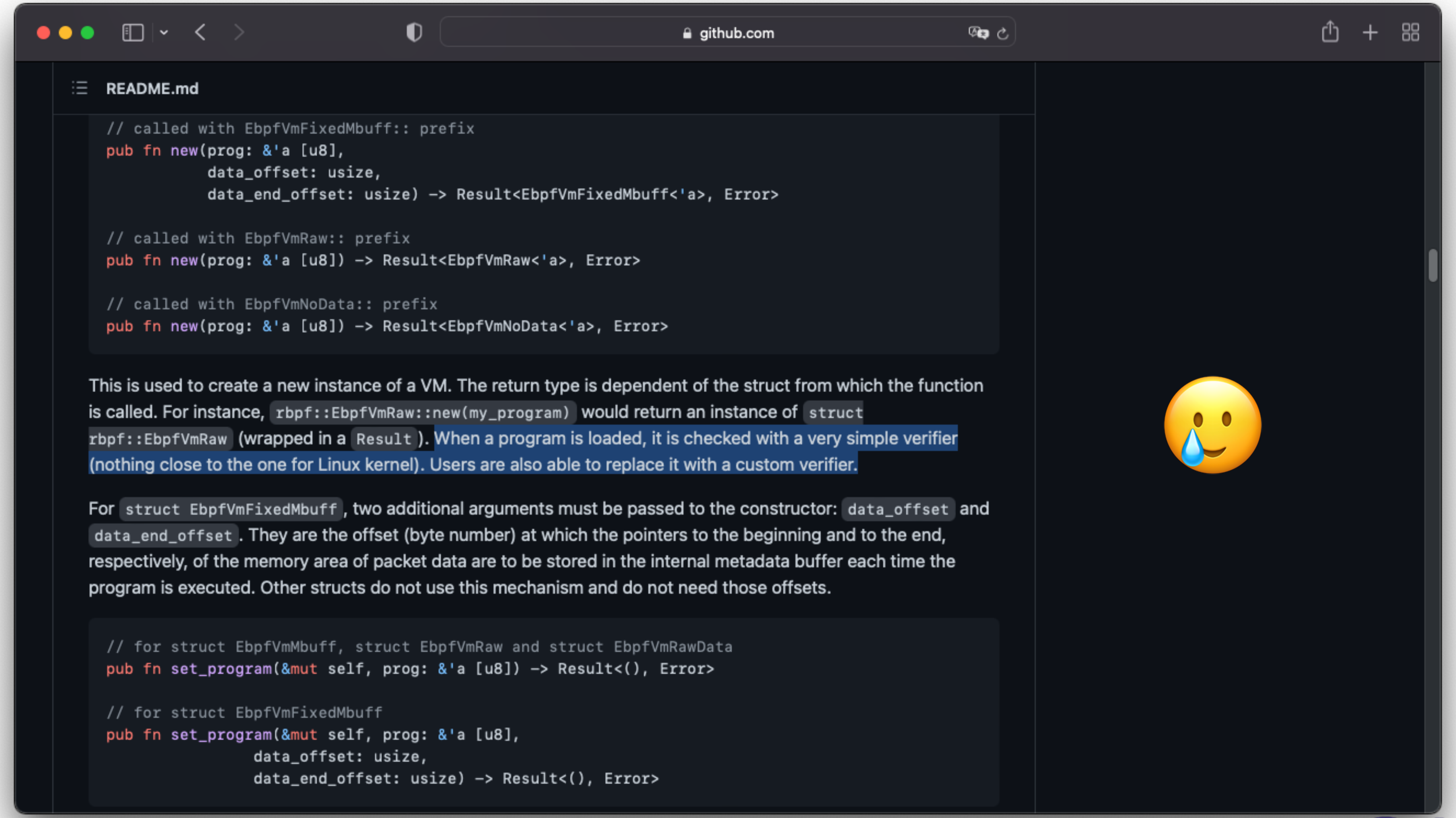
This is used to create a new instance of a VM. The return type is dependent of the struct from which the function is called. For instance, `rbpf::EbpfVmRaw::new(my_program)` would return an instance of `struct rbpf::EbpfVmRaw` (wrapped in a `Result`). When a program is loaded, it is checked with a very simple verifier (nothing close to the one for Linux kernel). Users are also able to replace it with a custom verifier.

For `struct EbpfVmFixedMbuff`, two additional arguments must be passed to the constructor: `data_offset` and `data_end_offset`. They are the offset (byte number) at which the pointers to the beginning and to the end, respectively, of the memory area of packet data are to be stored in the internal metadata buffer each time the program is executed. Other structs do not use this mechanism and do not need those offsets.

```rust
// for struct EbpfVmMbuff, struct EbpfVmRaw and struct EbpfVmRawData
pub fn set_program(&mut self, prog: &'a [u8]) -> Result<(), Error>

// for struct EbpfVmFixedMbuff
pub fn set_program(&mut self, prog: &'a [u8],
                   data_offset: usize,
                   data_end_offset: usize) -> Result<(), Error>
```

The program verifier...



github.com

README.md

```rust
// called with EbpfVmFixedMbuff:: prefix
pub fn new(prog: &'a [u8],
           data_offset: usize,
           data_end_offset: usize) -> Result<EbpfVmFixedMbuff<'a>, Error>

// called with EbpfVmRaw:: prefix
pub fn new(prog: &'a [u8]) -> Result<EbpfVmRaw<'a>, Error>

// called with EbpfVmNoData:: prefix
pub fn new(prog: &'a [u8]) -> Result<EbpfVmNoData<'a>, Error>
```

This is used to create a new instance of a VM. The return type is dependent of the struct from which the function is called. For instance, `rbpf::EbpfVmRaw::new(my_program)` would return an instance of `struct rbpf::EbpfVmRaw` (wrapped in a `Result`). When a program is loaded, it is checked with a very simple verifier (nothing close to the one for Linux kernel). Users are also able to replace it with a custom verifier.

For `struct EbpfVmFixedMbuff`, two additional arguments must be passed to the constructor: `data_offset` and `data_end_offset`. They are the offset (byte number) at which the pointers to the beginning and to the end, respectively, of the memory area of packet data are to be stored in the internal metadata buffer each time the program is executed. Other structs do not use this mechanism and do not need those offsets.

```rust
// for struct EbpfVmMbuff, struct EbpfVmRaw and struct EbpfVmRawData
pub fn set_program(&mut self, prog: &'a [u8]) -> Result<(), Error>

// for struct EbpfVmFixedMbuff
pub fn set_program(&mut self, prog: &'a [u8],
                   data_offset: usize,
                   data_end_offset: usize) -> Result<(), Error>
```

Disclosure...

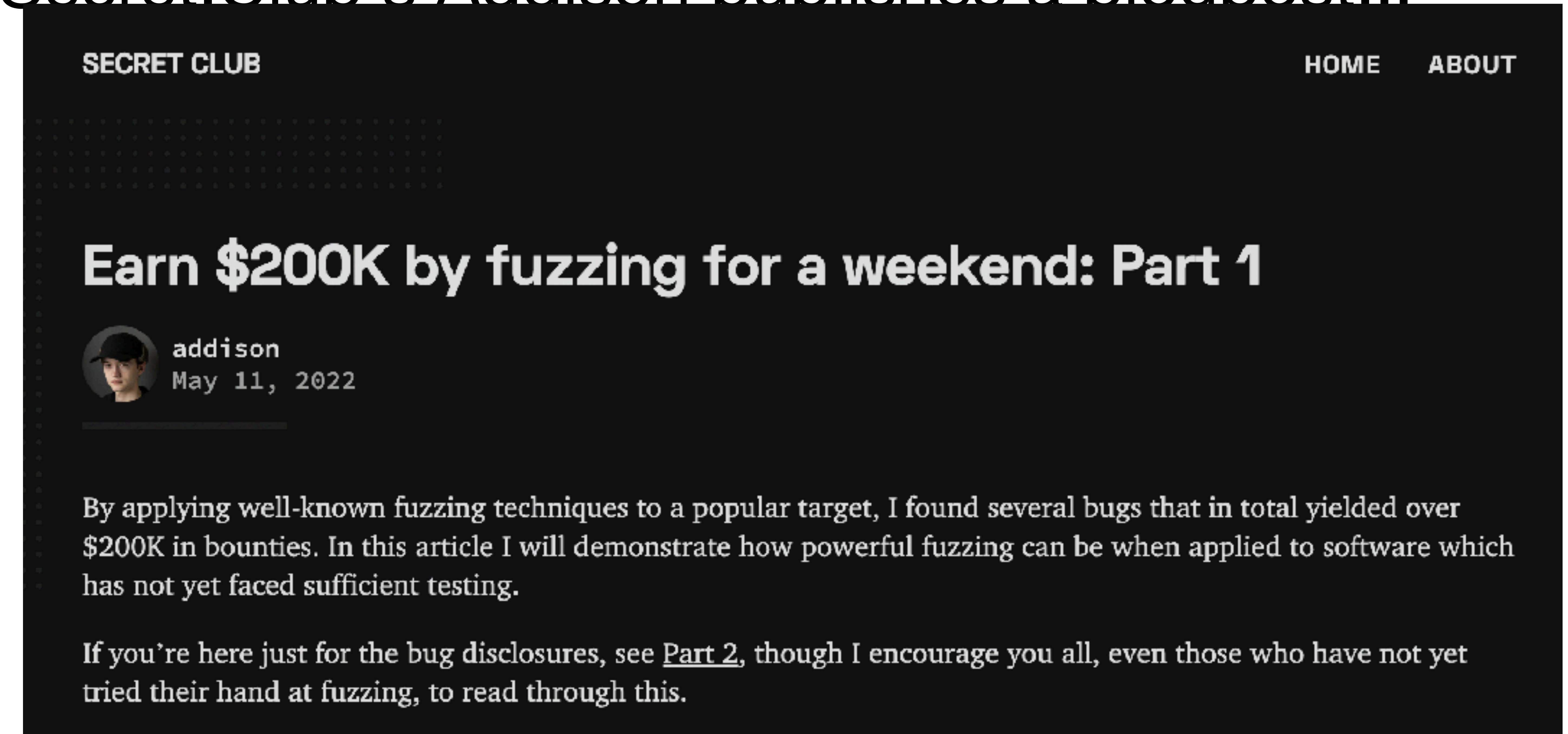
kraken

Timeline

- 04. Of April: Submitted this talk to $conference
- 01. Of May: Submitted this talk to DEF CON
- 11. Of May: Secret.Club's Addison publishes a blogpost...

Timeline

- 04. Of April: Submitted this talk to $conference
- 01. Of May: Submitted this talk to DEF CON
- 11. Of May: Secret.Club's Addison publishes a blogpost...



SECRET CLUB HOME ABOUT

Earn $200K by fuzzing for a weekend: Part 1

addison
May 11, 2022

By applying well-known fuzzing techniques to a popular target, I found several bugs that in total yielded over $200K in bounties. In this article I will demonstrate how powerful fuzzing can be when applied to software which has not yet faced sufficient testing.

If you're here just for the bug disclosures, see Part 2, though I encourage you all, even those who have not yet tried their hand at fuzzing, to read through this.

Bug collision

- Full bug collision with secret.club
- Patches completely patch any segmentation-fault we had
- Fuzzer only finds floating-point exceptions now

Takeaways

- JITs are fun and often pretty easy to fuzz
- Triaging JITs however is pretty difficult
- Performance sometimes impacts security
- Verify the full chain before getting to excited about a vulnerability
- Bug collisions happen

Thank you!